Lecture notes. Lecture 8

Forecasting models and learned laws of motion

Marcin Płodzień
Deep Learning in Classical and Quantum Physics
Institute of Theoretical Physics, Jagiellonian University in Kraków

1 The forecasting task

1.1 A record, a hidden state and a forecast map

A time series is a sequence of numbers $s_1, s_2, \ldots, s_N$ measured at equal time intervals Δt. Behind it stands a physical system whose state $\mathbf{x}(t) \in \mathbb{R}^d$, a list of d numbers, fixes its future through a law of motion

$$\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x}), \tag{1}$$

where the dot is the time derivative and the vector field $\mathbf{F}$ gives the velocity of the state at every point of the phase space $\mathbb{R}^d$. The flow map φ_t carries every initial state to the state reached after the time t, $\mathbf{x}(t) = \varphi_t(\mathbf{x}(0))$. The instrument records one number per step, $s_k = h(\mathbf{x}(k\Delta t)) + \eta_k$, with a measurement function h and measurement noise η_k. The record is therefore a projection of a motion in d dimensions onto one number per step. The state, the law and the measurement function are all unknown, and only the numbers s_k are available.

One measured value does not fix the state, since the same value occurs in states that move in different directions. A delay vector of m past values spaced by κ samples, $(s_k, s_{k-\kappa}, \ldots, s_{k-(m-1)\kappa})$, can fix it. The motion of a dissipative system settles on a set called the attractor, and the dimension of this set is often fractional, a fractal dimension, smaller than the number d of state variables. The embedding theorem of Takens, in the extension by Sauer, Yorke and Casdagli, states that the map from the state on the attractor to the delay vector is one-to-one for almost every choice of measurement if m exceeds twice the box-counting dimension D_B of the attractor [1, 2]. This is a sufficient condition. A smaller m often suffices in practice, and the method of false nearest neighbours, which counts pairs of neighbouring delay vectors that separate when a coordinate is added [15], estimates the smallest m for which the delay vectors no longer fold onto each other. When the delay vector fixes the state, the future measured value is a function of the delay vector. Let A be the attractor, an invariant set that the flow carries into itself, $\varphi_t(A) = A$, and let the delay map $\mathcal{E}\colon A \to \mathbb{R}^m$ assign to every state the delay vector that a noise-free record would produce, $\mathcal{E}(\mathbf{x}_k) = \big(h(\mathbf{x}_k), h(\mathbf{x}_{k-\kappa}), \ldots, h(\mathbf{x}_{k-(m-1)\kappa})\big)$. Where $\mathcal{E}$ is one-to-one, the forecast map for a horizon of $\tau \geq 1$ samples is

$$G_\tau\colon \mathcal{E}(A) \to \mathbb{R}, \qquad G_\tau := h \circ \varphi_{\tau\Delta t} \circ \mathcal{E}^{-1}, \qquad G_\tau(\mathcal{E}(\mathbf{x}_k)) = h(\mathbf{x}_{k+\tau}), \tag{2}$$

and for noise-free values $h(\mathbf{x}_{k+\tau}) = s_{k+\tau}$. The composition is read from right to left. The inverse delay map $\mathcal{E}^{-1}$ returns the state, the flow map $\varphi_{\tau\Delta t}$ advances it by τ samples, and the measurement function h gives the value recorded at that time. The map is defined only on the admissible delay vectors, the set $\mathcal{E}(A)$. A measured delay vector contains noise and lies slightly off this set, so a forecast from measured values needs an estimate of G_τ that extends to a neighbourhood of $\mathcal{E}(A)$. Where two states share one delay vector the inverse $\mathcal{E}^{-1}$ does not exist, and the composition (2) is undefined. The last relation of equation (2) still defines a function of the delay vector if all states that share a delay vector have the same future measured value. A detector that records an intensity, proportional to the square of a field amplitude, cannot distinguish a field from its negative. When the law of motion is unchanged under this change of sign, together with the variables coupled to the field, both

states have the same future intensity, and a forecast map of the intensity exists in this extended sense. The map exists and is unknown, so forecasting a deterministic system from one measured variable is a regression problem with a finite-dimensional input.

1.2 Window, horizon and three ways to forecast

The models of these notes read the window of the last L values of the record,

$$\mathbf{u}_k := (s_k,\, s_{k-1},\, \ldots,\, s_{k-L+1}) \in \mathbb{R}^L, \qquad \hat{s}_{k+\tau} = \hat{G}_\tau(\mathbf{u}_k), \tag{3}$$

where the hat marks an estimate and the step k is the forecast origin. A window of $L \geq (m-1)\kappa + 1$ consecutive values contains every entry of the delay vector, so it carries all the information about the present state that the delay vector carries, and the model $\hat{G}_\tau$ estimates the forecast map G_τ applied to the delay vector selected from the window. A forecast for many horizons can be produced in three ways (Figure 1). A one-step model predicts only the next value. A direct forecast fits a separate map $\hat{G}_\tau$ for every horizon, each fed with the measured window. A recursive forecast applies the one-step model repeatedly and appends each prediction to the window as if it were a measurement, so its errors are fed back. A recursive forecast continued over many steps from one origin is called a free run.

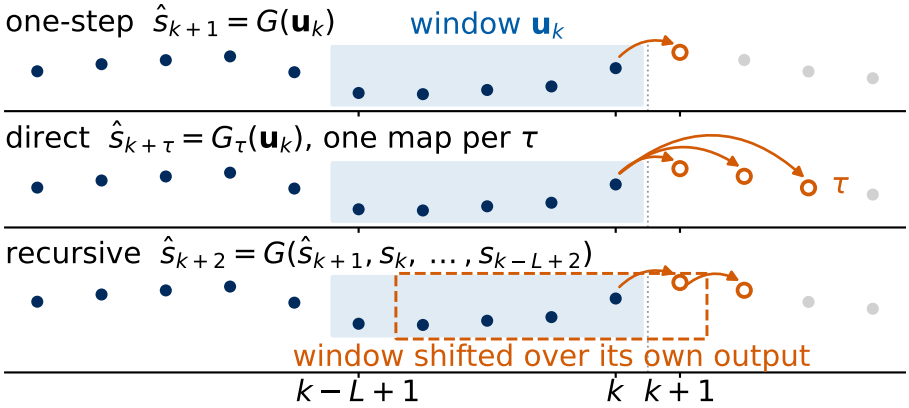


Figure 1: Three ways to forecast. Filled dots are samples of the record against the step index, the shaded box is the window $\mathbf{u}_k$ of the last L values, grey dots are future values, and open circles are forecasts. Top, one-step forecast. Middle, direct forecast with one map $\hat{G}_\tau$ for each horizon τ, written G in the drawing. Bottom, recursive forecast, in which the window (dashed) shifts over the model's own outputs.

1.3 Training, validation and test in time order

A model is fitted on one part of the record and judged on a subsequent part. Every record has 4000 samples, numbered from 0, and is divided into three consecutive blocks. The training block, samples 0 to 2499, is used to fit a model, the validation block, samples 2500 to 2999, to choose between variants of a model, such as the window length, and the test block, samples 3000 to 3999, once at the end to score the chosen model. A training pair must have its target inside the block used for the fit. The record is rescaled linearly so that its minimum and maximum over samples 0 to 2499 become 0 and 1, and validation and test values may lie slightly outside $[0, 1]$. The rescaling and every statistic that chooses a setting of a model, such as the delay and the dimension of the window, are computed on the training block alone. The windows are never shuffled across the blocks, since overlapping windows would then put test values into the training inputs.

1.4 Scoring a forecast

For two sets of n numbers a_i and b_i with means $\bar{a}$ and $\bar{b}$, the Pearson correlation coefficient is

$$r(a, b) := \frac{\sum_{i=1}^{n} (a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_{i=1}^{n} (a_i - \bar{a})^2 \; \sum_{i=1}^{n} (b_i - \bar{b})^2}}, \tag{4}$$

a number between -1 and 1 that measures how close the pairs lie to a straight line. A forecast at the horizon τ is scored on a set of test origins k, for which every target up to $\tau = 50$ lies in the test block, by the capacity and its sum over the horizons,

$$C(\tau) \coloneqq r\big(\hat{s}_{k+\tau}, s_{k+\tau}\big)^2, \qquad C_{\mathrm{tot}} \coloneqq \sum_{\tau=1}^{50} C(\tau) \le 50. \tag{5}$$

The capacity is the fraction of the variance of the targets that the best affine function of the forecasts accounts for. It ignores the offset, the scale and the sign of the forecast, so it is read together with the sign of r and a scatter plot of forecasts against targets [3]. A constant forecast is assigned $C = 0$. Persistence, $\hat{s}_{k+\tau} \coloneqq s_k$, needs no fitting and sets the floor that every model must exceed.

The pairs of persistence are the pairs $(s_k, s_{k+\tau})$ of the record, so its capacity is close to the square of the autocorrelation function (ACF) [4]. A record is called stationary when its statistical properties, such as the mean and the correlation of two values a fixed number of steps apart, do not change in time. For a stationary record with mean μ the ACF is

$$\rho(n) \coloneqq \frac{\big\langle (s_k - \mu)(s_{k+n} - \mu) \big\rangle}{\big\langle (s_k - \mu)^2 \big\rangle}, \tag{6}$$

the correlation of two values n steps apart, with the brackets denoting the average over the random process or, for one long record, over time. The numerator alone is the autocovariance $\Gamma(n)$. For white noise, a sequence of independent random numbers, $\rho(n) = 0$ for $n \ge 1$, and for the autoregressive process of order one, $s_k = \phi\, s_{k-1} + \varepsilon_k$ with $|\phi| < 1$ and white noise ε_k, it decays as $\rho(n) = \phi^n$.

1.5 Five records

The examples are five records of 4000 samples (Figure 2). The first is the coordinate x of the Lorenz-63 model of thermal convection [5],

$$\dot{x} = \sigma(y - x), \qquad \dot{y} = x(\rho - z) - y, \qquad \dot{z} = xy - \beta z, \tag{7}$$

with $\sigma = 10$, $\rho = 28$, $\beta = 8/3$, sampled every $\Delta t = 0.02$ time units. The letters σ, ρ and β denote these parameters only in equation (7), and $\rho(n)$ with an argument is the ACF. The next three are solutions of the Mackey–Glass delay equation for blood-cell production [6],

$$\dot{c}(t) = \frac{0.2\, c(t - \tau_{\mathrm{MG}})}{1 + c(t - \tau_{\mathrm{MG}})^{10}} - 0.1\, c(t), \tag{8}$$

sampled once per time unit, periodic for the delay $\tau_{\mathrm{MG}} = 15$ and irregular for $\tau_{\mathrm{MG}} = 25$ and 35. The fifth is a measured record, the intensity of a far-infrared ammonia laser in a regime of irregular pulsations [12], data set A of the Santa Fe time-series competition [13], recorded with 8 bits. Its power spectrum has a sharp peak at a pulsation period of 7.26 samples, and the amplitude of the pulsations grows over many periods and then collapses within a few samples.

Table 1 collects what the records tell about the forecasting task. The largest Lyapunov exponent λ_1 is the average rate at which two nearby states separate, $|\boldsymbol{\delta}(t)| \approx |\boldsymbol{\delta}(0)|\, e^{\lambda_1 t}$, and its inverse, the Lyapunov time $T_\lambda = 1/\lambda_1$, sets the natural unit of a horizon. For an irregular deterministic record ($\lambda_1 > 0$, chaos) even the exact law forecasts a trajectory only for a few Lyapunov times, since an initial error grows exponentially. A system in d dimensions has d Lyapunov exponents $\lambda_1 \ge \lambda_2 \ge \cdots \ge \lambda_d$, the growth rates of small errors in different directions. The Kaplan–Yorke dimension [54]

$$D_{\mathrm{KY}} \coloneqq j + \frac{\lambda_1 + \cdots + \lambda_j}{|\lambda_{j+1}|}, \tag{9}$$

where j is the largest index for which $\lambda_1 + \cdots + \lambda_j \ge 0$, estimates the fractal dimension of the attractor from these rates. For Lorenz-63 the exponents 0.907, 0.0001 and -14.573 per time unit give

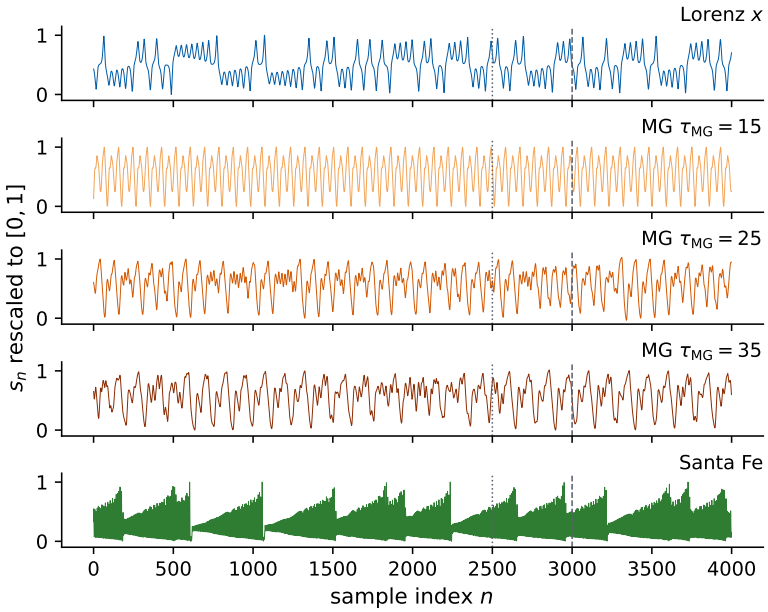


Figure 2: The five records, each rescaled to $[0, 1]$ and plotted against the sample index n. From top, the x coordinate of Lorenz-63, the Mackey–Glass solution at $\tau_{\mathrm{MG}} = 15$, 25 and 35, and the Santa Fe laser intensity. The dotted line marks the start of the validation block (sample 2500) and the dashed line the start of the test block (sample 3000).

$D_{\mathrm{KY}} = 2 + 0.907/14.573 \approx 2.06$. The delay κ and the dimension m of the table are computed on the training block alone, samples 0 to 2499, since they give the window $L = (m-1)\kappa + 1$ that we use as a first guess of the window length. The exponents and the dimension describe the records and choose no model setting, and they use all 4000 samples. The values of m are those of false nearest neighbours, and for every record they lie at or below the sufficient bound $2D_B$ of the embedding theorem, here estimated with D_{KY} in place of D_B, so the sufficient condition is not met and the choice rests on the false-neighbour test.

record	κ	m	$L = (m-1)\kappa + 1$	D_{KY}	λ_1 per sample	T_λ
Lorenz x	10	4	31	2.06	0.0181	55
Mackey–Glass, $\tau_{\mathrm{MG}} = 15$	8	2	9	1.00	≈ 0	∞
Mackey–Glass, $\tau_{\mathrm{MG}} = 25$	15	4	46	3.16	0.0099	101
Mackey–Glass, $\tau_{\mathrm{MG}} = 35$	21	4	64	3.41	0.0029	340
Santa Fe laser	2	3	5	2.05 / 2.16	0.029	34

Table 1: Characteristics of the five records. The delay κ comes from the first minimum of the mutual information of s_k and $s_{k-\kappa}$, a measure of their statistical dependence [14], the dimension m from the method of false nearest neighbours, and λ_1 and D_{KY} from the equations for the model records and from the record itself for the laser, where λ_1 is the estimate of Rosenstein and coauthors [7] and D_{KY} is given for $m = 3$ and $m = 4$. The delay κ, the window L and the Lyapunov time T_λ are in samples, λ_1 is per sample.

For the Lorenz record the longest horizon of 50 samples is about one Lyapunov time, for the chaotic Mackey–Glass records it lies well within one Lyapunov time, the periodic record has none, and the laser has the shortest Lyapunov time, so the skill of every forecast of the laser decays over the 50 horizons. The task of every model is to produce, from a window of past values, forecasts whose capacity on the test block exceeds that of persistence, at horizons within the limit that chaos sets for the record.

2 Linear models

2.1 The autoregressive model

The forecast map G_τ is unknown, and the simplest model of it is a linear function of the window. The autoregressive model of order p, AR(p) [16], writes the next value as a weighted sum of the last p values plus a random kick,

$$s_k = \phi_0 + \phi_1 s_{k-1} + \phi_2 s_{k-2} + \cdots + \phi_p s_{k-p} + \varepsilon_k, \tag{10}$$

where ϕ_0 is a constant, $\phi_1, \ldots, \phi_p$ are the AR coefficients and ε_k is white noise with variance σ_ε^2, called the innovation. The autoregressive process of order one, whose ACF ϕ^n was given with the scores, is the case $p = 1$, $\phi_0 = 0$. For a deterministic record the innovation stands for everything that a linear function of the last p values cannot describe. The one-step forecast drops the unknown future innovation, $\hat{s}_{k+1} = \phi_0 + \sum_{i=1}^{p} \phi_i s_{k+1-i}$.

The AR forecast is an instance of a general structure. The past of the record is first turned into a vector of M numbers, the features, and the forecast is a linear combination of the features,

$$(s_1, \ldots, s_k) \xrightarrow{\Phi} \boldsymbol{\varphi}_k \in \mathbb{R}^M \xrightarrow{\text{linear}} \hat{y}_k = \mathbf{w}^\top \boldsymbol{\varphi}_k, \tag{11}$$

where Φ is the feature map, $\boldsymbol{\varphi}_k$ the feature vector at the step k, written in bold to distinguish it from the flow map φ_t, $\mathbf{w} \in \mathbb{R}^M$ the readout weights and $\hat{y}_k$ the forecast of a target, here s_{k+1}. For the AR model the features are the constant 1 and the last p values, $\boldsymbol{\varphi}_k = (1, s_k, \ldots, s_{k-p+1})$, and the readout weights are the coefficients, $\mathbf{w} = (\phi_0, \phi_1, \ldots, \phi_p)$. At every step the feature vector is one point in $\mathbb{R}^M$, and as the record advances these points trace a path in the feature space, which the readout projects onto the direction $\mathbf{w}$ (Figure 3). Since the feature map is fixed and only $\mathbf{w}$ is fitted, the fit is a linear least-squares problem.

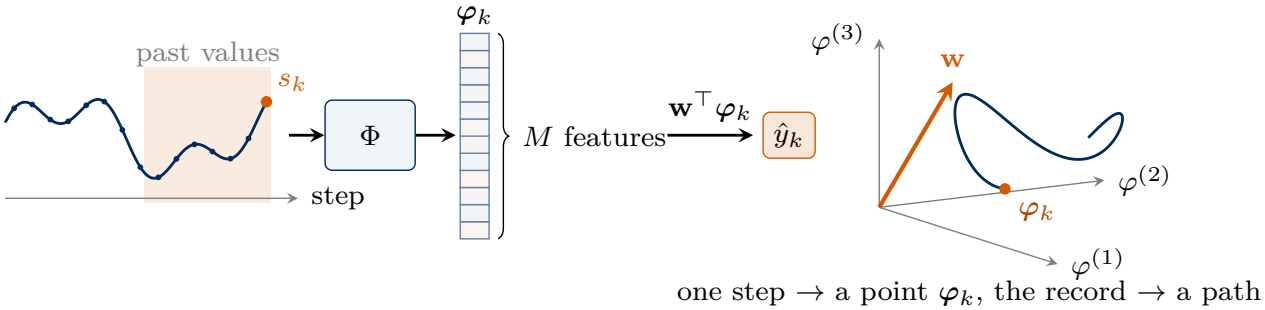


Figure 3: Features and a linear readout. The feature map Φ turns the past values of the record (left, the shaded part is the part it reads) into a vector $\boldsymbol{\varphi}_k$ of M features (middle, shades stand for feature values), and the forecast is the projection $\hat{y}_k = \mathbf{w}^\top \boldsymbol{\varphi}_k$. Right, each step gives one point in the feature space, here drawn with three features $\varphi^{(1)}, \varphi^{(2)}, \varphi^{(3)}$, and the record traces a path in it. For the AR model the features are the last p values and a constant.

2.2 Least squares and the autocovariance

The coefficients are found by least squares. For every target time t of the training block the row $(1, s_{t-1}, \ldots, s_{t-p})$ enters a matrix X and the value s_t a vector $\mathbf{y}$, and the coefficients $\boldsymbol{\phi} = (\phi_0, \phi_1, \ldots, \phi_p)$ minimise the squared error,

$$\hat{\boldsymbol{\phi}} = \arg\min_{\boldsymbol{\phi}} \; \|\mathbf{y} - X\boldsymbol{\phi}\|^2, \qquad X^\top X \, \hat{\boldsymbol{\phi}} = X^\top \mathbf{y}. \tag{12}$$

The second equation, the normal equations, follows by setting the gradient of the squared error to zero. The whole fit is one linear solve, and it has a unique solution whenever the columns of X are linearly independent. Minimising a mean squared error makes the forecast the best approximation,

within the model class, of the conditional mean of the target given the inputs, the average of all futures compatible with the window. For a linear model this is the best linear predictor, which equals the conditional mean only when that mean is a linear function of the window.

The normal equations show what a linear model can learn from a record. For a stationary record with zero mean and n rows, the entries of $X^\top X/n$ are averages of products $s_{t-i}s_{t-j}$ and approach the autocovariance $\Gamma(i-j)$, and the entries of $X^\top \mathbf{y}/n$ approach $\Gamma(i)$. The normal equations become the Yule–Walker equations [55, 56],

$$\sum_{j=1}^{p} \phi_j \, \Gamma(i-j) = \Gamma(i), \qquad i = 1, \ldots, p, \tag{13}$$

which say that the fitted coefficients depend on the record only through its autocovariance. Two records with the same ACF, for example a chaotic signal and a linear random process with the same power spectrum, give the same linear model. Whatever the record contains beyond its second-order statistics is invisible to it.

The order p has to be chosen. A larger p always lowers the error on the training block, and beyond some order the extra coefficients fit the noise of this particular block. One remedy is to compare the one-step capacity of models of different order on the validation block and keep the best. Another is the Akaike information criterion [17], which adds a penalty of two for every fitted parameter to the misfit, $\mathrm{AIC} = n \ln \hat{\sigma}_\varepsilon^2 + 2n_{\mathrm{par}}$ up to a constant, where $\hat{\sigma}_\varepsilon^2$ is the variance of the n residuals and n_{par} the number of parameters. It assumes residuals that are independent Gaussian noise. The residuals of a deterministic signal are the part of the dynamics that a linear map cannot represent, so for such records the AIC order is a convention. On the five records the AIC prefers orders between 37 and 40, at the end of the scanned range.

2.3 The iterated AR model

Iterated without innovations, equation (10) is a linear recursion. Its solutions are found by inserting the trial form $s_k = z^{-k}$ into the recursion without the constant, which gives $z^{-k} = \sum_i \phi_i z^{-k+i}$, or after multiplying by z^k,

$$Q_p(z) \coloneqq 1 - \phi_1 z - \phi_2 z^2 - \cdots - \phi_p z^p = 0. \tag{14}$$

Each root $z_j = |z_j| e^{\mathrm{i}\omega_j}$ of this polynomial gives a solution z_j^{-k}, and a pair of complex conjugate roots gives the real mode $|z_j|^{-k}\cos(\omega_j k)$, an oscillation with the frequency ω_j per sample that decays when $|z_j| > 1$ and grows when $|z_j| < 1$. A model with all roots outside the unit circle is stationary, and every recursive forecast then relaxes to the mean of the process, $\mu = \phi_0/(1 - \sum_i \phi_i)$. A linear map can therefore produce decaying, growing or, with roots on the unit circle, periodic and quasi-periodic motion. An irregular motion that stays bounded without repeating itself, as on a chaotic attractor, is beyond any linear model and requires a nonlinear one. For the five records the models of the AIC orders are stationary, with the smallest root moduli just outside the unit circle, between 1.004 (Mackey–Glass at $\tau_{\mathrm{MG}} = 15$) and 1.034 (Lorenz), so their modes decay slowly.

2.4 Moving averages and their combination

A second family builds the signal from the innovations themselves. The moving-average model of order q, MA(q),

$$s_k = \mu + \varepsilon_k + \theta_1 \varepsilon_{k-1} + \cdots + \theta_q \varepsilon_{k-q}, \tag{15}$$

describes a signal whose memory is a finite sum of recent random kicks, weighted by the MA coefficients $\theta_1, \ldots, \theta_q$, a meaning that θ_i with an index has only here. Its autocorrelation follows from the definition. With $\theta_0 \coloneqq 1$ and independent innovations, the product $(s_k - \mu)(s_{k+n} - \mu)$ has a nonzero average only from the pairs of terms that share an innovation, so

$$\rho(n) = \frac{\sum_{i=0}^{q-n} \theta_i \theta_{i+n}}{\sum_{i=0}^{q} \theta_i^2} \quad \text{for } n \le q, \qquad \rho(n) = 0 \quad \text{for } n > q. \tag{16}$$

For MA(2) with $\boldsymbol{\theta} = (0.8, 0.4)$ the denominator is $1+0.64+0.16 = 1.8$, and $\rho(1) = (0.8+0.8\cdot0.4)/1.8 = 0.622$ and $\rho(2) = 0.4/1.8 = 0.222$, with zero beyond. The ACF of an AR process instead decays gradually, as the AR(1) result $\rho(n) = \phi^n$ showed. This contrast, a cut-off for MA and a gradual decay for AR, is the classical way to recognise the two kinds of process in data (Figure 4).

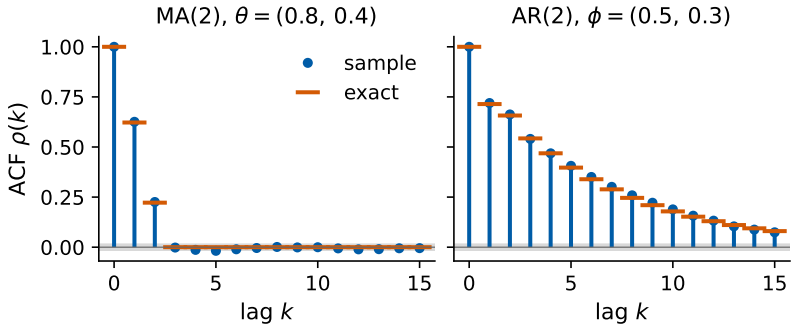


Figure 4: Autocorrelation ρ against the lag n (labelled k in the drawing) of two simulated processes. Left, MA(2) with $\boldsymbol{\theta} = (0.8, 0.4)$, whose ACF vanishes beyond lag 2. Right, AR(2) with $\boldsymbol{\phi} = (0.5, 0.3)$, whose ACF decays gradually. Bars, estimates from a simulated record. Dots, exact values.

The innovations of an MA model are not observed, so least squares on past values cannot be used to fit it. The parameters are estimated by maximising the likelihood, and the unknown innovations are reconstructed one by one from the measured values by a recursive estimator, the Kalman filter, which at every step subtracts the forecast from the new value and keeps the difference as the estimate of the current innovation. The same structure fixes the forecast. A value τ steps ahead contains the innovations $\varepsilon_{k+\tau}, \ldots, \varepsilon_{k+\tau-q}$, which all lie in the future when $\tau > q$. Their expected value is zero, so the MA forecast equals the mean μ beyond q steps, and its capacity drops to zero there (Figure 5). Selected by the AIC from $q \in \{2, 4, 8\}$, the MA model takes $q = 8$ on the laser and on the three Mackey–Glass records.

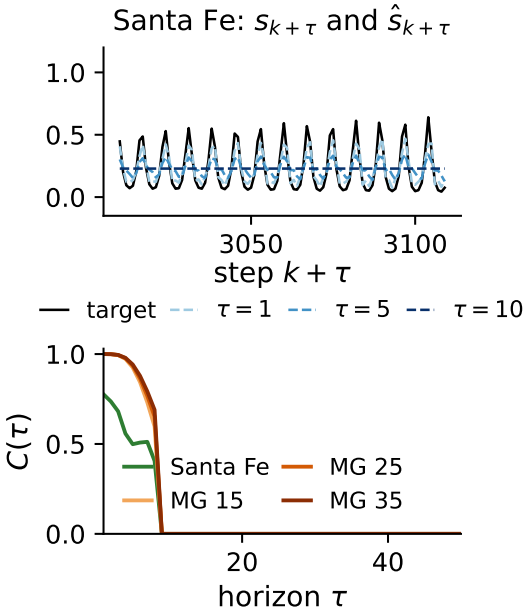


Figure 5: Forecasts of the MA(8) model. Top, the Santa Fe target $s_{k+\tau}$ and the forecasts at the horizons $\tau = 1$, 5 and 10 against the step $k+\tau$. At $\tau = 10$ the forecast is the constant mean. Bottom, the capacity $C(\tau)$ on the test block for the laser and the three Mackey–Glass records, which falls to zero beyond $\tau = 8$.

Combining both kinds of terms gives the ARMA(p, q) model, which can describe a signal with a few parameters where a pure AR model needs many. A record with a slow trend is often first differenced, $s_k \to s_k - s_{k-1}$, and the ARIMA(p, d, q) model, the autoregressive integrated moving-average model, applies an ARMA(p, q) model to the record differenced d times. Without a constant its forecasts

of the differences relax to zero, so its forecast of the record itself settles at a level set by the last measured values. Selected by the AIC, the ARMA models of the records have the orders $(p, q) = (8, 4)$, and $(12, 4)$ for the laser, and the ARIMA models the same orders with $d = 1$. MA terms describe correlated noise, and the records are deterministic or nearly so. Fitted to the records, the ARMA and ARIMA models forecast no better than AR models of similar order, and one of their likelihood fits failed to converge, so we continue with the AR model.

2.5 Direct and recursive forecasts

An AR model forecasts many steps ahead in the recursive or the direct way of Figure 1. The recursive forecast iterates the one-step model. The direct forecast fits, for each horizon, its own linear map from the window to $s_{k+\tau}$,

$$\hat{s}_{k+\tau} = \mathbf{w}_\tau^\top \mathbf{u}_k + b_\tau, \tag{17}$$

with $\mathbf{u}_k$ the window of equation (3), the weights $\mathbf{w}_\tau \in \mathbb{R}^L$ and the offset b_τ. Neighbouring entries of the window are nearly equal for a smooth record, so the columns of the data matrix are nearly parallel and the least-squares problem is badly conditioned. Ridge regression [18] adds a penalty on the size of the weights to the squared error,

$$(\mathbf{w}_\tau, b_\tau) = \arg\min_{\mathbf{w}, b} \sum_k \left(s_{k+\tau} - \mathbf{w}^\top \mathbf{u}_k - b\right)^2 + \beta \|\mathbf{w}\|^2, \tag{18}$$

where $\beta \geq 0$ sets the strength of the penalty and the offset b is left unpenalised. The solution is again one linear solve, $\mathbf{w}_\tau = (X_c^\top X_c + \beta I)^{-1} X_c^\top \mathbf{y}_c$ and $b_\tau = \bar{y} - \mathbf{w}_\tau^\top \bar{\mathbf{u}}$, where the rows of X_c are the windows and $\mathbf{y}_c$ the targets, both with their means $\bar{\mathbf{u}}$ and $\bar{y}$ subtracted, and all 50 horizons share the same matrix. With $\sigma_{\max}$ and $\sigma_{\min}$ the largest and smallest singular values of X_c, the condition number of the matrix to be inverted is $(\sigma_{\max}^2 + \beta)/(\sigma_{\min}^2 + \beta)$. It measures how strongly errors in the data are amplified in the weights, and the penalty bounds it even when $\sigma_{\min}$ is nearly zero. The window length and the penalty are chosen on the validation block. The chosen model is then refitted on the training and validation blocks together and scored on the 950 test origins $k = 3000, \ldots, 3949$.

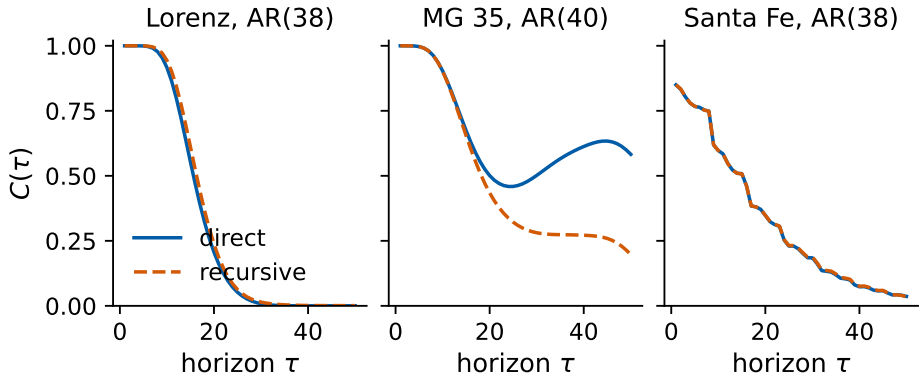


Figure 6: Capacity $C(\tau)$ on the test block against the horizon τ of the direct (solid) and the recursive (dashed) AR forecast, both reading the window of the AIC order p, for Lorenz x with $p = 38$, Mackey–Glass at $\tau_{\mathrm{MG}} = 35$ with $p = 40$ and the Santa Fe laser with $p = 38$.

Figure 6 compares the two forecasts made from the same window of the AIC order. On the chaotic Mackey–Glass record at $\tau_{\mathrm{MG}} = 35$ the direct forecast reaches $C_{\mathrm{tot}} = 33.0$ against 24.6 for the recursive one, because each direct map is the best linear predictor of its own horizon, while the recursive forecast relaxes towards the mean as its modes decay. On Lorenz and on the laser both forecasts lose the signal within about twenty steps and differ little. With the window length and the penalty chosen on the validation block, $L = 12$ for Lorenz and $L = 64$ for the other records, the direct AR forecast reaches $C_{\mathrm{tot}} = 15.3$ on Lorenz, 49.8, 32.6 and 38.7 on the three Mackey–Glass records and 16.8 on the laser. Persistence reaches 8.4, 19.5, 13.3, 11.8 and 5.5 on the same records, so the linear model lies above persistence on every record.

2.6 Linear forecasts and the laser return map

The one-step forecasts show the difference between the records (Figure 7). On the Mackey–Glass records the AR forecast one step ahead reaches $C = 1.0000$ to four digits, since a smooth signal sampled once per time unit changes little in one step and a linear extrapolation of a few past values follows it. On the laser the forecast reaches only $C = 0.85$, and its points bend away from the diagonal at high intensities.

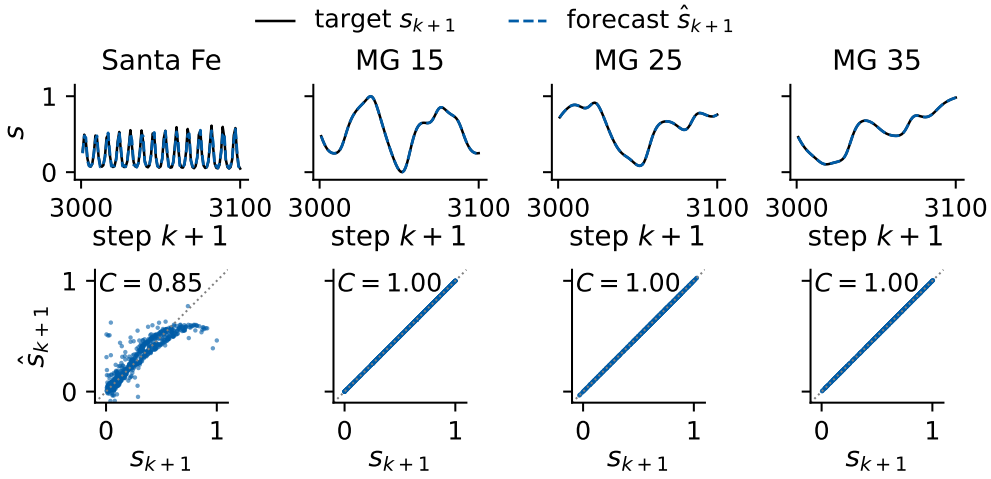


Figure 7: One-step AR forecasts on the test block. Top, the target s_{k+1} (solid) and the forecast $\hat{s}_{k+1}$ (dashed) against the step for the first 100 test targets of the Santa Fe laser and of Mackey–Glass at the three delays. Bottom, the forecast against the target for all 950 test targets, with the capacity C.

The reason is visible in the return map of the laser, Figure 8, which plots each value s_{k+1} against the previous one s_k. The same value s_k is followed by very different values s_{k+1}, depending on whether the intensity is rising or falling, the ambiguity of one measured value that does not fix the state. The points form a folded cloud, a surface that runs upwards along the rising branch, turns over at high intensities and comes back along the falling branch, so that a vertical line meets it twice. The window of past values resolves the ambiguity, since the rising and falling branches differ in the earlier values. The one-step forecast is a function on the L-dimensional space of windows, and its graph, the set of points $(\mathbf{u}_k, s_{k+1})$, is a surface in $L + 1$ dimensions, of which the return map shows the projection onto two of them. A linear function of the window has a flat hyperplane as its graph, and a hyperplane cannot follow a surface that turns over. Forecasting the laser well requires a function of the window that can bend.

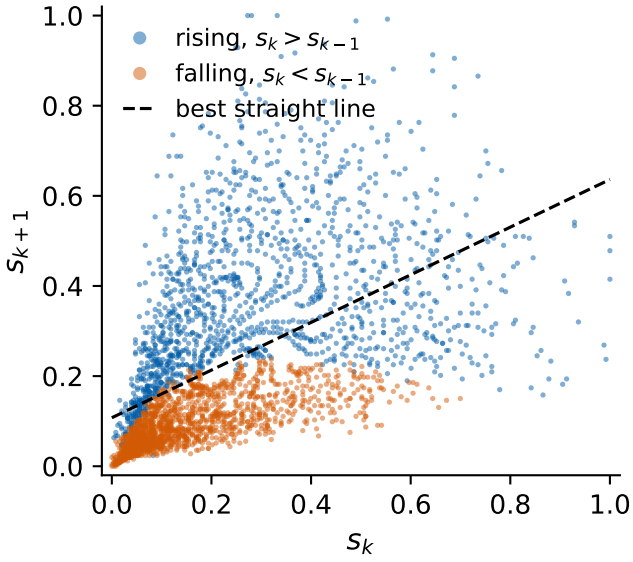


Figure 8: Return map of the Santa Fe training block, each value s_{k+1} against the previous one s_k. Blue points are rising steps, $s_k > s_{k-1}$, orange points falling ones. The dashed line is the best straight line $s_{k+1} = as_k + b$.

3 Lifting and the Koopman operator

3.1 A linear map for a nonlinear flow

The return map of the laser showed that a linear function of past values cannot follow a folded dynamics. A linear map has two advantages that are worth keeping. It is fitted by one linear solve, and its behaviour under iteration is set by the eigenvalues of one matrix, which decay, grow or rotate. The flow map φ_t of a nonlinear law (1) has neither property, since the image of a sum of two states is not the sum of their images. There is a way to keep the linear map and change what it acts on. Instead of the state itself, the map advances functions of the state, chosen so that their values at the next step are linear functions of their values now. The idea goes back to Koopman [19].

3.2 Observables and the Koopman operator

An observable is any function $g\colon \mathbb{R}^d \to \mathbb{C}$ of the state, such as one coordinate, the energy, or the quantity a detector records. The Koopman operator $\mathcal{K}_t$ acts on observables by following the flow,

$$(\mathcal{K}_t g)(\mathbf{x}) \coloneqq g(\varphi_t(\mathbf{x})), \tag{19}$$

so $\mathcal{K}_t g$ is the observable whose value now is the value of g after the time t. The operator composes every observable with the flow map, $\mathcal{K}_t g = g \circ \varphi_t$. Composition acts on the values of the functions point by point, so for any numbers c_1 and c_2

$$\mathcal{K}_t(c_1 g_1 + c_2 g_2) = (c_1 g_1 + c_2 g_2) \circ \varphi_t = c_1\, \mathcal{K}_t g_1 + c_2\, \mathcal{K}_t g_2, \tag{20}$$

and $\mathcal{K}_t$ is linear for every flow, linear or not. The flow map obeys $\varphi_0 = \mathrm{id}$ and $\varphi_{t+s} = \varphi_t \circ \varphi_s$, since moving a state first by s and then by t is the same as moving it by $t + s$. Applied to an observable this gives $\mathcal{K}_t \mathcal{K}_s g = (g \circ \varphi_t) \circ \varphi_s = g \circ \varphi_{t+s}$, so that

$$\mathcal{K}_0 = \mathrm{id}, \qquad \mathcal{K}_{t+s} = \mathcal{K}_t\, \mathcal{K}_s, \qquad g(\mathbf{x}_{k+n}) = (\mathcal{K}^n g)(\mathbf{x}_k), \quad \mathcal{K} \coloneqq \mathcal{K}_{\Delta t}, \tag{21}$$

where the last relation holds for states sampled with the interval Δt, $\mathbf{x}_k = \mathbf{x}(k\Delta t)$. Advancing an observable by n samples is the n-th power of one linear operator. The nonlinearity has moved into the space on which the operator acts, which is a space of functions and has infinitely many dimensions. As an analogy, the construction resembles the Heisenberg picture of quantum mechanics, in which the observables evolve and the state stays fixed.

Differentiating equation (19) at $t = 0$ with the chain rule gives the generator of the operator,

$$\mathcal{A}g \coloneqq \frac{d}{dt}\Big|_{t=0} g(\varphi_t(\mathbf{x})) = \mathbf{F}(\mathbf{x}) \cdot \nabla g(\mathbf{x}), \qquad \mathcal{K}_t = e^{t\mathcal{A}}, \tag{22}$$

the rate of change of g along the flow. The exponential form follows from the composition rule (21), since $\frac{d}{dt}\mathcal{K}_t g = \mathcal{K}_t \mathcal{A} g$. The generator is a first-order differential operator, and it is linear in g for any vector field $\mathbf{F}$.

3.3 Eigenfunctions and eigenvalues

A linear operator is understood through its eigenfunctions. An observable ξ with

$$\mathcal{A}\xi = \Lambda\,\xi \quad \Longleftrightarrow \quad \mathcal{K}_t\xi = e^{\Lambda t}\xi, \qquad \xi(\mathbf{x}(t)) = e^{\Lambda t}\,\xi(\mathbf{x}(0)), \tag{23}$$

is a Koopman eigenfunction with the eigenvalue Λ, a complex number. Along every trajectory its value changes by one exponential factor. The real part of Λ is a growth rate and the imaginary part an angular frequency, so an eigenfunction grows, decays or rotates in the complex plane at a fixed rate, whatever the state does. For data sampled with the interval Δt the same eigenfunction is multiplied at every step by one number,

$$\xi(\mathbf{x}_{k+1}) = \nu\,\xi(\mathbf{x}_k), \qquad \nu \coloneqq e^{\Lambda\Delta t}, \qquad \Lambda = \frac{\ln \nu}{\Delta t}, \tag{24}$$

so $|\nu| < 1$ means decay, $|\nu| > 1$ growth, and $|\nu| = 1$ an undamped oscillation with the angular frequency $\arg\nu/\Delta t$. If an observable is a combination of eigenfunctions, $g = \sum_j c_j\xi_j$, linearity gives its whole future,

$$g(\mathbf{x}(t)) = \sum_j c_j\, e^{\Lambda_j t}\,\xi_j(\mathbf{x}(0)), \tag{25}$$

a sum of exponentials and oscillations, and forecasting it is exact and linear. Products of eigenfunctions are eigenfunctions, since $\mathcal{A}(\xi_1\xi_2) = \xi_2\mathcal{A}\xi_1 + \xi_1\mathcal{A}\xi_2 = (\Lambda_1 + \Lambda_2)\xi_1\xi_2$, so the eigenvalues of products are sums of eigenvalues.

The harmonic oscillator $\dot{x} = v$, $\dot{v} = -\omega_0^2 x$ with the angular frequency ω_0 shows these relations. Its generator is $\mathcal{A} = v\,\partial_x - \omega_0^2 x\,\partial_v$, and the complex combination $\xi_\pm = x \pm \mathrm{i}v/\omega_0$ of position and velocity obeys $\mathcal{A}\xi_+ = v - \mathrm{i}\omega_0 x = -\mathrm{i}\omega_0\,\xi_+$, so $\Lambda = \mp\mathrm{i}\omega_0$. The position $x = (\xi_+ + \xi_-)/2$ is a combination of the two, and equation (25) gives $x(t) = \mathrm{Re}\big[e^{-\mathrm{i}\omega_0 t}\xi_+(\mathbf{x}(0))\big]$, the familiar oscillation. The energy $\frac{1}{2}v^2 + \frac{1}{2}\omega_0^2 x^2 = \frac{1}{2}\omega_0^2\xi_+\xi_-$ has $\Lambda = \mathrm{i}\omega_0 - \mathrm{i}\omega_0 = 0$, since it does not change. For a linear law the eigenfunctions are linear functions of the state. For a nonlinear law they are nonlinear functions, and the next example shows a law for which a few of them are known exactly.

3.4 An exact example of lifting

Consider the map

$$x_{k+1} = a\,x_k, \qquad y_{k+1} = b\,y_k + c\,x_k^2, \tag{26}$$

with real parameters a, b and c, which carry this meaning only in the present example. The update of the state $\mathbf{x}_k = (x_k, y_k)^\top$ can be written with a matrix,

$$\begin{pmatrix} x_{k+1} \\ y_{k+1} \end{pmatrix} = B(\mathbf{x}_k) \begin{pmatrix} x_k \\ y_k \end{pmatrix}, \qquad B(\mathbf{x}_k) = \begin{pmatrix} a & 0 \\ c\,x_k & b \end{pmatrix}, \tag{27}$$

whose entry $c\,x_k$ depends on the state. Equation (27) is therefore still the nonlinear map (26). A linear map applies one fixed matrix to every state, so that it maps the doubled state to the doubled image. Here the doubled state $2\mathbf{x}_k$ is mapped to $(2a\,x_k,\ 2b\,y_k + 4c\,x_k^2)$, which differs from twice the image of $\mathbf{x}_k$ whenever $c\,x_k \neq 0$, and no constant 2×2 matrix reproduces the term $c\,x_k^2$. Lifting removes the state-dependent entry. We add the square x^2 as a third coordinate and form the vector of observables

$\mathbf{q} \coloneqq (x, y, x^2)^\top$. In the second equation the term $c\,x_k^2$ becomes c times the third coordinate, and the third coordinate evolves as $x_{k+1}^2 = a^2 x_k^2$, which is again linear in $\mathbf{q}$. The three equations together read

$$\mathbf{q}_{k+1} = K\,\mathbf{q}_k, \qquad K = \begin{pmatrix} a & 0 & 0 \\ 0 & b & c \\ 0 & 0 & a^2 \end{pmatrix}, \tag{28}$$

with a constant matrix K, the same for every state. The state-dependent form (27) of the map has become the linear map (28) on a space of one more dimension, without any approximation (Figure 9). In the language of the previous subsections, the span of x, y and x^2 is mapped into itself by $\mathcal{K}$, and K is the matrix of $\mathcal{K}$ on this span.

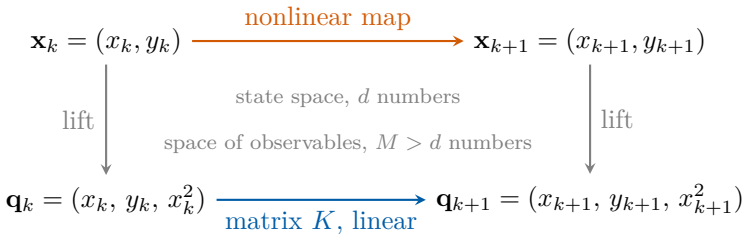


Figure 9: Lifting. Top, the state is advanced by a nonlinear map. Bottom, a list of observables of the state, here the two coordinates and the square x^2 of the map in equation (26), is advanced by the constant matrix K of equation (28). Both paths from $\mathbf{x}_k$ to $\mathbf{q}_{k+1}$ give the same result, so the nonlinear forecast can be computed with the linear map on the bottom row.

A forecast n steps ahead is a matrix power, $y_{k+n} = \mathbf{e}_2^\top K^n \mathbf{q}_k$ with $\mathbf{e}_2 = (0, 1, 0)^\top$. For $a = 0.9$, $b = 0.5$ and $c = 1$ the matrix power and the direct iteration of equation (26) agree to 4×10^{-16} over 50 steps from 200 random starts, the level of round-off.

The eigenvalues a, b and a^2 of K belong to three eigenfunctions in the sense of equation (24). Two of them are $\xi_a \coloneqq x$ and $\xi_{a^2} \coloneqq x^2$. The third follows from a left eigenvector of K, a row vector $\mathbf{l}$ with $\mathbf{l}K = b\,\mathbf{l}$. The function $\xi_b(\mathbf{q}) \coloneqq \mathbf{l} \cdot \mathbf{q}$ then obeys $\xi_b(\mathbf{q}_{k+1}) = \mathbf{l}K\mathbf{q}_k = b\,\xi_b(\mathbf{q}_k)$. Solving $\mathbf{l}K = b\,\mathbf{l}$ gives $\mathbf{l} = (0, 1, c/(b - a^2))$, which exists when $b \neq a^2$, so

$$\xi_b(x, y) = y + \frac{c}{b - a^2}\,x^2, \qquad \xi_b(x_{k+1}, y_{k+1}) = b\,\xi_b(x_k, y_k). \tag{29}$$

For $|a| < 1$ and $|b| < 1$, as for $a = 0.9$ and $b = 0.5$, these three functions decay geometrically along any trajectory, each with its own rate, while the state itself moves on a curved path (Figure 10).

The lifting is exact because the span of x, y and x^2 is invariant, and a larger list is exact only if it is invariant as well. The square y^2 is mapped to $(b\,y + c\,x^2)^2 = b^2y^2 + 2bc\,x^2y + c^2x^4$, which contains the new monomials x^2y and x^4. These close among themselves, $x^2y \mapsto a^2b\,x^2y + a^2c\,x^4$ and $x^4 \mapsto a^4x^4$, so the six observables $(x, y, x^2, y^2, x^2y, x^4)$ span an invariant subspace with the matrix

$$K_6 = \begin{pmatrix} a & 0 & 0 & 0 & 0 & 0 \\ 0 & b & c & 0 & 0 & 0 \\ 0 & 0 & a^2 & 0 & 0 & 0 \\ 0 & 0 & 0 & b^2 & 2bc & c^2 \\ 0 & 0 & 0 & 0 & a^2b & a^2c \\ 0 & 0 & 0 & 0 & 0 & a^4 \end{pmatrix}. \tag{30}$$

Its eigenvalues are the diagonal entries a, b, a^2, b^2, a^2b and a^4, products of a and b, and its eigenfunctions are the products ξ_a, ξ_b, ξ_a^2, ξ_b^2, $\xi_a^2\xi_b$ and ξ_a^4 of the eigenfunctions found above. In contrast, for the map $x_{k+1} = a\,x_k + e\,x_k^2$ with $e \neq 0$ the image of x^n contains x^{2n}, every new power demands a higher one, and no finite list of powers closes.

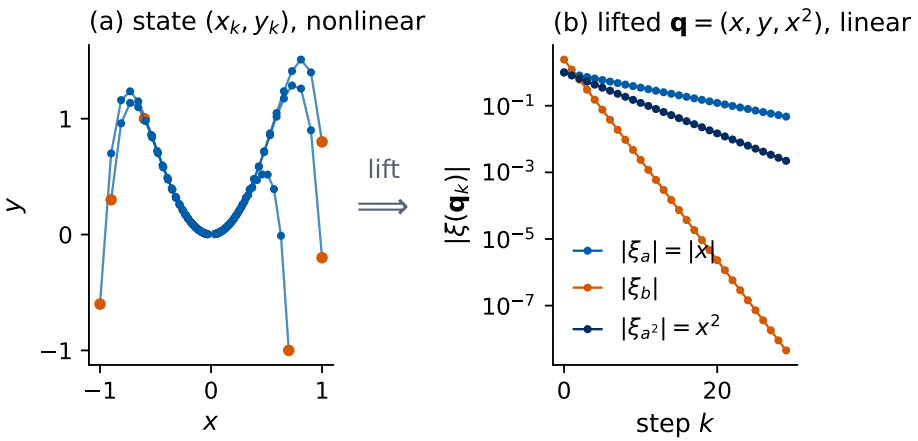


Figure 10: Lifting the map of equation (26) with $a = 0.9$, $b = 0.5$, $c = 1$. Left, trajectories in the state plane (x, y), curved because x^2 drives y. Right, the moduli of the three eigenfunctions $|\xi_a| = |x_k|$, $|\xi_b|$ and $|\xi_{a^2}| = x_k^2$ along one trajectory against the step k, on a logarithmic scale. Each is a straight line whose slope is the logarithm of the modulus of its eigenvalue.

3.5 The pendulum and other oscillations

The pendulum shows which eigenvalues a nonlinear physical system has. With the angle θ, the angular velocity ω, the small-amplitude frequency ω_p and the damping rate γ its law is

$$\dot\theta = \omega, \qquad \dot\omega = -\omega_p^2 \sin\theta - \gamma\,\omega, \tag{31}$$

with the generator $\mathcal{A}g = \omega\,\partial_\theta g - (\omega_p^2 \sin\theta + \gamma\omega)\,\partial_\omega g$. Near the hanging position $(0, 0)$ the flow is close to its linearisation with the Jacobian $J = \begin{pmatrix} 0 & 1 \\ -\omega_p^2 & -\gamma \end{pmatrix}$, whose eigenvalues

$$\Lambda_\pm = -\frac{\gamma}{2} \pm \mathrm{i}\sqrt{\omega_p^2 - \frac{\gamma^2}{4}} \tag{32}$$

are $\Lambda_\pm = -0.1 \pm 0.99499\,\mathrm{i}$ for $\omega_p = 1$ and $\gamma = 0.2$. A left eigenvector $\mathbf{l}$ of J with $\mathbf{l}J = \Lambda\mathbf{l}$ gives the linear function $\xi^{\mathrm{lin}} := \mathbf{l}\cdot(\theta, \omega)$, as in the lifting example, and with $\mathbf{l} = (-\omega_p^2/\Lambda,\, 1)$ the generator yields

$$\mathcal{A}\xi^{\mathrm{lin}} - \Lambda\,\xi^{\mathrm{lin}} = \omega_p^2\,(\theta - \sin\theta) = \omega_p^2\Big(\frac{\theta^3}{6} - \frac{\theta^5}{120} + \dots\Big). \tag{33}$$

The defect starts at third order in the angle, so ξ^{lin} is an eigenfunction only close to the fixed point. Without friction, $\gamma = 0$, the eigenvalues become $\Lambda_\pm = \pm\mathrm{i}\omega_p$, and ξ^{lin} is, up to a constant factor, the eigenfunction $\xi_\mp$ of the harmonic oscillator with $\omega_0 = \omega_p$, the small-amplitude limit of the pendulum. The Koopman eigenfunctions with the eigenvalues $\Lambda_\pm$, called principal eigenfunctions, agree with ξ^{lin} to first order at the fixed point and contain higher powers of θ and ω farther away. They are defined on the whole region of states that come to rest at $(0, 0)$ [23], and their modulus and phase serve as polar coordinates in which the damped swing is a uniform rotation with a uniform decay. Products of the two give the eigenvalues $n_+\Lambda_+ + n_-\Lambda_-$ for non-negative integers n_+ and n_-, among them $\Lambda_+ + \Lambda_- = -\gamma$, the decay rate of $|\xi|^2$, which for a small swing is proportional to the energy.

The energy $E := \frac{1}{2}\omega^2 + \omega_p^2(1 - \cos\theta)$ obeys $\mathcal{A}E = -\gamma\,\omega^2$. Without friction it is conserved, $\mathcal{A}E = 0$, so the energy, and every function of it, is a Koopman eigenfunction with $\Lambda = 0$. Friction changes the spectrum qualitatively. Without friction the phase of a swing advances with the frequency $\Omega(E)$ of its own orbit, which decreases from ω_p at small amplitude towards zero near the energy $2\omega_p^2$ of the upright position. A function of the phase is then multiplied by $e^{\mathrm{i}\Omega(E)t}$ only on a single orbit, and over a range of energies the frequencies form a continuous band. With friction every swing ends at the same fixed point, and the single pair $\Lambda_\pm$ of equation (32) governs the whole basin.

A self-sustained oscillation, such as the output of an electronic oscillator, is described by a limit cycle, a closed orbit that attracts the nearby trajectories. The phase along such a cycle of angular

frequency Ω defines an eigenfunction $e^{\mathrm{i}\vartheta}$ with $\Lambda = \mathrm{i}\Omega$, and its powers give $\Lambda = \mathrm{i}n\Omega$ for every integer n. A periodic motion is thus described by eigenvalues on the imaginary axis at the multiples of its frequency, one for each harmonic of the signal. A chaotic flow is different. Its correlation functions decay, a property called mixing, and its Koopman operator then has a continuous spectrum [57, 22], a continuum of values Λ that admits no eigenfunctions in the ordinary sense. No finite set of observables that contains the measured coordinate is mapped into its own span, and every finite linear model of a mixing flow is an approximation.

3.6 Linear maps fitted to data

For an unknown system the eigenfunctions are unknown, and one chooses a finite set of observables, a dictionary $\mathbf{g}(\mathbf{x}) = (g_1(\mathbf{x}), \ldots, g_M(\mathbf{x}))^\top$, and fits the matrix that best maps the dictionary at one step onto the dictionary at the next step. With n_p pairs of states $(\mathbf{x}_i, \mathbf{x}'_i)$, where $\mathbf{x}'_i$ is the state one step after $\mathbf{x}_i$, the fit minimises the mean squared mismatch with a ridge penalty β,

$$\mathcal{L}(K) = \frac{1}{n_p} \sum_{i=1}^{n_p} \left\| \mathbf{g}(\mathbf{x}'_i) - K\mathbf{g}(\mathbf{x}_i) \right\|^2 + \beta \left\| K \right\|_F^2, \tag{34}$$

where $\|K\|_F^2$, the squared Frobenius norm, is the sum of the squared entries of K. The loss is quadratic in the entries of K. Its derivative with respect to K is $-2\Sigma_1 + 2K\Sigma_0 + 2\beta K$, and setting it to zero gives

$$K = \Sigma_1 \left(\Sigma_0 + \beta I\right)^{-1}, \qquad \Sigma_0 = \frac{1}{n_p} \sum_i \mathbf{g}(\mathbf{x}_i)\mathbf{g}(\mathbf{x}_i)^\top, \quad \Sigma_1 = \frac{1}{n_p} \sum_i \mathbf{g}(\mathbf{x}'_i)\mathbf{g}(\mathbf{x}_i)^\top. \tag{35}$$

The matrix Σ_0 is the covariance of the dictionary with itself and Σ_1 its covariance with the dictionary one step later. This is extended dynamic mode decomposition (EDMD) [21]. With the state coordinates as the dictionary, $\mathbf{g}(\mathbf{x}) = \mathbf{x}$, it reduces to dynamic mode decomposition (DMD), a standard tool of fluid dynamics [20]. For the closed lifting example the fitted matrix reproduces K of equation (28). Whatever the dictionary misses shows up in the closure error,

$$\mathbf{r}(\mathbf{x}) \coloneqq \mathbf{g}(\mathbf{x}') - K\mathbf{g}(\mathbf{x}), \tag{36}$$

which vanishes for every state exactly when the dictionary is mapped into its own span.

For a dictionary that is not closed, equation (35) is a projection. Each row of K is the least-squares fit of one observable at the next step by the observables now, so $K\mathbf{g}$ is the part of $\mathcal{K}\mathbf{g}$ that lies in the span of the dictionary, with distances measured by averages over the data. The fitted matrix, and with it its eigenvalues, therefore depends on where the data lie. The lifting example makes this visible. We take 500 pairs with (x, y) uniform in the square $[-1, 1]^2$. With the dictionary (x, y), which is DMD, the fitted matrix leaves a root-mean-square closure error of 0.445 in the y component. This is the size of the missing term $c\,x^2$, whose root mean square over the square is $\sqrt{1/5} \approx 0.447$. The fitted eigenvalue that should be $b = 0.5$ comes out as 0.471. Over the whole square x^2 is uncorrelated with x and with y, and infinitely many pairs would return b, so the deviation comes from the correlation of x^2 and y in the finite sample. With (x, y, x^2) the fit recovers K to 9×10^{-16} and the error drops to round-off. Adding y^2 alone keeps the first three components exact and introduces an error of 0.193 in the new component, while the closed six-function dictionary of equation (30) has closure errors below 1×10^{-15} and reproduces K_6 to 1×10^{-14}. Closure is a property of the subspace that the dictionary spans, and a larger dictionary can close worse or better than a smaller one.

The pendulum (31) has no polynomial eigenfunctions, so a polynomial dictionary can only approximate them. We fit EDMD with all 36 monomials $1, \theta, \omega, \theta^2, \theta\omega, \ldots, \omega^7$ of total degree up to 7 on 4000 pairs of states one step $\Delta t = 0.1$ apart, the first state of each pair uniform in $[-1, 1]^2$, with $\beta = 10^{-8}$, and convert the fitted eigenvalues with equation (24). With friction EDMD returns $\Lambda = -0.1001 \pm 0.9942\,\mathrm{i}$ against the exact $-0.1 \pm 0.99499\,\mathrm{i}$, the real value -0.1991 against $\Lambda_+ + \Lambda_- = -0.2$, and $-0.1749 \pm 2.0063\,\mathrm{i}$ against $2\Lambda_\pm$ (Figure 11, left). The low-lying values sit close to the lattice $n_+\Lambda_+ + n_-\Lambda_-$. The value $\Lambda = 0$ belongs to the constant function, which every flow

leaves unchanged. Without friction one eigenfunction with Λ close to zero reproduces the energy, up to a constant and a factor, with a relative error of 5×10^{-4}. The energies of the sampled square reach 0.960, where the frequency of the swing is $\Omega = 0.8549$, so the sampled swings have frequencies between 0.8549 and 1. The oscillating EDMD eigenvalues scatter over this band and beyond it and have spurious positive real parts up to 0.042 (Figure 11, right). A finite dictionary cannot represent a continuous band of frequencies, and its eigenvalues then describe the data distribution more than the flow.

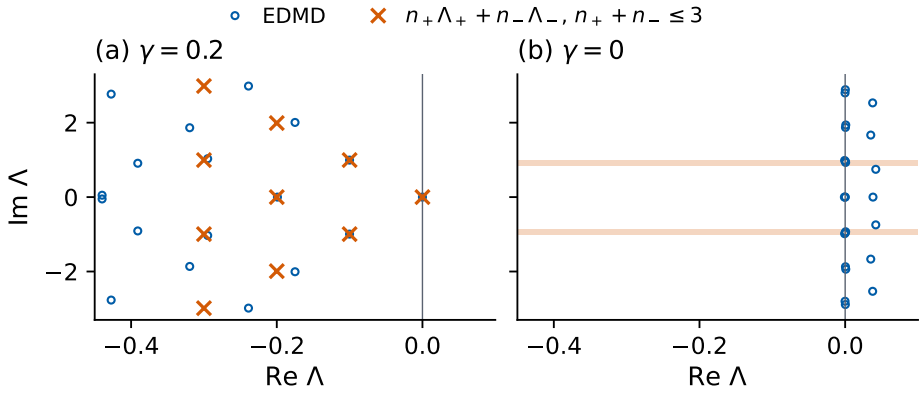


Figure 11: EDMD eigenvalues $\Lambda = \ln \nu / \Delta t$ of the pendulum in the complex plane, for the dictionary of the 36 monomials in θ and ω of total degree up to 7 and $\Delta t = 0.1$. Left, with friction, $\gamma = 0.2$, together with the exact values $n_+\Lambda_+ + n_-\Lambda_-$ of equation (32) (crosses). Right, without friction, with the band $0.8549 \le |\mathrm{Im}\,\Lambda| \le 1$ of the frequencies $\Omega(E)$ of the sampled swings shaded.

3.7 A measured signal as its own dictionary

A record offers no state and no dictionary chosen in advance, only the values $s_k = h(\mathbf{x}_k)$ of one observable. Its past values are the same observable evaluated at earlier states, so the window $\mathbf{u}_k = (s_k, s_{k-1}, \ldots, s_{k-L+1})$ of equation (3) is a dictionary that the dynamics itself generates from h. For an invertible flow $s_{k-j} = (\mathcal{K}^{-j}h)(\mathbf{x}_k)$, so the window collects the measurement and its images under the inverse Koopman operator. Stacking the windows as columns gives a matrix that is constant along its antidiagonals, a Hankel matrix. Fitting a linear map from $\mathbf{u}_k$ to $\mathbf{u}_{k+1}$ with equation (35) is dynamic mode decomposition on the window, called delay or Hankel DMD, and iterating the fitted matrix gives a recursive forecast.

We test the construction on three signals. The first is a sum of two harmonics, $s(t) = \sin t + \frac{1}{2}\sin 2t$, sampled every $\Delta t = 0.05$, whose Koopman spectrum on its orbit consists of the frequencies ± 1 and ± 2. The others are the periodic Mackey–Glass record and Lorenz x. Each record is rescaled to the interval $[0, 1]$ with the minimum and maximum of its training block and then centred with the mean of the training block, so the plotted values fluctuate around zero. The forecasts start from origins in the last 1000 samples, every fifth sample for which the window and 300 forecast steps lie inside this block, which gives 137 origins for the windows of $L = 20$ and 121 for the window of $L = 100$. The normalised error is the root-mean-square difference between forecast and truth at one lead, over the origins, divided by the standard deviation of the test block, so a value of one means an error as large as the signal itself.

For the two harmonics and for the periodic Mackey–Glass record the forecast follows the truth over 300 steps (Figure 12), with the largest normalised errors 9×10^{-8} and 7×10^{-5}. For Lorenz x the forecast relaxes to the mean within about one Lyapunov time, $T_\lambda = 1/\lambda_1 = 1.10$ time units or 55 samples with λ_1 of Table 1. The normalised error first exceeds one half after 14 samples and is 0.99 at one Lyapunov time, when the amplitude of the forecast has fallen to 0.12 of the standard deviation of the signal.

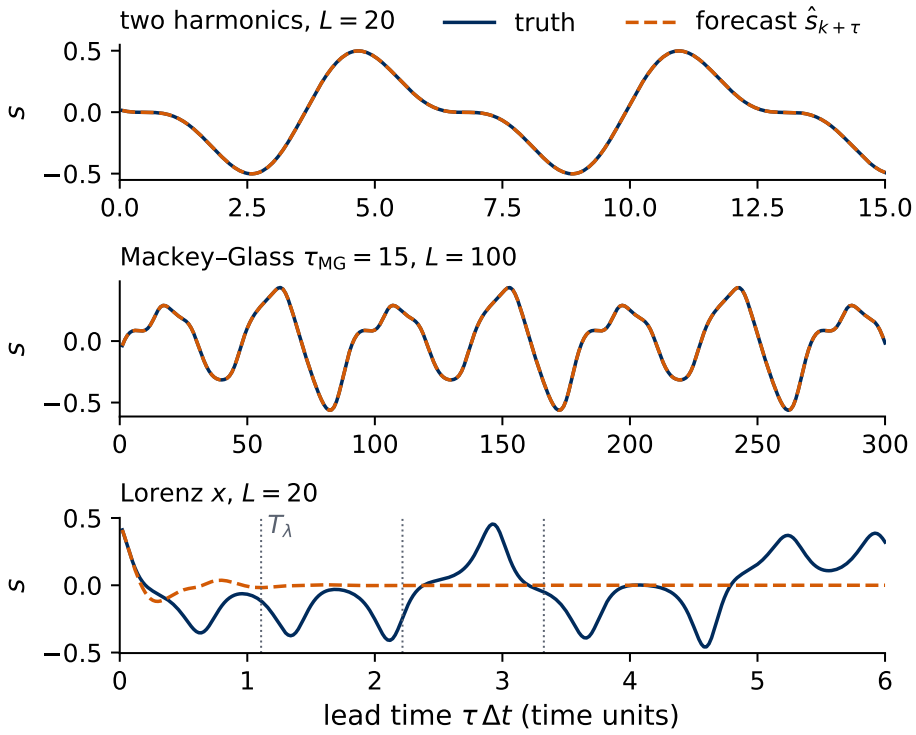


Figure 12: Recursive forecasts of a linear map fitted to windows of L past values, from one origin in the test block. The truth (solid) and the forecast $\hat{s}_{k+\tau}$ (dashed) against the lead time $\tau\Delta t$. Top, the sum of two harmonics with $L = 20$. Middle, the periodic Mackey–Glass record with $L = 100$. Bottom, Lorenz x with $L = 20$, where the dotted lines mark multiples of the Lyapunov time T_λ.

3.8 Autoregression is the smallest dictionary

The linear map fitted to windows has a simple structure. All entries of $\mathbf{u}_{k+1}$ except the first are entries of $\mathbf{u}_k$, shifted by one place, so the fitted matrix copies them without error. Only its first row is fitted, and it is the least-squares prediction of s_{k+1} from the last L values, the AR(L) model of equation (10),

$$K_{\mathrm{AR}} = \begin{pmatrix} \phi_1 & \phi_2 & \cdots & \phi_{L-1} & \phi_L \\ 1 & 0 & \cdots & 0 & 0 \\ 0 & 1 & \cdots & 0 & 0 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & 0 \end{pmatrix}. \tag{37}$$

A matrix of this form, with the coefficients of a recursion in its first row and ones just below the diagonal, is called a companion matrix, and its eigenvalues are the inverse roots $\nu_j = 1/z_j$ of the AR polynomial Q_L. For the Lorenz record with $L = 20$ the first row of the fitted matrix agrees with the least-squares AR(20) coefficients to 6×10^{-7}. The stationarity condition of the AR model, all roots outside the unit circle, is therefore the condition that all eigenvalues of the fitted matrix lie inside it, and the smallest root modulus of the Lorenz AR(20) model, 1.0541, places all eigenvalues inside the circle. The AR model is the Koopman approximation with the smallest dictionary that a record offers, its raw past values, and its innovation is the closure error (36) of that dictionary. A dictionary that closes better has to contain nonlinear functions of the past values.

3.9 Eigenvalues of the fitted maps and their limits

The eigenvalues of the fitted window maps (Figure 13) read as the Koopman eigenvalues of equation (24) wherever the window closes. For the two harmonics five eigenvalues lie on the unit circle, with the frequencies ± 1 and ± 2 of the signal and the frequency 0, which carries the small constant offset left after subtracting the training mean, and all other eigenvalues vanish, since the windows span only five

dimensions. This is the spectrum $\Lambda = in\Omega$ of a periodic motion, restricted to the harmonics that the signal contains. For the periodic Mackey–Glass record 88 of the 100 eigenvalues have a modulus above 0.999, the largest being 1.00005. Its strongly non-sinusoidal oscillation contains many harmonics of the fundamental frequency, whose period is 89.86 samples, and the window of 100 values is longer than one period.

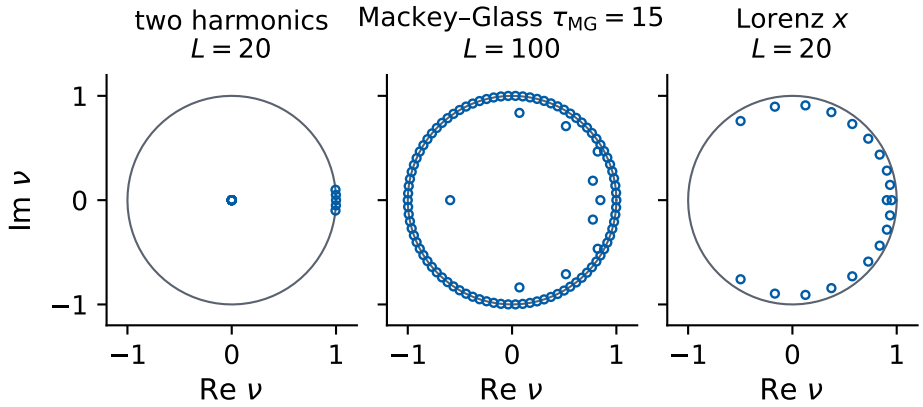


Figure 13: Eigenvalues ν in the complex plane of the linear maps fitted to windows of L past values, for the two harmonics with $L = 20$, the periodic Mackey–Glass record with $L = 100$ and Lorenz x with $L = 20$, with the unit circle drawn.

For Lorenz x all eigenvalues lie inside the unit circle, the largest modulus being 0.951, so every mode of the fitted map decays, the slowest with the rate $\ln 0.951/0.02 \approx -2.5$ per time unit, while the Lorenz flow keeps oscillating. These eigenvalues are not Koopman eigenvalues of the Lorenz flow, which as a mixing flow has a continuous spectrum. They are the eigenvalues of the projection of $\mathcal{K}$ onto the window dictionary, and the damping is the closure error of that dictionary.

The fitted eigenvalues have three limits. The first is the closure error. When the dictionary is not invariant, the eigenvalues depend on the data distribution, as the lifting and pendulum examples showed, and they can be damped or spuriously growing. The second is conditioning. Neighbouring values of a finely sampled signal are nearly equal, and longer windows add nearly dependent coordinates. For Lorenz x with $L = 30$ the matrix of windows has the condition number 3×10^9. A pseudoinverse through the singular value decomposition then gives a largest eigenvalue modulus of 1.0107 and a growing forecast, while the least-squares solver of the `numpy` library gives 0.9635 and a decaying one, so round-off in the solver decides the answer. A ridge penalty $\beta > 0$ in equation (35) removes this ambiguity. The third is noise. Measurement noise enters the windows $\mathbf{u}_k$, the inputs of the least-squares problem, as well as its targets, which biases the fitted matrix [24]. For one variable the effect is seen directly. Fitting $s_{k+1} = \phi\, s_k$ to values with independent noise of variance σ^2 added to a signal of variance σ_s^2 gives, for a long record, $\hat{\phi} = \phi\, \sigma_s^2/(\sigma_s^2 + \sigma^2)$, a modulus smaller than $|\phi|$, so the fitted mode decays faster than the true one. An eigenvalue of a fitted map is a statement about the dynamics only when the dictionary is close to invariant, the fit is well conditioned and the noise is small, and a forecast from such a map must be checked against data.

4 Nonlinear vector autoregression

4.1 Products of past values

The window of raw past values is the smallest dictionary a record offers, and it cannot bend around the fold of the laser. The simplest nonlinear functions to add are products of past values. Any smooth function of the window can be expanded in powers of its entries, and the expansion truncated after the second order keeps a constant, the entries themselves and all their pairwise products. This truncation is the nonlinear vector autoregression (NVAR) of Gauthier and coworkers [25]. It reads L values spaced by κ samples, $(s_k, s_{k-\kappa}, \ldots, s_{k-(L-1)\kappa})$, the same construction as the delay vector, with

a spacing chosen on the validation block, and forms the feature vector

$$\boldsymbol{\varphi}_k := (1,\ s_k,\ s_{k-\kappa},\ \ldots,\ s_{k-(L-1)\kappa},\ s_k^2,\ s_k s_{k-\kappa},\ \ldots,\ s_{k-(L-1)\kappa}^2), \tag{38}$$

with all products $s_{k-i\kappa}s_{k-j\kappa}$ for $i \le j$. For $\kappa = 1$ the linear entries are the window of equation (3). In the scheme of Figure 3 only the feature map changes. The forecast for each horizon is a linear readout of the features, $\hat{s}_{k+\tau} = \mathbf{w}_\tau^\top \boldsymbol{\varphi}_k$, fitted by the ridge regression of equation (18) with $\boldsymbol{\varphi}_k$ in place of $\mathbf{u}_k$. The features are fixed functions of the past values. Only the readout is fitted, and the fit is still one linear solve.

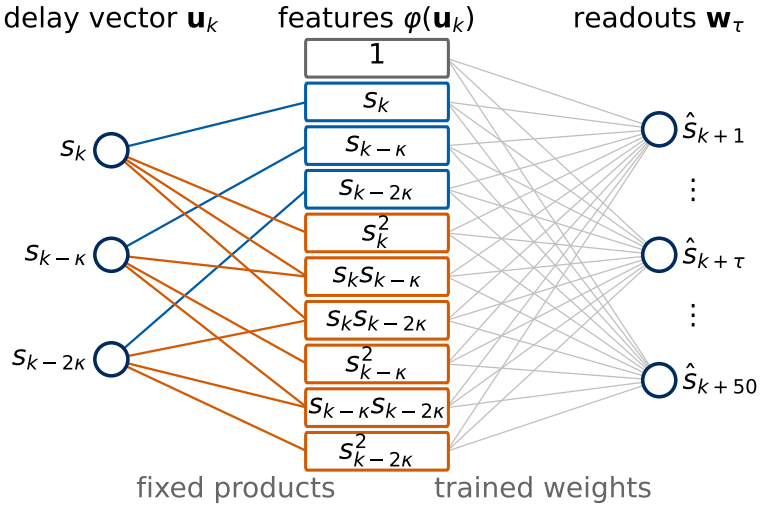


Figure 14: Feature map of NVAR for $L = 3$ values spaced by κ samples. Each input value feeds its own linear feature (blue) and the quadratic features that contain it (orange). Every feature is connected to every readout $\hat{s}_{k+\tau}$, and only these connections are fitted.

The number of features grows quadratically with the window, $1 + L + L(L+1)/2$, from 15 features for $L = 4$ to 561 for $L = 32$. A quadratic function of the window can bend the hyperplane that a linear model fits through the folded return map of the laser.

A quadratic dictionary has a special status for a law of motion whose vector field is itself a quadratic polynomial, as for Lorenz-63 in equation (7). When the full state is measured, the Taylor expansion of the flow over one sampling interval reads

$$\varphi_{\Delta t}(\mathbf{x}) = \mathbf{x} + \Delta t\, \mathbf{F}(\mathbf{x}) + \frac{\Delta t^2}{2}\, J(\mathbf{x})\, \mathbf{F}(\mathbf{x}) + O(\Delta t^3), \tag{39}$$

where $J = \partial \mathbf{F}/\partial \mathbf{x}$ is the Jacobian. For a quadratic $\mathbf{F}$ the first two terms are quadratic in the state, while $J\mathbf{F}$, a linear function times a quadratic one, is cubic. The one-step map of the full state is therefore a quadratic polynomial up to corrections of order Δt^2, which a quadratic dictionary of the state contains. With only the coordinate x measured, the unmeasured coordinates y and z must be recovered from the window, and they are in general nonlinear functions of the past values of x, so the forecast map of the window need not be close to a quadratic polynomial.

4.2 Ridge regularisation of the features

Many features fitted to a finite training block can describe its noise and its peculiarities as well as its dynamics. The ridge penalty β limits the size of the weights and with it this overfitting. Figure 15 shows the effect on the laser. On the training block C_{tot} grows with the window and falls as the penalty grows, as expected for a fit with more freedom. On the validation block the effect of the penalty depends on the window. For $L = 8$, with 45 features, the penalty hardly matters. For $L = 40$, with 861 features fitted to 2190 windows, the nearly unpenalised model reaches $C_{\text{tot}} = 46.8$ on the training block and only 5.7 on the validation block, while $\beta = 10^{-2}$ raises the validation score to 36.3. Large sets of features are useful only together with a penalty, and the window, the spacing and the penalty are chosen together on the validation block.

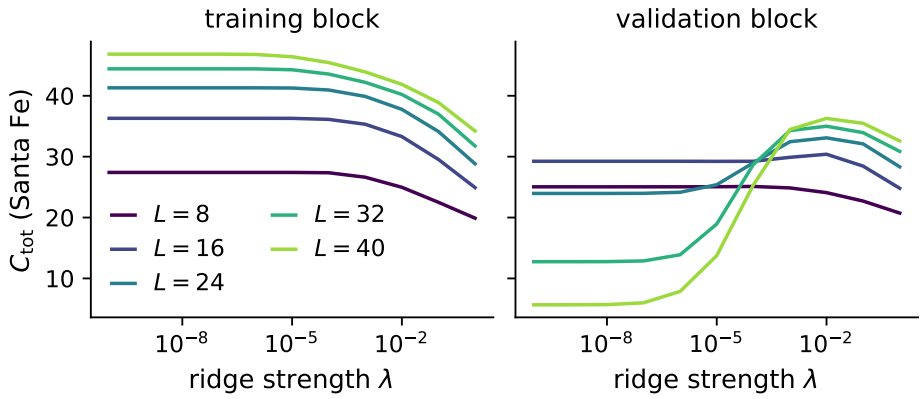


Figure 15: Overfitting and its cure for NVAR on the Santa Fe laser. Total capacity C_{tot} against the ridge strength β, labelled λ in the drawing, on a logarithmic scale, for windows of $L = 8$ to 40 values with spacing 1. Left, on the training block the model was fitted to. Right, on the validation block.

4.3 Capacity gained by the products

Chosen on the validation block, NVAR uses windows of 16 to 32 values. On the laser it reaches the one-step capacity $C = 0.97$ against 0.85 for the linear model, and its points stay close to the diagonal at high intensities, where the linear forecast bent away. Over all horizons it reaches $C_{\mathrm{tot}} = 33.7$ on the laser, twice the 16.8 of the direct AR model, and 50.0, 44.7 and 48.7 on the three Mackey–Glass records, against 49.8, 32.6 and 38.7 (Figure 16). On Lorenz the direct AR and NVAR forecasts reach the same $C_{\mathrm{tot}} = 15.3$ and lose the signal after about 16 samples, so on this record the products bring nothing over these horizons.

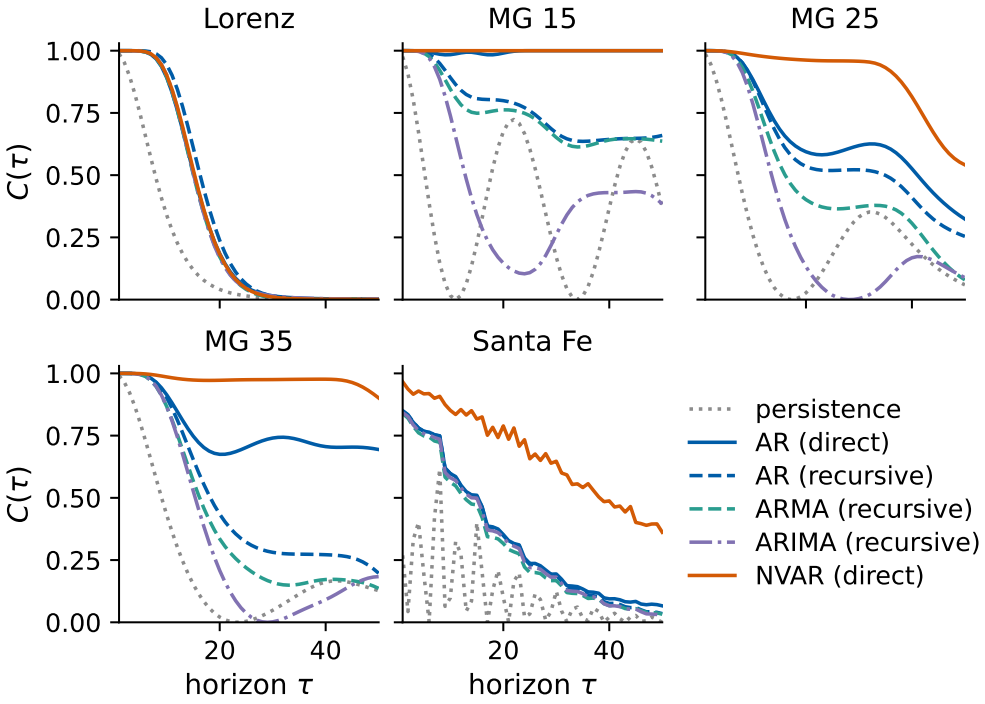


Figure 16: Capacity $C(\tau)$ on the test block against the horizon τ for the five records. Dotted, persistence. Solid blue, direct AR. Dashed, the recursive forecasts of the AR, ARMA and ARIMA models. Solid orange, direct NVAR.

4.4 Free runs of the fitted models

The direct forecasts above are computed from the measured window at every origin. Iterated on its own outputs, the one-step NVAR model behaves differently (Figure 17). Started from the measured

history at the first test origin, it leaves the range of the signal after 5 steps on the laser and after 15 steps on Mackey–Glass at $\tau_{\mathrm{MG}} = 25$. A small error of the first step moves the window slightly away from the windows of the training block, into a region where the quadratic features were never fitted, and the polynomial map amplifies the deviation at every further step. The linear models, iterated in the same way, stay bounded. The AR and ARMA runs follow the signal for a few oscillations and then relax to a constant close to the mean of the data, as their decaying modes require. The ARIMA run on Mackey–Glass at $\tau_{\mathrm{MG}} = 25$ settles at 0.93, far above the data mean of 0.60, because its forecast of the differences relaxes to zero and leaves the record at the level it had reached. A model that forecasts well from measured windows can be useless as a dynamical system of its own, and the two uses have to be tested separately.

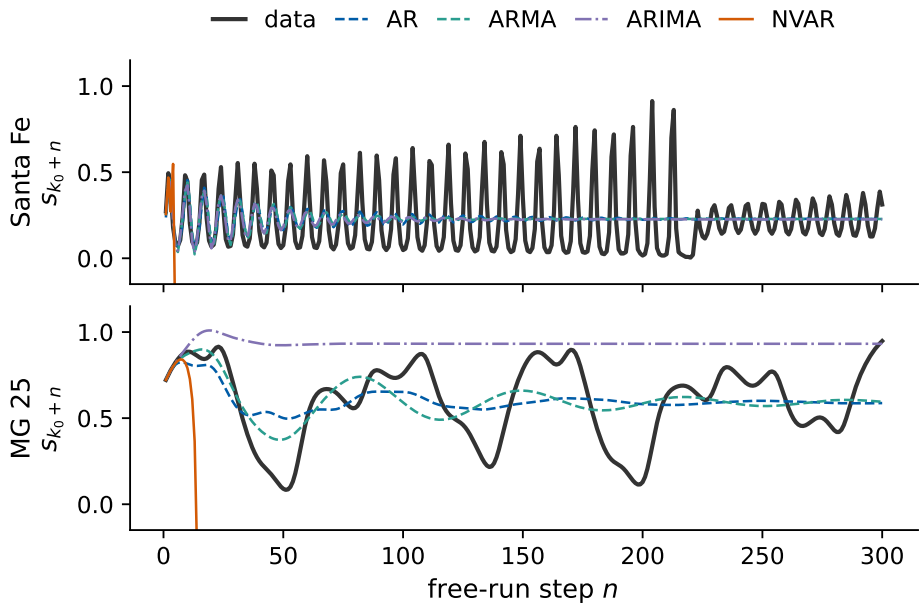


Figure 17: Recursive forecasts over 300 steps from the first test origin, against the free-run step n, for the Santa Fe laser (top) and Mackey–Glass at $\tau_{\mathrm{MG}} = 25$ (bottom). Black, the data. The AR, ARMA and ARIMA runs relax to a constant, and the NVAR run leaves the range of the signal within a few steps.

The products of NVAR are chosen in advance and are the same for every record. A quadratic expansion suits a dynamics that is close to quadratic over the span of one window, and it grows unwieldy when higher powers are needed, since the number of products of order n grows as L^n. On the Lorenz record, where the hidden coordinates have to be recovered from the window, the quadratic products gave no gain at all. Features adapted to the record have to be learned from the data.

5 Neural network forecasters

5.1 The multilayer perceptron on the window

NVAR prescribes its dictionary, the same quadratic products for every record, and on the Lorenz record these products added nothing to the linear model. When the useful functions of the past are unknown, they have to be learned from the record. A neural network does this. The multilayer perceptron (MLP) with one hidden layer reads the window $\mathbf{u}_k$ of the last L values and returns all 50 horizons at once,

$$\hat{\mathbf{y}}_k = W_2 \tanh\left(W_1 \mathbf{u}_k + \mathbf{b}_1\right) + \mathbf{b}_2, \qquad \hat{\mathbf{y}}_k = (\hat{s}_{k+1}, \ldots, \hat{s}_{k+50}), \tag{40}$$

where W_1 and $\mathbf{b}_1$ map the window to n_{hid} hidden units, the hyperbolic tangent acts on each unit, and W_2 and $\mathbf{b}_2$ form the linear readout. In the scheme of Figure 3 the fixed products of NVAR are replaced by the learned features $\tanh(W_1 \mathbf{u}_k + \mathbf{b}_1)$, and the readout is unchanged. The price is

the training. The features depend on W_1 nonlinearly, so the weights are no longer found by one linear solve. They are found by gradient descent, which starts from random weights and moves all of them repeatedly against the gradient of the mean squared error of the forecasts, the gradient being computed by backpropagation, the chain rule applied from the output back to every weight.

All networks of these notes are trained with one protocol. The fitting windows are shuffled at every pass through the data, an epoch, and cut into minibatches of 128 windows. Each minibatch gives one update by Adam [58], an adaptive-moment method, which keeps running averages of the gradient and of its square for every weight and divides the first by the square root of the second, so that every weight moves by a step of similar size whatever the scale of its gradient. The learning rate is 5×10^{-3}. A gradient $\nabla\mathcal{L}$ whose norm exceeds 1 is rescaled to norm 1 before the update, a safeguard called gradient clipping against rare, very large gradients. After every epoch the loss on the validation windows is computed, the weights with the lowest validation loss are kept, and training stops when this loss has not improved for 20 epochs, at most after 250 epochs. This early stopping is the only protection against overfitting.

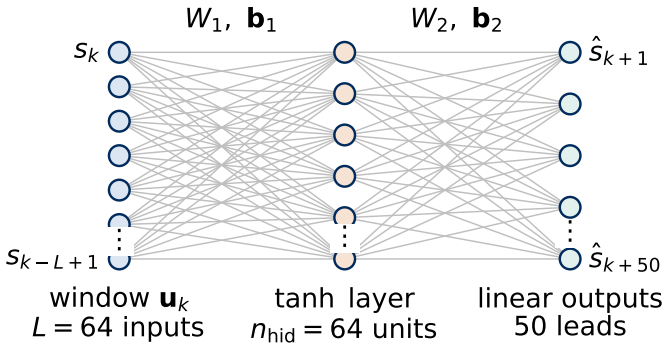


Figure 18: The MLP forecaster. The window of $L = 64$ past values feeds a layer of $n_{\text{hid}} = 64$ hidden units with the hyperbolic tangent, and a linear layer returns the 50 horizons. The weights $W_1, \mathbf{b}_1$ and $W_2, \mathbf{b}_2$ are all trained.

With $L = n_{\text{hid}} = 64$ the network has 7410 parameters. On the laser it reaches $C_{\text{tot}} = 35.3$, close to NVAR, which with the same fitting windows and the same choice on the validation block reaches 34.7. The window matters (Figure 19). With $n_{\text{hid}} = 64$ the total capacity grows from 18.1 for $L = 2$ to 32.9 for $L = 32$ and 35.3 for $L = 64$, so the network uses values far older than the minimal window of Table 1. With $L = 64$ the capacity grows with the number of hidden units from 14.6 for two units to 35.3 for 64 and 36.6 for 256. The MLP and NVAR share a limitation. Their memory is the window and nothing else, so a dependence on values older than L steps is out of reach.

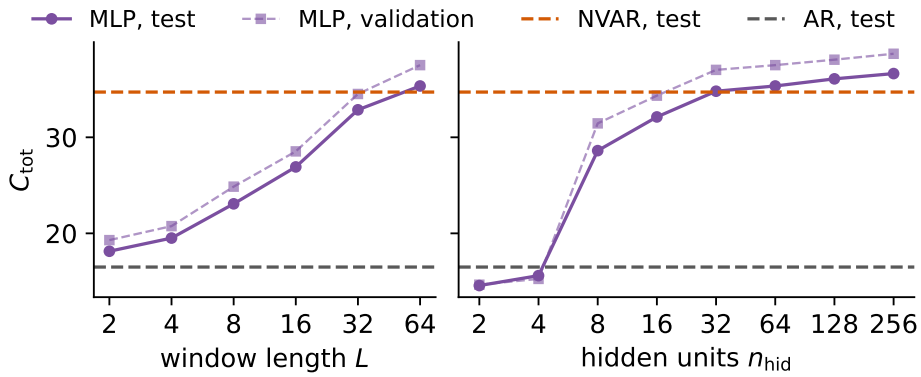


Figure 19: Total capacity C_{tot} of the MLP on the Santa Fe laser on the test block (solid) and the validation block (dashed). Left, against the window length L with $n_{\text{hid}} = 64$. Right, against the number of hidden units n_{hid} with $L = 64$. Horizontal lines, NVAR and AR on the test block.

5.2 The recurrent neural network

A recurrent neural network (RNN) carries a state from step to step, a learned summary of the past whose number of parameters does not depend on the window length. The Elman network [26] updates a hidden state $\mathbf{h}_k \in \mathbb{R}^{n_h}$ with every new value,

$$\mathbf{h}_k = \tanh\left(W_h \mathbf{h}_{k-1} + W_x s_k + \mathbf{b}\right), \qquad \hat{\mathbf{y}}_k = W_y \mathbf{h}_k + \mathbf{b}_y, \tag{41}$$

where the matrix W_h feeds the old state back, the vector W_x brings in the new value and W_y, $\mathbf{b}_y$ form the readout of the 50 horizons. The bold $\mathbf{h}_k$ is the hidden state and is unrelated to the measurement function h. The state depends on s_k, s_{k-1}, and through them on the whole past, while the number of parameters does not grow with the length of the history, since the same weights act at every step. The network is itself a driven dynamical system whose law is learned. Here it reads a window of $L = 64$ values starting from $\mathbf{h} = 0$ at every forecast origin, and the forecast is read from the last state. Unrolled over the window, the network becomes a chain of identical cells that share the weights W_h and W_x, with one input per step and one readout at the end (Figure 20). The training gradient travels the chain in the opposite direction, from the readout back to the earlier states, along the dashed arrows of the figure.

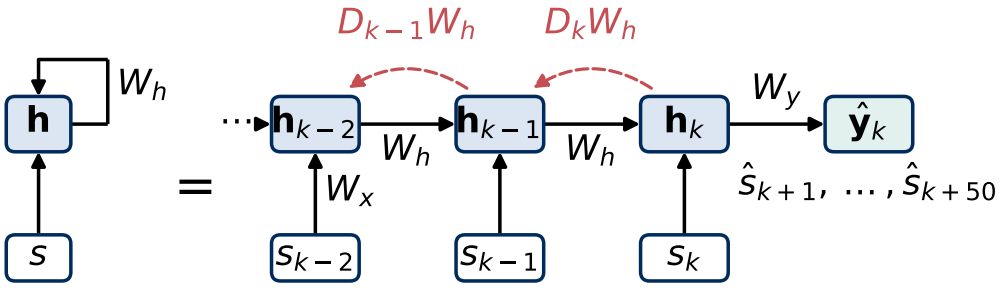


Figure 20: The recurrent network of equation (41). Left, the cell with its feedback loop. Right, the same cell unrolled over the last three steps of the window, with the inputs s_{k-2}, s_{k-1}, s_k entering through W_x, the states $\mathbf{h}_{k-2}$, $\mathbf{h}_{k-1}$, $\mathbf{h}_k$ linked by the same matrix W_h at every step, and the readout W_y from the last state $\mathbf{h}_k$ to the forecast vector $\hat{\mathbf{y}}_k = (\hat{s}_{k+1}, \ldots, \hat{s}_{k+50})$. The dashed red arrows point backwards and are labelled with the step Jacobians $\partial \mathbf{h}_k / \partial \mathbf{h}_{k-1} = D_k W_h$, with $D_k = \mathrm{diag}(1 - \mathbf{h}_k^2)$. The training gradient reaches earlier states along them, multiplied at each step by the transpose $(D_k W_h)^\top = W_h^\top D_k$. The biases are left out of the drawing.

A network with a single unit shows how the state retains past input. With the update $h_k = \tanh(0.5\, h_{k-1} + s_k)$, the initial state $h_0 = 0$ and the inputs $s_1 = 1$, $s_2 = 0$,

$$h_1 = \tanh 1 \approx 0.762, \qquad h_2 = \tanh(0.5 \cdot 0.762) \approx 0.363. \tag{42}$$

The newest input is zero, and the state still carries the earlier input, attenuated by the recurrent weight and by the saturation of the hyperbolic tangent. Each further step multiplies the trace of the old input by about the recurrent weight, so with a weight of 0.5 an input has shrunk by a factor of about $0.5^{10} \approx 10^{-3}$ after ten steps. The two terms inside the hyperbolic tangent of equation (41) have two roles. The term $W_h \mathbf{h}_{k-1}$ carries forward what the network has retained from all earlier values, and the term $W_x s_k$ writes the new value into the state. The hyperbolic tangent keeps every component between -1 and 1, so the state cannot grow without bound. After the last value of the window, the state is a summary of fixed length of the whole window. The delay vector stores selected past values themselves, and the state stores whatever functions of the past the forecast needs. For $n_h = 32$ hidden units the cell has $32 \cdot 32 + 32 + 32 = 1088$ parameters and the readout of the 50 horizons $50 \cdot 32 + 50 = 1650$, in total 2738, fewer than half of the 7410 of the MLP.

5.3 Backpropagation through time

Training the RNN by gradient descent requires the derivative of the forecast with respect to the weights, and the weights act at every step. Unrolling the recursion over the window turns the network

into a deep feedforward network with shared weights, and backpropagation through this unrolled network is called backpropagation through time [27, 28]. The influence of the state n steps back on the last state is the product of the Jacobians of the single steps,

$$\frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-n}} = \prod_{i=k-n+1}^{k} D_i W_h, \qquad D_i = \operatorname{diag}(1 - \mathbf{h}_i^2), \tag{43}$$

where D_i holds the derivatives of the hyperbolic tangent, all between 0 and 1. The norm of a product is at most the product of the norms, and $\|D_i\| \leq 1$, so

$$\left\| \frac{\partial \mathbf{h}_k}{\partial \mathbf{h}_{k-n}} \right\| \leq \|W_h\|^n, \tag{44}$$

where $\|W_h\|$ is the largest singular value of the matrix. A product of many matrices typically grows or shrinks exponentially with the number of factors, $\|\partial \mathbf{h}_k / \partial \mathbf{h}_{k-n}\| \sim e^{\lambda_h n}$. The product has the same structure as the propagation of a small error along a trajectory of a flow, which gave the separation $e^{\lambda_1 t}$ of nearby states. The rate λ_h is therefore a finite-time Lyapunov exponent of the hidden dynamics driven by the input. For $\lambda_h < 0$ the contributions of early inputs to the gradient vanish, and the network cannot learn to use them. For $\lambda_h > 0$ they explode, and gradient descent takes destructive steps.

In practice the gradient is not assembled from these products one by one. It is computed by a backward recursion that visits every step once. Let $\mathcal{L}$ be the loss of one window and write $\bar{\mathbf{h}}_i \coloneqq \partial \mathcal{L} / \partial \mathbf{h}_i$ for the sensitivity of the loss to the state at the step i, including all the ways in which $\mathbf{h}_i$ influences the subsequent states. The last state $\mathbf{h}_k$ enters only the readout, so $\bar{\mathbf{h}}_k = W_y^\top \, \partial \mathcal{L} / \partial \hat{\mathbf{y}}_k$. Every earlier state enters only the next update, and the chain rule through equation (41) gives

$$\bar{\mathbf{h}}_{i-1} = W_h^\top D_i \, \bar{\mathbf{h}}_i, \qquad i = k, k-1, \ldots, \tag{45}$$

so the sensitivity flows backwards through the same network that the state flows forwards through, with the transposed step Jacobian $W_h^\top D_i$ of the dashed arrows in Figure 20. The weights act at every step, so their gradients are sums over the steps of the window,

$$\frac{\partial \mathcal{L}}{\partial W_h} = \sum_i D_i \bar{\mathbf{h}}_i \, \mathbf{h}_{i-1}^\top, \qquad \frac{\partial \mathcal{L}}{\partial W_x} = \sum_i D_i \bar{\mathbf{h}}_i \, s_i, \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{b}} = \sum_i D_i \bar{\mathbf{h}}_i, \tag{46}$$

where $D_i \bar{\mathbf{h}}_i$ is the sensitivity to the argument of the hyperbolic tangent at the step i. Iterating equation (45) from the end of the window produces the products of equation (43). The backward pass costs as much as the forward pass, and all states of the forward pass must be stored, since both equations use them. Automatic differentiation carries out these recursions.

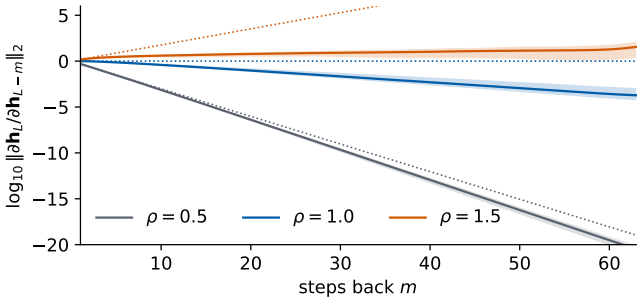


Figure 21: Vanishing and exploding gradients. The logarithm of the norm of $\partial \mathbf{h}_k / \partial \mathbf{h}_{k-n}$ against the number n of steps back, written $\partial \mathbf{h}_L / \partial \mathbf{h}_{L-m}$ in the drawing, where L is the last position of the window and $m = n$, for untrained RNNs whose matrix W_h has all singular values equal to ϱ, labelled ρ in the legend, ten networks for each ϱ. Dotted lines, the slope $\ln \varrho$ per step.

Figure 21 measures the rate λ_h on untrained networks. With all singular values of W_h equal to ϱ it is -0.755 per step for $\varrho = 0.5$ and -0.146 for $\varrho = 1$, where the factors D_i smaller than one still

shrink the product, and +0.030 for $\varrho = 1.5$, where the saturation of the hyperbolic tangent limits the growth. Three remedies are standard. Gradient clipping rescales a gradient $\nabla\mathcal{L}$ whose norm exceeds a threshold c_{clip},

$$\nabla\mathcal{L} \;\to\; \nabla\mathcal{L}\,\frac{c_{\text{clip}}}{\|\nabla\mathcal{L}\|} \qquad \text{if } \|\nabla\mathcal{L}\| > c_{\text{clip}}, \tag{47}$$

which keeps its direction and limits the step, and protects against explosion. A network started with $\varrho = 2$ produced gradients of order 10^3, and its training failed without clipping, with C_{tot} between 6.6 and 10.1, while clipping at norm one raised it to between 22.6 and 37.8 (Figure 22). An orthogonal W_h at the start [29], with all singular values one, places the network close to the boundary between shrinking and growing products. A third remedy changes the cell itself, with gates that let the state pass a step almost unchanged.

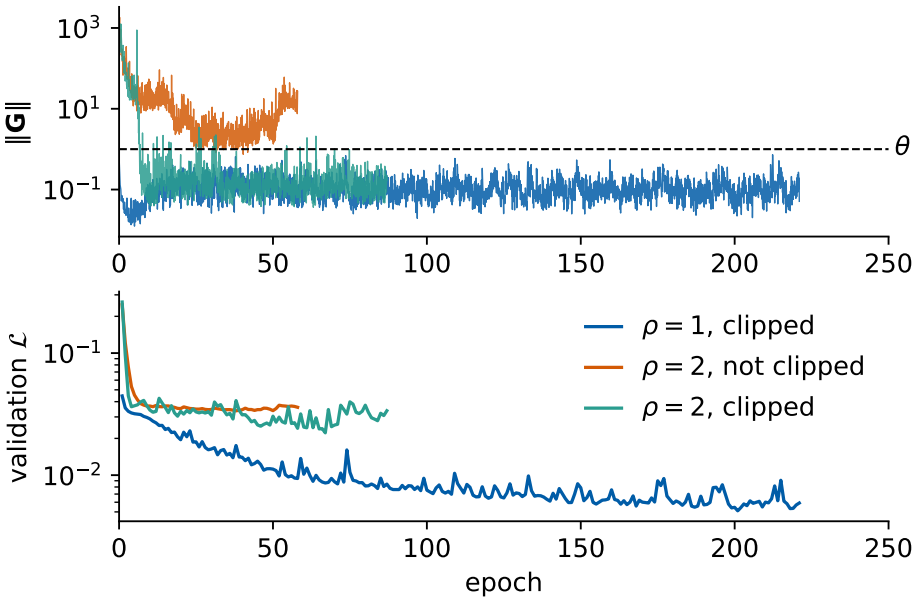


Figure 22: Gradient clipping. Top, the norm $\|\nabla\mathcal{L}\|$ of the gradient before clipping at every update, for a network started with $\varrho = 2$ with and without clipping at norm one and for the network started orthogonal, $\varrho = 1$ in the legend, with the clipping threshold as a dashed line. Bottom, the validation loss after every epoch for the same runs. Without clipping the gradient norm starts near 10^3, and the run stops early at a higher validation loss.

Training moves the recurrent matrix far from its orthogonal start. After training on the laser its spectral radius is 1.40, its singular values range from 0.080 to 2.33, and the product of equation (43) first grows and then decays slowly, at -0.048 per step, so that a perturbation at the beginning of the window still reaches the last state (Figure 23, left). The trained network has placed its hidden dynamics near the boundary between contraction and expansion. Which inputs it uses is measured by the sensitivity $S(n) = \langle \|\partial \hat{\mathbf{y}}_k / \partial s_{k-n}\| \rangle_k$ of the forecast to the value n steps before the origin. It is largest about ten steps back, a little more than one pulsation period of 7.26 samples, and falls to 5% of its maximum at the oldest value of the window, so the forecasts of the RNN depend mainly on the last few pulsations. The sensitivity of the MLP is largest for the newest value and scatters over the whole window (Figure 23, right).

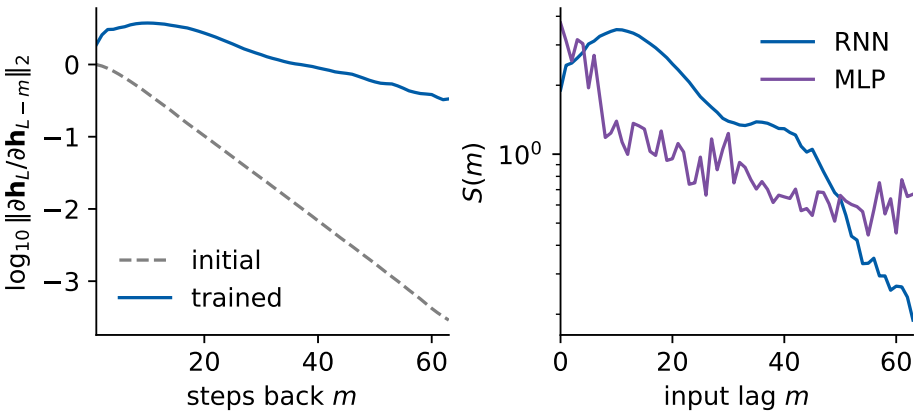


Figure 23: The trained RNN on the Santa Fe laser. Left, $\log_{10}\|\partial \mathbf{h}_k/\partial \mathbf{h}_{k-n}\|$ against the number n of steps back, written $\partial \mathbf{h}_L/\partial \mathbf{h}_{L-m}$ with $m = n$ in the drawing, for the network at its orthogonal start (dashed) and after training (solid). Right, the sensitivity $S(n)$ of the forecast to the input n steps before the origin, on a logarithmic scale, for the RNN and the MLP.

5.4 The long short-term memory cell

The trained RNN uses the last few pulsations, and gradient clipping only limits the explosion of the backward products without stopping their decay. The long short-term memory (LSTM) cell [30] provides a path along which the backward product decays only through gate values, with no weight matrix and no derivative of a saturating function. It adds a second state, the cell state $\mathbf{c}_k$, which is changed only by multiplication and addition, and three gates that control it,

$$\begin{aligned}
\mathbf{f}_k &= \sigma\big(W_f[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_f\big) && \text{forget gate,}\\
\mathbf{i}_k &= \sigma\big(W_i[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_i\big) && \text{input gate,}\\
\mathbf{g}_k &= \tanh\big(W_g[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_g\big) && \text{candidate memory,}\\
\mathbf{c}_k &= \mathbf{f}_k \odot \mathbf{c}_{k-1} + \mathbf{i}_k \odot \mathbf{g}_k && \text{cell-memory update,}\\
\mathbf{o}_k &= \sigma\big(W_o[\mathbf{h}_{k-1}, s_k] + \mathbf{b}_o\big) && \text{output gate,}\\
\mathbf{h}_k &= \mathbf{o}_k \odot \tanh(\mathbf{c}_k) && \text{hidden-state output.}
\end{aligned} \tag{48}$$

Here $[\mathbf{h}_{k-1}, s_k]$ stacks the old hidden state and the new value into one vector, $\sigma(x) = 1/(1+e^{-x})$ is the logistic function with values between zero and one, and $\odot$ multiplies vectors component by component. The three gates $\mathbf{f}_k$, $\mathbf{i}_k$ and $\mathbf{o}_k$ pass through the logistic function, so each of their components lies between zero and one and acts as a fraction. The forget gate $\mathbf{f}_k$ sets which fraction of the old cell state is kept, the input gate $\mathbf{i}_k$ which fraction of the candidate memory $\mathbf{g}_k$ is written, and the output gate $\mathbf{o}_k$ which fraction of the cell state is shown in the hidden state. The candidate memory passes through the hyperbolic tangent, takes values between -1 and 1, and is the new content itself. The cell state $\mathbf{c}_k$ is the memory, changed only by the multiplication with the forget gate and the addition of the gated candidate.

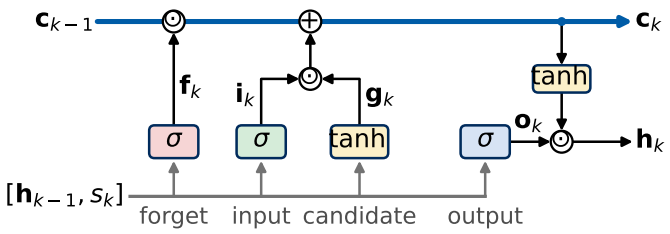


Figure 24: The LSTM cell. The cell state runs from $\mathbf{c}_{k-1}$ to $\mathbf{c}_k$ along the top line, changed only by the forget gate $\mathbf{f}_k$ and the gated candidate $\mathbf{i}_k \odot \mathbf{g}_k$. The hidden state $\mathbf{h}_k$ is the cell state passed through a hyperbolic tangent and the output gate $\mathbf{o}_k$.

A single memory component shows the equations at work. Let the cell state hold $c_{k-1} = 0.8$, and let the current input produce the forget value $f_k = 0.9$, the input value $i_k = 0.2$, the candidate

$g_k = -0.5$ and the output value $o_k = 0.7$. Equation (48) gives

$$c_k = 0.9 \cdot 0.8 + 0.2 \cdot (-0.5) = 0.62, \qquad h_k = 0.7 \tanh 0.62 \approx 0.386. \tag{49}$$

Most of the old content is kept and a small amount of new content of the opposite sign is added. Each of the four affine maps acts on the n_h+1 numbers of $[\mathbf{h}_{k-1}, s_k]$, so the cell has $4(n_h(n_h+1)+n_h) = 4352$ parameters for $n_h = 32$, and with the readout of the 50 horizons the forecaster has 6002.

The point of the construction is the backward recursion along the cell state. Write $\bar{\mathbf{c}}_j \coloneqq \partial\mathcal{L}/\partial\mathbf{c}_j$ and $\bar{\mathbf{h}}_j \coloneqq \partial\mathcal{L}/\partial\mathbf{h}_j$. The cell state $\mathbf{c}_j$ enters the hidden state $\mathbf{h}_j$ and the next cell state $\mathbf{c}_{j+1}$, and the chain rule along these two paths, at fixed gates, gives

$$\bar{\mathbf{c}}_j = \bar{\mathbf{h}}_j \odot \mathbf{o}_j \odot (1 - \tanh^2 \mathbf{c}_j) + \mathbf{f}_{j+1} \odot \bar{\mathbf{c}}_{j+1}. \tag{50}$$

The second term carries the sensitivity from one step to the previous one multiplied only by the forget values, $\partial\mathbf{c}_{j+1}/\partial\mathbf{c}_j = \operatorname{diag}(\mathbf{f}_{j+1})$, a diagonal matrix that contains no weight matrix and no derivative of a saturating function. The gates themselves depend on $\mathbf{h}_j$ and add further paths, which this direct term leaves out. With a forget value near one, a stored value decays slowly, as f^n after n steps, with the memory time $\tau_{\text{mem}} = -1/\ln f$, for example a hundred steps for $f = 0.99$, and the sensitivity flows back along the same path and decays as slowly. The network can learn how long to remember, through the forget gate, instead of having the memory set by the products of equation (43).

Trained on the laser, the forget gates settle on memory times of a few steps, with a median of 2.5 and a largest value of 3.5 steps over the units, down from a median of 3.3 steps before training (Figure 26). The memory time measures only the direct retention along the cell state, and information can also travel through the hidden state and the gates, so a short memory time does not show that the network ignores older values. The LSTM reaches a higher total capacity on the laser than the RNN, and one hypothesis, which these runs do not test, is that its advantage comes from the gated processing of the last few pulsations.

The long memory that the gates make possible shows up when it is needed. Driven by windows of 200 random values, an untrained RNN loses about 22 orders of magnitude of sensitivity to the oldest input. An untrained LSTM whose forget gates start near one, through the positive bias $b_f = 3$ [31] in equation (48), keeps this sensitivity of order one over all 200 steps, while with $b_f = 1$ it decays far more slowly than for the RNN and with $b_f = 0$ faster than for the RNN (Figure 25). The bias sets the starting memory time, $\sigma(b_f)$ being about 0.5, 0.73 and 0.95 for $b_f = 0$, 1 and 3, which correspond to memory times of about 1.4, 3.2 and 20 steps. A memory time of 20 steps along the cell state alone would shrink the sensitivity by many orders of magnitude over 200 steps, so for $b_f = 3$ the paths through the hidden state and the gates also carry it.

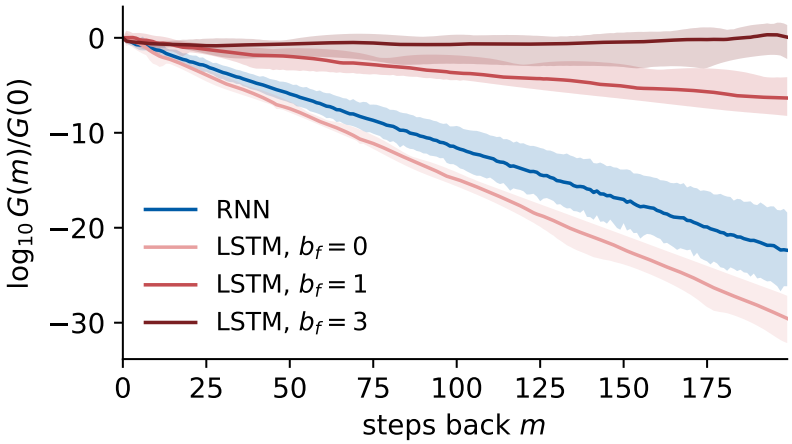


Figure 25: Sensitivity of untrained networks to inputs far back. The norm of the derivative of the output with respect to the input n steps back, on a logarithmic scale, for windows of 200 random values, for the RNN and for the LSTM with the forget biases $b_f = 0$, 1 and 3, averaged over windows and initialisations.

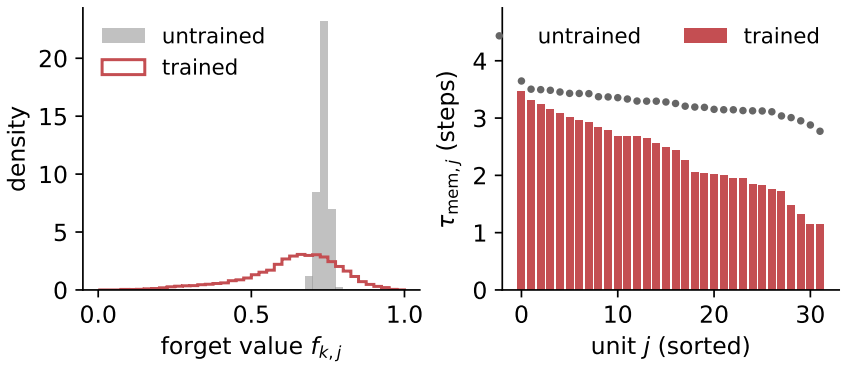


Figure 26: Forget gates of the LSTM on the Santa Fe laser. Left, the distribution of forget values f over units and steps, before (grey) and after (red) training. Right, the memory time $\tau_{\mathrm{mem}} = -1/\ln f$ of each unit in steps, sorted, before and after training.

5.5 The gated recurrent unit

The LSTM carries two states and four affine maps. The gated recurrent unit (GRU) keeps the gated, additive update and simplifies the rest [32]. It carries a single state $\mathbf{h}_k$, which serves both as memory and as output, and replaces it at every step by a weighted average of the old state and a new candidate,

$$\begin{aligned} \mathbf{z}_k &= \sigma\left(\mathbf{w}_z s_k + W_{hz}\mathbf{h}_{k-1} + \mathbf{b}_z\right) && \text{update gate,} \\ \mathbf{r}_k &= \sigma\left(\mathbf{w}_r s_k + W_{hr}\mathbf{h}_{k-1} + \mathbf{b}_r\right) && \text{reset gate,} \\ \mathbf{n}_k &= \tanh\left(\mathbf{w}_n s_k + \mathbf{b}_n + \mathbf{r}_k \odot \left(W_{hn}\mathbf{h}_{k-1}\right)\right) && \text{candidate state,} \\ \mathbf{h}_k &= \left(1 - \mathbf{z}_k\right) \odot \mathbf{n}_k + \mathbf{z}_k \odot \mathbf{h}_{k-1} && \text{hidden-state update,} \end{aligned} \tag{51}$$

with the input weights $\mathbf{w}_z, \mathbf{w}_r, \mathbf{w}_n \in \mathbb{R}^{n_h}$, the recurrent matrices $W_{hz}, W_{hr}, W_{hn} \in \mathbb{R}^{n_h \times n_h}$ and the biases $\mathbf{b}_z, \mathbf{b}_r, \mathbf{b}_n$. The update gate $\mathbf{z}_k$ sets, unit by unit, how much of the old state is kept. A unit with $z \approx 1$ passes its value through the step almost unchanged, and a unit with $z \approx 0$ takes the candidate $\mathbf{n}_k$. The reset gate $\mathbf{r}_k$ sets how much of the old state enters the candidate, so with $\mathbf{r}_k \approx 0$ the candidate depends on the new value alone, as if the history had been reset (Figure 27). The bold $\mathbf{r}_k$ denotes the reset gate and is unrelated to the Pearson coefficient r. We apply the reset gate after the matrix, $\mathbf{r}_k \odot (W_{hn}\mathbf{h}_{k-1})$, as the common software libraries do. Cho and coworkers apply it before the matrix, $W_{hn}(\mathbf{r}_k \odot \mathbf{h}_{k-1})$, which gives a slightly different cell of the same family. Compared with the LSTM, the update gate plays the roles of the forget and the input gate tied together, so whatever is written displaces an equal fraction of what was there, and there is no output gate.

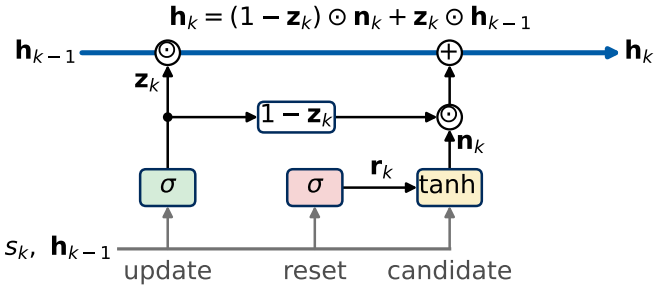


Figure 27: The GRU cell. The state runs from $\mathbf{h}_{k-1}$ to $\mathbf{h}_k$ along the top line, where it is multiplied by the update gate $\mathbf{z}_k$ and added to the candidate $\mathbf{n}_k$ weighted by $1 - \mathbf{z}_k$. The update gate (green), the reset gate (red) and the candidate (yellow) receive the new value s_k and the old state, and the reset gate $\mathbf{r}_k$ scales the recurrent term inside the candidate.

The interpolation in the last line of equation (51) gives the GRU the same protected path as the cell state of the LSTM. Differentiating it at fixed gates gives the direct contribution $\partial \mathbf{h}_k / \partial \mathbf{h}_{k-1} = \mathrm{diag}(\mathbf{z}_k)$, which contains no weight matrix, so a unit with its update gate near one passes both its content and its gradient through many steps. With three affine maps in place of four, the GRU cell has three

quarters of the parameters of an LSTM cell of the same width, $3 \cdot 1088 = 3264$ for $n_h = 32$, and the forecaster with its readout has 4914 parameters. Trained on the laser with the same windows and protocol as the RNN and the LSTM, it reaches $C_{\mathrm{tot}} = 43.0$, between the RNN with 42.5 and the LSTM with 44.3. Over three random initialisations its total capacity varies between 43.0 and 43.5, and over random initialisations those of the RNN and the LSTM vary between 41.6 and 42.5 and between 44.0 and 44.9. The three ranges do not overlap, so the order RNN, GRU, LSTM on the laser holds beyond the spread of the initialisation.

5.6 Capacities of all window models

The models have so far been compared on the laser. Table 2 collects the total capacities of all models that read a window on all five records, each fitted and scored on identical data. For this comparison all models, the linear ones included, are fitted on the origins $k = 63, \ldots, 2449$, whose targets lie in the training block, chosen with the origins $k = 2499, \ldots, 2949$ and scored on the 951 test origins $k = 2999, \ldots, 3949$, without the refit on the validation block used for the linear models earlier, and NVAR reads consecutive values, $\kappa = 1$. For these reasons the values for AR and NVAR differ slightly from those given before, for example 16.5 against 16.8 for AR on the laser. The networks read windows of $L = 64$ values, and AR and NVAR use window lengths chosen on the validation block.

model	Lorenz x	MG 15	MG 25	MG 35	laser
persistence	8.4	19.5	13.3	11.8	5.5
AR	15.4	49.8	33.1	38.0	16.5
NVAR	15.4	50.0	45.5	48.6	34.7
MLP	29.4	50.0	48.4	49.5	35.3
RNN	43.8	50.0	49.4	49.7	42.5
GRU	45.1	50.0	49.8	49.9	43.0
LSTM	43.8	50.0	49.9	49.9	44.3

Table 2: Total capacity C_{tot}, at most 50, on the test blocks of the five records for persistence and the models that read a window of past values, all fitted on the same origins and scored on the same 951 test origins. MG denotes Mackey–Glass with the delay τ_{MG} given.

On the periodic Mackey–Glass record every fitted model reaches or nearly reaches the maximum of 50, since a periodic signal lies in a finite invariant subspace of the window and a linear model suffices. On the irregular records the forecast map is nonlinear, and the models separate according to how much of it they can represent. On the laser and the chaotic Mackey–Glass records AR is lowest, NVAR and the MLP, with fixed or learned nonlinear features of a fixed window, are higher, and the recurrent networks, which in addition build a learned state, are highest, except on Mackey–Glass at $\tau_{\mathrm{MG}} = 35$, where the MLP and the recurrent networks all lie within half a unit of the maximum. On Lorenz AR and NVAR coincide at 15.4, the MLP reaches 29.4, and the recurrent networks reach between 43.8 and 45.1. The forecasts of AR and NVAR fall below $C = 0.5$ after 16 samples, while the Lyapunov time of the record is 55 samples, so chaos does not set this limit. The recurrent networks still reach $C(50)$ between 0.74 and 0.77, which shows that the window carries information about the state far beyond 16 samples. One possible reason, which we have not tested, is that the unmeasured coordinates y and z are complicated functions of the window, which a quadratic dictionary misses and a learned state captures. These conclusions hold for these records, this protocol and these model sizes.

Figure 28 shows the capacity of the models on the laser against the horizon. The capacity rises with the flexibility of the features, and the LSTM keeps $C(\tau) > 0.5$ up to the last horizon. The GRU, which the figure does not show, lies between the RNN and the LSTM. NVAR is still the best model at the first few horizons, with $C(1) = 0.98$ against 0.94 for the GRU. This NVAR reads $L = 32$ consecutive values, and its readout is fitted on the training block alone. The one-step capacity

$C = 0.97$ given earlier comes from the NVAR with $L = 32$ values spaced by $\kappa = 2$, whose readout was refitted on the training and validation blocks. The two values thus belong to two fits that differ in the spacing and in the block used for the readout. A possible reason is that each horizon of NVAR is a separate regression, while the networks share one state among all 50 horizons.

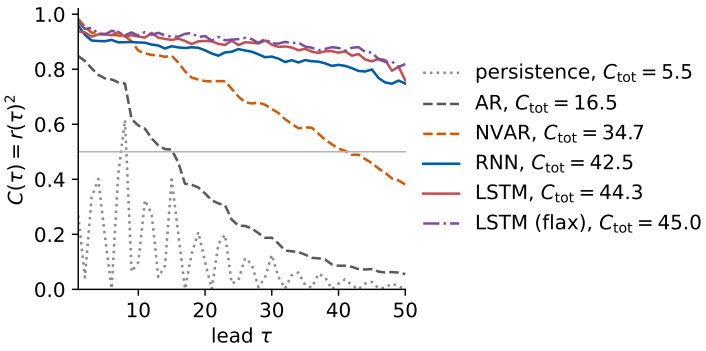


Figure 28: Capacity $C(\tau)$ on the Santa Fe test block against the horizon τ, written lead on the axis, for persistence, AR, NVAR, the RNN, the LSTM and the same LSTM forecaster built from the LSTM cell of the flax library, LSTM (flax), all fitted on the same origins and chosen on the same validation origins, with their totals C_{tot} in the legend. The thin line marks $C = 0.5$.

The direct forecasts are computed from measured windows, the inputs on which the models were trained. A free run tests a different property. From the origin $k_0 = 2999$, the last sample before the test block, every model predicts one step ahead with its $\tau = 1$ output, appends the prediction to its window and repeats for 200 steps. The recurrent networks read the shifted window of 64 values afresh from $\mathbf{h} = 0$ at every step. The error stays under 0.2, a fifth of the training range, for the first 38 steps for AR, 36 for NVAR, 9 for the MLP, 23 for the RNN, 38 for the GRU and 110 for the LSTM. NVAR follows the pulses here for 36 steps, while the NVAR free run on the laser in Figure 17 left the range of the signal after 5 steps. The two runs use different NVAR models, 32 values spaced by two samples in Figure 17 and 32 consecutive values here, fitted on different origins, and they start one sample apart. Both runs eventually diverge, the present one within the 200 steps, and the step at which an unstable polynomial map leaves the attractor depends on such details. A model trained only on measured inputs, a procedure called teacher forcing, meets its own errors for the first time in a free run, and nothing in its training controls its response to them. This mismatch between training and use is called exposure bias. The ranking comes from one origin and one initialisation per model, and chaos amplifies small differences between models, so it is a single example and no general ranking.

All models so far have learned a forecast from a scalar record alone and know nothing about the system that produced it. For many physical systems the whole state can be measured and the form of the law of motion is known, and none of the models above can use this knowledge.

6 Neural ordinary differential equations

6.1 From a forecast map to a vector field

Every model so far read a scalar record and learned a map from past values to future values for one fixed sampling interval. We now change the assumption. The whole state $\mathbf{x}$ is measured, and the law is assumed to have the form $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x})$ of equation (1) with an unknown vector field. All forecasts then follow from $\mathbf{F}$ by integration. The flow map inherits three properties from the uniqueness of the solutions, $\varphi_0 = \mathrm{id}$, $\varphi_{t+t'} = \varphi_t \circ \varphi_{t'}$ and $\varphi_{-t} = \varphi_t^{-1}$, so a single vector field fixes the forecasts for every horizon, every sampling interval and every starting time. One can therefore learn the vector field itself instead of a forecast map. This is the idea of the neural ordinary differential equation (Neural ODE) [33].

6.2 Euler steps and residual networks

Learning a vector field requires a way to turn it into a trajectory, a numerical solver. The simplest one follows from the Taylor expansion of the solution around the time $t_n = n\Delta t$,

$$\mathbf{x}(t_n + \Delta t) = \mathbf{x}(t_n) + \Delta t\, \mathbf{F}\big(\mathbf{x}(t_n)\big) + \tfrac{1}{2}\Delta t^2\, \ddot{\mathbf{x}}(t_n) + O(\Delta t^3). \tag{52}$$

Dropping the terms of second and higher order gives the explicit Euler method,

$$\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta t\, \mathbf{F}(\mathbf{x}_n), \tag{53}$$

which moves along the arrow of the field over a short step and evaluates the field again at the new point. Each step makes an error of order Δt^2, the first neglected Taylor term, called the local error. Reaching a fixed time takes a number of steps proportional to $1/\Delta t$, so the local errors add up to a global error of order Δt, and the method is of first order.

For the harmonic oscillator $\ddot{x} = -x$ with $v = \dot{x}$ the error has a systematic direction. One Euler step maps (x, v) to $(x + \Delta t\, v,\ v - \Delta t\, x)$, and the cross terms cancel in

$$(x + \Delta t\, v)^2 + (v - \Delta t\, x)^2 = (1 + \Delta t^2)(x^2 + v^2). \tag{54}$$

Every step multiplies the radius of the circular orbit by $\sqrt{1 + \Delta t^2}$, so the numerical solution spirals outwards (Figure 29, right). After one period of $2\pi/\Delta t$ steps the radius is $(1 + \Delta t^2)^{\pi/\Delta t}$ instead of one, which for 13 steps per period nearly quadruples it. The step is a linear map with the determinant $1+\Delta t^2$, so it enlarges every area of the phase plane by this factor, while the exact flow of a Hamiltonian system preserves areas by Liouville's theorem. A small change of the method, the symplectic Euler step $(x, v) \to (x + \Delta t\, v',\ v')$ with $v' = v - \Delta t\, x$, has the determinant one and preserves areas. A network trained through Euler steps would learn to compensate the artificial expansion, and the learned law would then describe the solver as much as the system.

Higher accuracy comes from evaluating the field several times within a step. The classical fourth-order Runge–Kutta method (RK4) [59] computes four slopes,

$$\begin{aligned} &\mathbf{k}_1 = \mathbf{F}(\mathbf{x}_n), && \mathbf{k}_2 = \mathbf{F}\big(\mathbf{x}_n + \tfrac{1}{2}\Delta t\, \mathbf{k}_1\big), \\ &\mathbf{k}_3 = \mathbf{F}\big(\mathbf{x}_n + \tfrac{1}{2}\Delta t\, \mathbf{k}_2\big), && \mathbf{k}_4 = \mathbf{F}\big(\mathbf{x}_n + \Delta t\, \mathbf{k}_3\big), \end{aligned} \qquad \mathbf{x}_{n+1} = \mathbf{x}_n + \frac{\Delta t}{6}\big(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4\big), \tag{55}$$

at the start of the step, twice at trial points in the middle and once at a trial point at the end. The weights $1, 2, 2, 1$ make the step agree with the Taylor expansion up to the term Δt^4, so the local error is of order Δt^5 and the global error of order Δt^4. Halving the step reduces the error sixteen-fold, at the cost of four evaluations of the field per step. We integrate every learned field with RK4 at a fixed step.

The Euler step has the same form as a block of a residual network [34], a deep network whose layers obey $\mathbf{h}_{l+1} = \mathbf{h}_l + \mathbf{f}_l(\mathbf{h}_l)$, each layer adding a learned correction to its input. If all layers share one function, $\mathbf{f}_l = \Delta t\, \mathbf{F}_\theta$, the residual network is the Euler method for the ODE $\dot{\mathbf{x}} = \mathbf{F}_\theta(\mathbf{x})$, and the depth of the network corresponds to time. In the limit of small steps the network becomes the ODE itself.

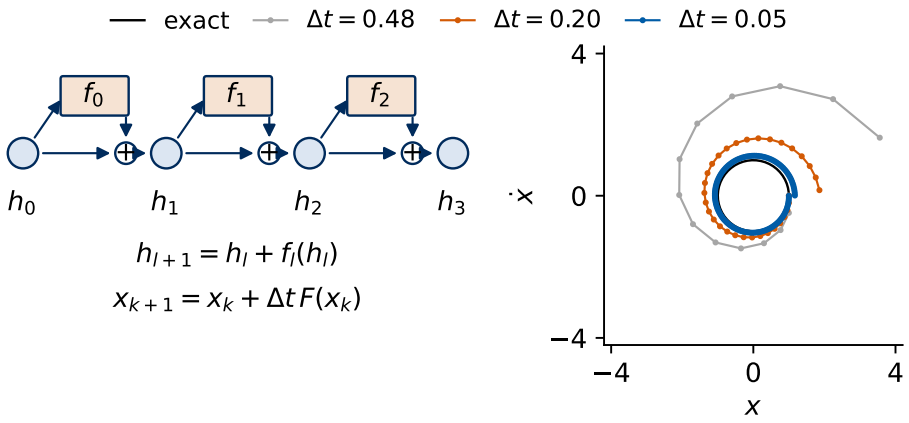


Figure 29: Residual networks and Euler steps. Left, three residual blocks $\mathbf{h}_{l+1} = \mathbf{h}_l + \mathbf{f}_l(\mathbf{h}_l)$. Right, Euler solutions of the harmonic oscillator $\ddot{x} = -x$ over one period in the phase plane $(x, \dot{x})$ for the steps $\Delta t = 0.48$, 0.20 and 0.05, together with the exact circle. The coarser the step, the further the polygon spirals outwards.

6.3 The Neural ODE

A Neural ODE takes the vector field to be a neural network $\mathbf{F}_\theta$ with weights θ,

$$\dot{\mathbf{x}} = \mathbf{F}_\theta(\mathbf{x}). \tag{56}$$

A forecast starts from a measured state and integrates this equation with RK4 to the times at which the state is wanted. The forecasts are compared with the measured states, and the weights are fitted by gradient descent on the mean squared difference. The forward pass of the model is the solver (Figure 30). The subscript θ, and θ as the argument or the variable of differentiation of a loss, denote the weights of a network, while θ elsewhere denotes an angle. The model is defined in continuous time, so it can be evaluated at any time, between samples or at irregular times, and the learned $\mathbf{F}_\theta$ is an object of the same kind as a law of motion.

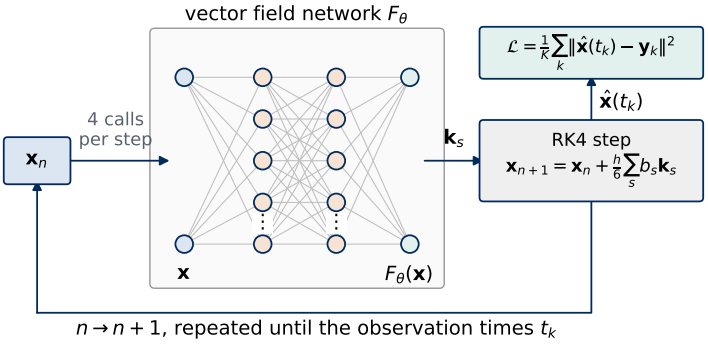


Figure 30: A Neural ODE. The network $\mathbf{F}_\theta$ receives the state and returns its time derivative. An ODE solver calls it repeatedly to integrate the state from a measured initial value to the forecast times, and the forecasts are compared with the measured states.

6.4 Learning the damped pendulum

The damped pendulum, $\dot{\theta} = \omega$, $\dot{\omega} = -\omega_p^2 \sin\theta - \gamma\omega$, with $\omega_p = 1$ and the damping rate $\gamma = 0.2$, has a known answer to compare with. Its hanging position $(0, 0)$ is a stable spiral, and its upright position $(\pi, 0)$ is a saddle. The data are eight trajectories in the phase plane, with both coordinates measured every 0.1 time units for ten time units and Gaussian noise of standard deviation $\sigma = 0.01$ (Figure 31). The trajectories are computed from the true law and then perturbed by the noise, so the true field is known and serves only to check the result. The training uses the noisy states alone. The network has two hidden layers of 64 units and 4482 parameters.

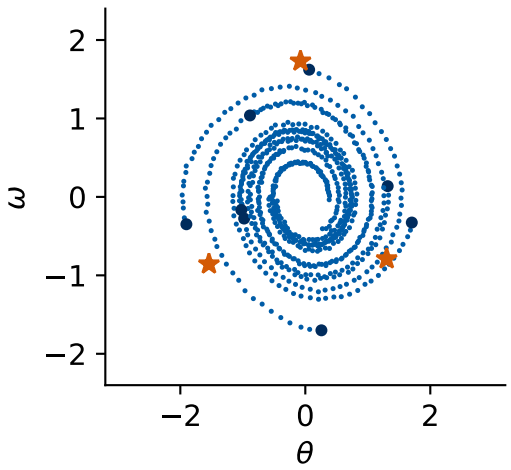


Figure 31: The training data of the damped pendulum in the state plane, angle θ against angular velocity ω. Small blue dots, the measured states of the eight training trajectories, every 0.1 time units with noise of standard deviation 0.01. Large dark dots, the first measured state of each trajectory. Orange stars, the three initial states of the test trajectories, which are not used in training.

The measurements are states. The time derivative, the object that the network has to represent, is never measured. A finite difference of two successive states, $(\mathbf{x}_{n+1} - \mathbf{x}_n)/\Delta t$, would be an estimate of it, and its noise has the standard deviation $\sqrt{2}\,\sigma/\Delta t \approx 0.14$ in each component, fourteen times the noise of the states, and it also carries the error of order Δt of the difference quotient. The Neural ODE avoids this estimate. It integrates the candidate field from a measured state and compares the integrated states with the measured ones, so the noise enters only through the states.

The network is trained on short segments, each starting at a measured state and covering the next ten samples, and the gradient of the loss with respect to the weights is obtained by differentiating through the RK4 steps of each segment. A single long integration from the first state would let a small error of the field shift the phase of all later points, and the loss would then depend on the weights in an erratic way. Restarting the integration at measured states keeps forecast and data close at every step, a simplified form of the multiple shooting of numerical analysis. Each segment starts from a noisy measured state and is compared with noisy targets, so even the true field leaves a mean squared error of about $2\sigma^2 = 2 \times 10^{-4}$, the noise of the start and that of the target added, and the final loss of the training lies close to this floor. The task thus has three stages, the measured states of Figure 31, the learned field $\mathbf{F}_\theta$ of Figure 32, and the trajectories obtained by integrating this field from new initial states in Figure 33.

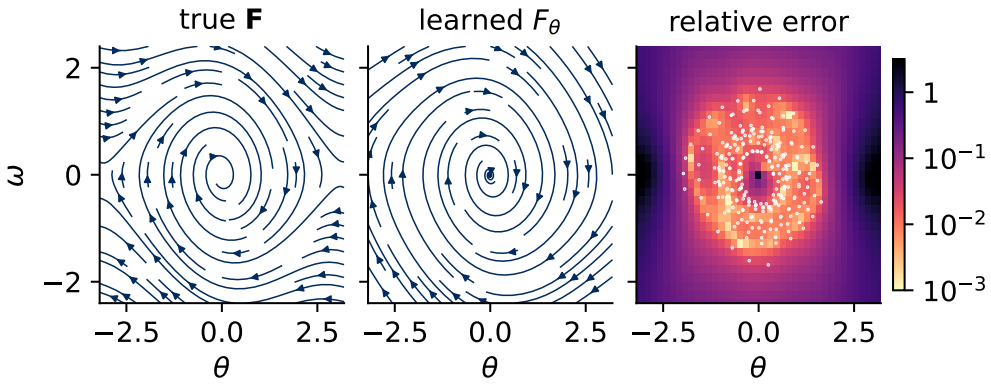


Figure 32: True and learned vector field of the damped pendulum in the state plane, angle θ and angular velocity ω. Left, streamlines of the true field $\mathbf{F}$, known because the data are synthetic and used only for this check. Middle, streamlines of the learned field $\mathbf{F}_\theta$. Right, the relative error $|\mathbf{F}_\theta - \mathbf{F}|/|\mathbf{F}|$ on a logarithmic colour scale, with the measured states as white dots.

Figure 32 compares the learned and the true field. Within a distance of 0.2 from the measured

states the median relative error is 0.012, and farther away it is 0.23. The network learns the field where the trajectories went and continues it smoothly elsewhere, and near the saddles at $\theta = \pm\pi$, where no data lie, its streamlines differ visibly from the true ones. The data constrain the field only on the region that the eight trajectories cover, an annulus around the hanging position in the right panel. Inside the innermost turn of the spirals, outside the outermost one and near the saddles the network extrapolates, and the relative error rises to order one at the saddles. Integrated from the three test states, the learned field forecasts the angle up to $t = 30$, three times the training window, with errors under 8.2×10^{-2} (Figure 33). The test trajectories start inside the region of the data and spiral into the region where the data are densest, so this forecast tests the field where the data constrain it.

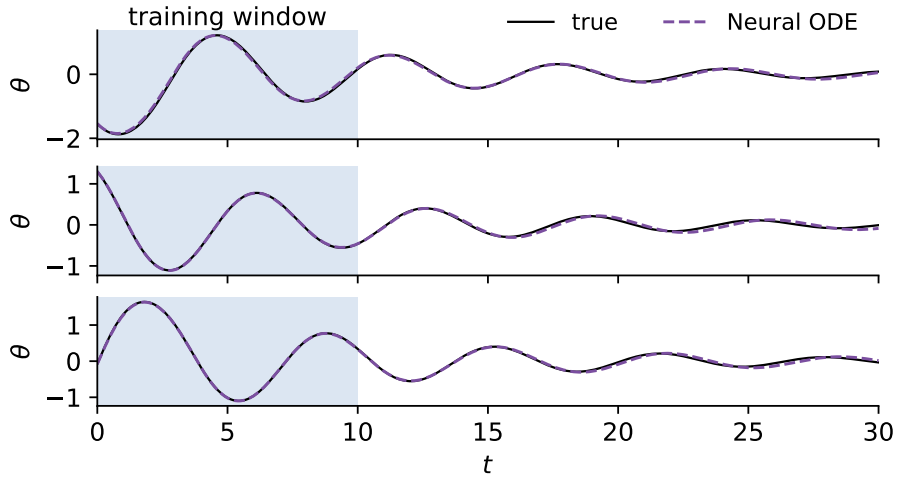


Figure 33: Trajectories from the learned field. The angle $\theta(t)$ against time for the three test initial states of Figure 31, from the true law (solid black) and from integrating the learned field $\mathbf{F}_\theta$ with RK4 (dashed). The shaded interval $0 \le t \le 10$ is the length of the training trajectories.

6.5 Gradients through the solver

The loss depends on the weights through every step of the solver. The direct way to compute its gradient differentiates through all steps. Its structure is clearest for the Euler step (53) with the field $\mathbf{F}_\theta$, which is a recurrent update of the kind met in equation (41), with the state $\mathbf{x}_n$ and without an input. With $\bar{\mathbf{x}}_n \coloneqq \partial\mathcal{L}/\partial\mathbf{x}_n$ and the Jacobian J_n of $\mathbf{F}_\theta$ at $\mathbf{x}_n$, the chain rule gives the backward recursion

$$\bar{\mathbf{x}}_n = \left(\mathbf{1} + \Delta t\, J_n^\top\right) \bar{\mathbf{x}}_{n+1} + \frac{\partial\mathcal{L}}{\partial\mathbf{x}_n}\Big|_{\text{direct}}, \qquad \frac{\partial\mathcal{L}}{\partial\theta} = \sum_n \Delta t \left(\frac{\partial\mathbf{F}_\theta}{\partial\theta}\Big|_{\mathbf{x}_n}\right)^\top \bar{\mathbf{x}}_{n+1}, \tag{57}$$

where $\mathbf{1}$ is the unit matrix and the direct term is the derivative of the loss with respect to $\mathbf{x}_n$ when $\mathbf{x}_n$ is compared with a measurement, and zero otherwise. The sensitivity is carried backwards by the transposed Jacobian of one step, as in equation (45) and Figure 20, and every step contributes to the gradient of the shared weights, as in equation (46). A model with one parameter shows what this computes. For the decay law $\dot{x} = -\alpha x$ from $x_0 = 1$, one Euler step of size 0.1 predicts $\hat{x}_1 = 1 - 0.1\alpha$, and a measured value $x_1 = 0.92$ gives the loss $\mathcal{L}(\alpha) = (0.08 - 0.1\alpha)^2$ and the derivative $d\mathcal{L}/d\alpha = -0.2\,(0.08 - 0.1\alpha) = -0.006$ at $\alpha = 0.5$. The derivative is negative, so gradient descent increases α and produces the faster decay that the data require. Backpropagation through the solver needs the states of all steps in memory, so the memory grows with the number of steps. For a two-dimensional system and a network with two hidden layers it grows from 212 kB at 50 steps to 13592 kB at 3200 steps.

The adjoint method avoids the stored trajectory [33, 35]. It is the limit $\Delta t \to 0$ of equation (57). Dividing $\bar{\mathbf{x}}_n - \bar{\mathbf{x}}_{n+1} = \Delta t\, J_n^\top \bar{\mathbf{x}}_{n+1}$ by Δt gives a differential equation for the sensitivity $\mathbf{a}(t) \coloneqq \partial\mathcal{L}/\partial\mathbf{x}(t)$.

For a loss of the final state $\mathbf{x}(T)$ it reads

$$\dot{\mathbf{a}} = -J(t)^\top \mathbf{a}, \qquad \mathbf{a}(T) = \frac{\partial \mathcal{L}}{\partial \mathbf{x}(T)}, \qquad \frac{d\mathcal{L}}{d\theta} = \int_0^T \mathbf{a}(t)^\top \frac{\partial \mathbf{F}_\theta}{\partial \theta}\Big|_{\mathbf{x}(t)} dt, \tag{58}$$

with $J(t)$ the Jacobian of $\mathbf{F}_\theta$ along the trajectory. The gradient formula, the third relation, follows from a short argument. A change $\delta\theta$ of the weights changes the trajectory by $\delta\mathbf{x}(t)$, which obeys the linearised equation $\delta\dot{\mathbf{x}} = J\,\delta\mathbf{x} + (\partial\mathbf{F}_\theta/\partial\theta)\,\delta\theta$ with $\delta\mathbf{x}(0) = 0$, since the initial state is measured. By the product rule,

$$\frac{d}{dt}(\mathbf{a}^\top \delta\mathbf{x}) = \dot{\mathbf{a}}^\top \delta\mathbf{x} + \mathbf{a}^\top \delta\dot{\mathbf{x}} = -\mathbf{a}^\top J\,\delta\mathbf{x} + \mathbf{a}^\top J\,\delta\mathbf{x} + \mathbf{a}^\top \frac{\partial \mathbf{F}_\theta}{\partial \theta}\,\delta\theta = \mathbf{a}^\top \frac{\partial \mathbf{F}_\theta}{\partial \theta}\,\delta\theta, \tag{59}$$

since the adjoint equation was chosen so that the terms with J cancel. Integrating from 0 to T gives $\mathbf{a}(T)^\top \delta\mathbf{x}(T) = \int_0^T \mathbf{a}^\top (\partial\mathbf{F}_\theta/\partial\theta)\,dt\,\delta\theta$, and the left side is $\delta\mathcal{L}$ by the chain rule. The adjoint equation is the equation for a small error, $\delta\dot{\mathbf{x}} = J\delta\mathbf{x}$, with the transposed Jacobian and run backwards. Errors propagate forwards with J and sensitivities backwards with $J^\top$. When the loss compares the forecast with measurements at several times, $\mathbf{a}$ jumps by $\partial\mathcal{L}/\partial\mathbf{x}(t_i)$ at every such time.

The state is recomputed backwards together with $\mathbf{a}$, so the memory stays constant, 25 to 26 kB for the same numbers of steps. The two gradients are those of two different problems. Backpropagation gives the gradient of the discretised loss and the adjoint method that of the continuous one, so they differ by the discretisation error, 4.2×10^{-8} at 50 steps, and approach each other in proportion to Δt^4, the order of the solver, down to 3.2×10^{-15} at 3200 steps. The backward recomputation of the state has a weakness. Running a dissipative flow backwards expands phase-space volumes and amplifies small errors, and running a chaotic flow backwards amplifies them along its contracting directions, so for such systems the reconstructed trajectory can lose accuracy. For small networks and short segments the memory of backpropagation is affordable, and we train by backpropagation through RK4.

6.6 Divergence and fixed points of the learned field

The learned pendulum field $\mathbf{F}_\theta$ is an approximation of the law of motion, valid where the data are, and it is a differentiable function of the state. Automatic differentiation gives its Jacobian $J_\theta = \partial\mathbf{F}_\theta/\partial\mathbf{x}$ at every point, and with it the quantities by which physicists characterise a law of motion.

The divergence of the field, the trace of the Jacobian, is the rate at which areas in the phase plane change. For the damped pendulum it is constant, $\nabla \cdot \mathbf{F} = -\gamma$. It vanishes for a conservative system, whose phase-space areas are preserved, and in two dimensions a vanishing divergence is the condition for the field to be Hamiltonian, $\dot{\theta} = \partial H/\partial\omega$, $\dot{\omega} = -\partial H/\partial\theta$, on a region without holes. A negative divergence marks a dissipative system. Near the data the learned field has the median divergence -0.197, with quartiles -0.229 and -0.167, against the true -0.2. Zero lies far outside this range, so the learned field identifies the pendulum as dissipative (Figure 34). In more than two dimensions a vanishing divergence only means that volumes are preserved, which every Hamiltonian flow satisfies, while the Hamiltonian structure requires more.

The zeros of the field are the fixed points, found by Newton's method. The learned field has a zero close to the hanging position, and its Jacobian there has the eigenvalues $-0.076 \pm 0.944\,\mathrm{i}$ against the true $-0.1 \pm 0.995\,\mathrm{i}$, a stable spiral as in the true system. For a damped oscillator the trace of the Jacobian is $-\gamma$ and its determinant ω_p^2, which gives $\gamma = 0.151$ and $\omega_p = 0.947$ against the true 0.2 and 1. The frequency is within about 5%, and the damping read from a single point is 25% too small, less accurate than the divergence, which averages over the whole region of the data. Near the upright position no zero is found at all. The saddle lies where there are no data, and the learned field does not contain it.

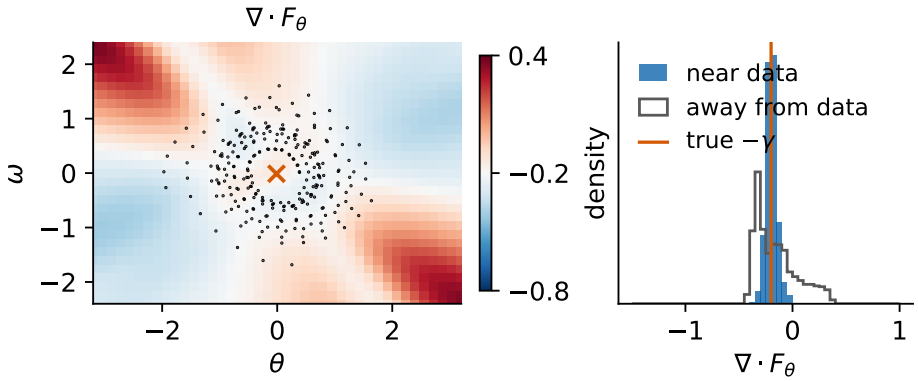


Figure 34: Divergence of the learned pendulum field. Left, $\nabla \cdot \mathbf{F}_\theta$ over the state plane, with the measured states as dots and the zero of $\mathbf{F}_\theta$ found by Newton's method as a cross. Right, the distribution of its values within a distance 0.2 of the data and farther away, with the true value $-\gamma$ as a vertical line.

6.7 A Neural ODE for the Lorenz flow

The pendulum spirals into a fixed point, and its data cover a simple region of the phase plane. A chaotic flow tests whether a learned field reproduces a motion that never settles. On Lorenz-63, with all three coordinates measured every 0.02 time units and each coordinate standardised to zero mean and unit variance, a Neural ODE with 4611 parameters is trained on segments of n_{seg} samples. Chaos limits the useful length of a segment. Over a few Lyapunov times a small change of the field changes the end of a segment by an amount comparable with the attractor, and the loss and its gradient become erratic functions of the weights. Very short segments, on the other hand, constrain only the next few steps. The forecasts are judged by the divergence time, the first lead at which the root-mean-square error over the three standardised coordinates exceeds 0.5, half the standard deviation of a coordinate, measured in Lyapunov times. On the validation block its median is 2.80 Lyapunov times for $n_{\text{seg}} = 5$, 3.30 for $n_{\text{seg}} = 10$ and 2.48 for $n_{\text{seg}} = 20$, and we use $n_{\text{seg}} = 10$.

Over the test block the median divergence time of the chosen model is 2.88 Lyapunov times. A discrete map with the same task, an MLP from one state to the next, reaches 2.71, and AR on the state 0.24. NVAR on the state, which predicts the increment $\mathbf{x}_{k+1} - \mathbf{x}_k$ from the present and past states and their pairwise products, reaches 7.69 Lyapunov times (Figure 35). This is the case of equation (39). The Lorenz field is a quadratic polynomial, the one-step map of the full state is quadratic up to corrections of order Δt^2, and the quadratic dictionary of NVAR contains it. The same NVAR on the scalar record x gained nothing over the linear model.

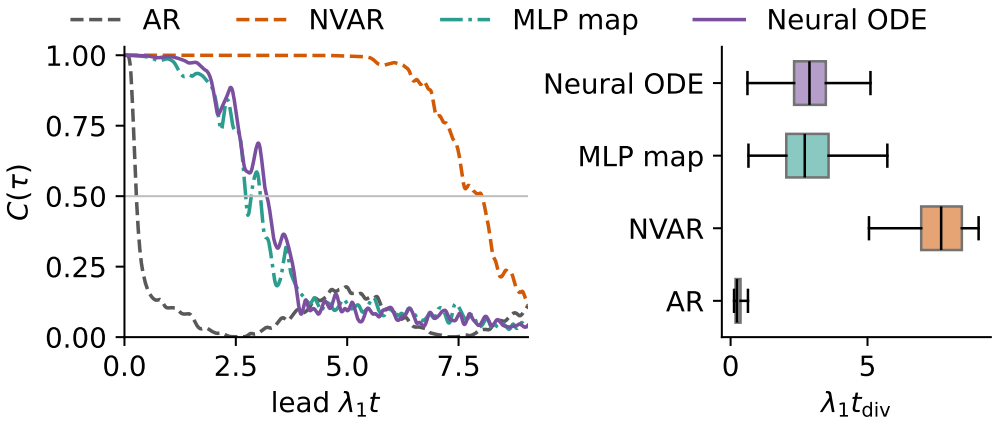


Figure 35: Forecasts of the Lorenz state by four models. Left, the capacity $C(\tau)$ against the lead in Lyapunov times $\lambda_1 \tau \Delta t$, with τ in samples and $C = 0.5$ in grey. Right, the distribution of the divergence times $\lambda_1 t_{\text{div}}$ over the test origins as box plots, the box spanning the middle half of the values and the black line marking the median.

Figure 36 shows one forecast whose divergence time is close to the median. It follows all three

coordinates for almost three Lyapunov times and departs where the true trajectory switches to the other lobe while the forecast stays on the first one.

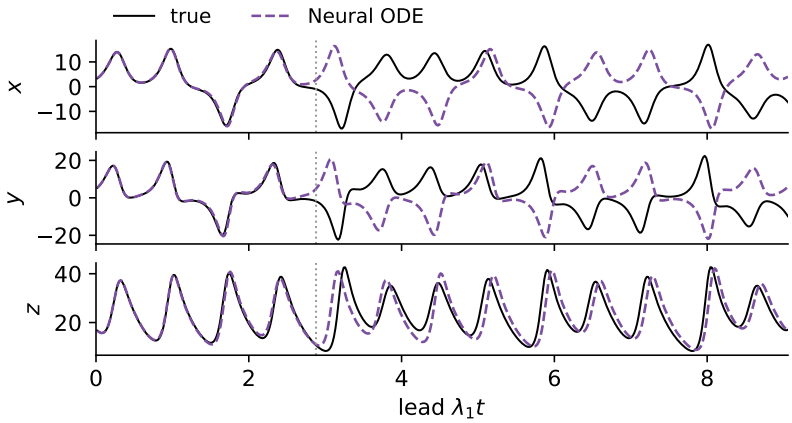


Figure 36: One Neural ODE forecast of the Lorenz state. The coordinates x, y, z of the truth and of the forecast against the lead in Lyapunov times, with the divergence time as a dotted line.

The learned flow can also be analysed as a dynamical system, and its properties fall into two groups. The properties averaged over the attractor are reproduced well. Its largest Lyapunov exponent is 0.899 ± 0.010 per time unit, in agreement with 0.906 ± 0.007 from the same estimate applied to the true Lorenz field, consistent with the value 0.907 per time unit used throughout these notes, and the mean divergence of its vector field, $\nabla \cdot \mathbf{F}_\theta$, over the test states is -13.88, close to the constant -13.67 of the Lorenz flow. A long free run of the learned field, over 400 time units, stays on an attractor with the two lobes of the true one (Figure 37), with the mean of z equal to 23.57 against 23.42 in the data and the standard deviation of x equal to 7.88 against 7.89. The local properties at the fixed points, which the trajectory visits rarely or never, are reproduced poorly. At the origin the Jacobian of the Lorenz field has the eigenvalues -22.83, -2.67 and 11.83, and that of the learned field -13.53, -3.27 and 11.03, so the strongly contracting direction is off by a factor of about 1.7. The two convection states of the Lorenz model are weakly unstable spirals, with a real part of the complex eigenvalues of about one percent of their imaginary part, and in the learned field one of them is a stable spiral. The data never stay near these points, and the weak instability is not resolved.

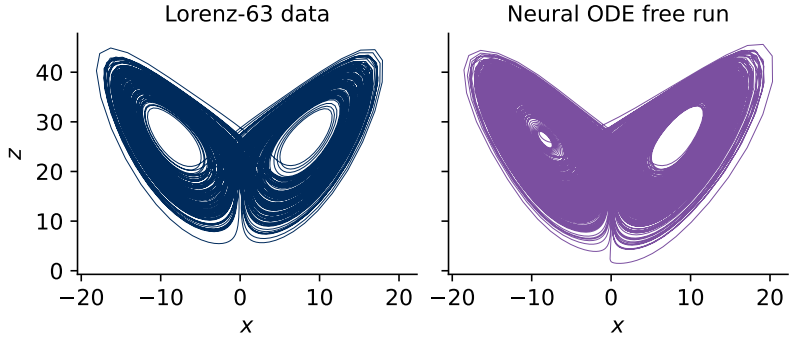


Figure 37: The Lorenz attractor in the plane of x and z. Left, the data. Right, a free run of the learned vector field over 400 time units.

6.8 Irregular sampling

A model in continuous time [36] handles irregular measurement times without change, since the solver integrates to whatever time is requested. For the pendulum observed at random times, with the same number of samples, the Neural ODE forecasts the angle with a root-mean-square error of 0.033, as accurately as when trained on the regular grid, 0.034. A discrete map assumes a fixed interval

between its input and its output. Given the gap as an extra input it reaches 0.115, and without it 0.193, since the same input is then followed by outputs at different times.

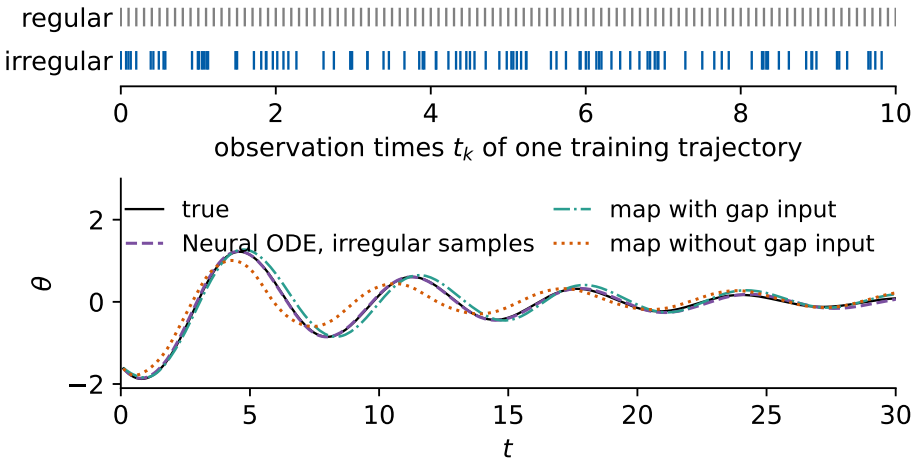


Figure 38: Irregular sampling. Top, observation times of one training trajectory, irregular (blue) and regular (grey). Bottom, the true angle $\theta(t)$ of a test trajectory and the forecasts with output step 0.1 of the Neural ODE trained on irregular samples and of two discrete maps, one with and one without the gap as input.

The Neural ODE assumes only the form of the law and learns the whole vector field, and the pendulum and the Lorenz flow showed the price. The field is reliable where the data are and uncertain elsewhere, at the saddle of the pendulum and at the fixed points of Lorenz. In many problems more is known, the law itself up to a parameter, and the data are few.

7 Physics-informed neural networks

7.1 Known laws and few measurements

The Neural ODE learned a whole vector field from many measured states, and it was reliable only where the data were. In many problems the opposite holds. The law of motion is known and the measurements are few. The damping of an oscillator follows $m_0\ddot{x} + c\dot{x} + k_s x = 0$, heat in a rod follows the heat equation, the wave function of a quantum particle follows the Schrödinger equation. Three kinds of task arise. In a forward problem the equation and its initial and boundary conditions are given and the solution is wanted. In an inverse problem a parameter of the law is unknown, a damping or a diffusivity, and has to be found from the data. In data assimilation a field is measured at a few points and has to be filled in between them. A physics-informed neural network (PINN) [37, 38, 39] treats all three with one construction, which puts the law into the loss of a network.

7.2 Fitting the data alone

The damped harmonic oscillator $m_0\ddot{x} + c\dot{x} + k_s x = 0$ with the mass $m_0 = 1$, the damping coefficient $c = 0.4$ and the spring constant $k_s = 4$, started at $x(0) = 1$ with zero velocity, has the natural angular frequency $\omega_0 = \sqrt{k_s/m_0} = 2$ and the damping ratio $\zeta = c/(2\sqrt{k_s m_0}) = 0.10$, so it oscillates with a slowly decaying amplitude and a period of 3.16 time units. Suppose its position is known at twelve evenly spaced times on $[0, 4]$, without noise, and the solution is wanted up to $t = 10$. A network $x_\theta(t)$ with the single input t, fitted to the twelve points by least squares, is an ordinary curve fit. It reproduces the data, with a largest error of 2.6×10^{-2} inside the measured window, and beyond the window it has no information, with an error of 1.2×10^{0}, as large as the oscillation itself (Figure 40).

7.3 The residual of the law

The law adds the missing information. Inserting the network into the equation of motion gives the residual

$$r_\theta(t) \coloneqq \frac{m_0\,\ddot{x}_\theta(t) + c\,\dot{x}_\theta(t) + k_s\,x_\theta(t)}{k_s}, \tag{60}$$

divided by k_s to give it the unit of x. The residual r_θ, always written with the subscript, is unrelated to the Pearson coefficient r. It measures by how much a trial function violates the law at the time t. It can be evaluated at any time, so it is computed at collocation times t_j chosen freely over the whole interval, where no measurement is needed. The time derivatives of the network are exact functions of t, obtained by automatic differentiation, the same chain rule that computes gradients with respect to the weights. For a network with one hidden layer, $x_\theta(t) = \sum_j a_j \tanh(w_j t + b_j) + a_0$, with $v_j \coloneqq \tanh(w_j t + b_j)$ it gives

$$\dot{x}_\theta(t) = \sum_j a_j w_j (1 - v_j^2), \qquad \ddot{x}_\theta(t) = -2 \sum_j a_j w_j^2\, v_j (1 - v_j^2), \tag{61}$$

and no grid or finite difference enters. The activation function must have a second derivative. A network of rectified linear units, $\max(0, \cdot)$, is piecewise linear in t, its second derivative vanishes almost everywhere, and its residual cannot represent the acceleration term of the law.

The loss combines up to three terms,

$$\begin{aligned}\mathcal{L}(\theta) = {} & \frac{w_r}{N_r} \sum_{j=1}^{N_r} r_\theta(t_j)^2 + w_{\mathrm{IC}} \Big[\big(x_\theta(0) - x_0\big)^2 + \frac{\big(\dot{x}_\theta(0) - v_0\big)^2}{\omega_0^2} \Big] \\ & + \frac{w_d}{N_d} \sum_{i=1}^{N_d} \big(x_\theta(t_i) - x_i\big)^2,\end{aligned} \tag{62}$$

the mean squared residual at N_r collocation times, a penalty on the deviation from the initial position x_0 and velocity v_0, where the division by ω_0^2 gives the velocity term the units of a squared position, and the misfit to N_d measured pairs (t_i, x_i). The weights w_r, w_{IC} and w_d are set to one or to zero. The forward problem uses the residual and the initial-condition term. The fits to measured positions use the residual and the data term, and the data at early times then take the place of the initial condition (Figure 39).

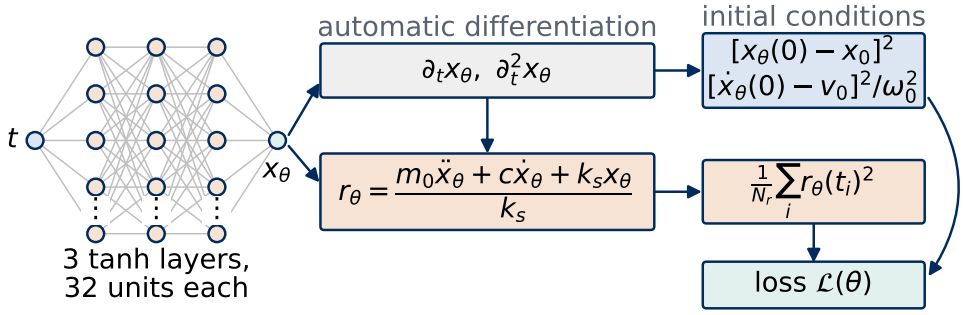


Figure 39: A physics-informed neural network for the oscillator. The network with input t returns $x_\theta(t)$, automatic differentiation gives $\dot{x}_\theta$ and $\ddot{x}_\theta$, these form the residual r_θ of the equation of motion at the collocation times, and the loss adds the mean squared residual to the data or initial-condition terms.

What the residual enforces follows from the theory of ordinary differential equations. A function whose residual vanishes at every time and which has the correct initial position and velocity is the solution, since the solution of the initial-value problem is unique. Two trial functions show why both conditions are needed. The function $x = 0$ has zero residual everywhere and violates the initial condition $x(0) = 1$. The function $x = \cos 2t$ meets both initial conditions, and its residual, $(-4\cos 2t - 0.8\sin 2t + 4\cos 2t)/4 = -0.2\sin 2t$, does not vanish, since it is the undamped oscillation and misses the friction term. A residual that vanishes only at finitely many collocation times is a

weaker condition, since a function can pass through zero residual at these times and violate the law in between, and the optimiser, which only lowers the loss, may also stop at a function with a small but nonzero residual. The law enters as a penalty, so it is satisfied approximately, to the extent the optimisation succeeds. A trained PINN represents one solution, so a new initial condition requires a new training.

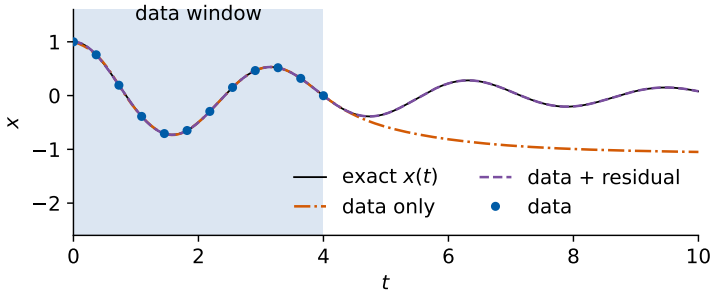


Figure 40: The damped oscillator from twelve measured positions. The exact solution $x(t)$, the measurements on $[0, 4]$ (shaded window), the network fitted to the data alone (dash-dotted) and the network fitted to the data together with the residual of the law on $[0, 10]$ (dashed).

With the same twelve points and the same network, adding the residual on 200 collocation times over $[0, 10]$ lowers the error beyond the measured window from 1.2×10^{0} to 3.0×10^{-4} (Figure 40). The residual requires the solution to obey the law on the whole interval, and the data on $[0, 4]$ select among the solutions of the law, so the law carries the solution across the interval where nothing was measured.

7.4 A PINN against a classical solver

For a forward problem with known initial conditions the PINN competes with classical solvers. The PINN for the oscillator reaches a largest error of 1.2×10^{-3} after about 2.6×10^{6} evaluations of the residual during training. RK4 with the step 0.1 reaches 5.0×10^{-5} with 400 evaluations of the right-hand side, in a fraction of a millisecond (Figure 42). Training a PINN is itself hard. Adam, the adaptive gradient method of the networks above, stalls at an error of 1.5×10^{-1}. A second optimiser, run afterwards, is needed to reach the final accuracy. It is the limited-memory Broyden–Fletcher–Goldfarb–Shanno method (L-BFGS) [60], a quasi-Newton method. Newton's method divides the gradient by the matrix of second derivatives of the loss, which a network with thousands of weights makes too costly. L-BFGS builds an approximation of the inverse of this matrix from the changes of the gradient over the last few steps and uses it to choose the direction and length of each step (Figure 41).

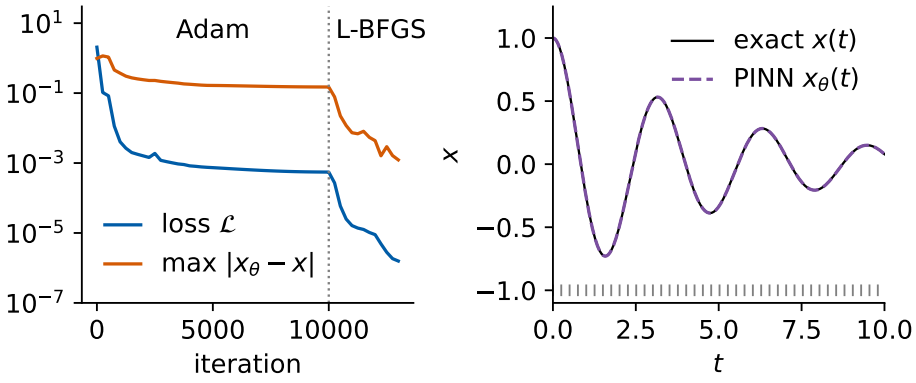


Figure 41: Training a PINN for the oscillator. Left, the loss $\mathcal{L}$ and the largest error $\max_t |x_\theta - x|$ against the iteration, first with Adam and then with L-BFGS. Right, the exact solution and the PINN solution, with the collocation times as ticks.

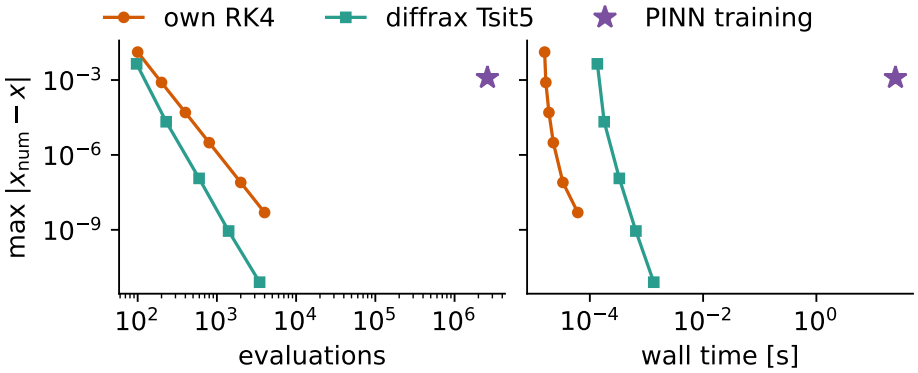


Figure 42: Cost and accuracy for the oscillator. The largest error of the solution against the number of evaluations of the right-hand side (left) and against the wall time (right), for RK4 with several steps, an adaptive solver and the training of the PINN.

7.5 An unknown damping

A forward problem leaves the PINN behind a classical solver, and an inverse problem uses what the PINN offers in addition, a parameter of the law fitted together with the solution. If the damping c of the oscillator is unknown, it becomes a trainable parameter next to the weights, written as $c = e^{\alpha}$ with a trainable α, which keeps it positive. The data are the positions at 25 equally spaced times on $[0, 10]$ with Gaussian noise of standard deviation 0.02, and the initial state is treated as unknown, so the loss of equation (62) contains the residual and the data term. The residual ties the unknown parameter to the shape of the solution. The data decide the shape, and only a solution that oscillates carries information on c, since the trivial solution $x = 0$ makes the residual vanish for every damping. From the noisy positions the PINN finds $c = 0.401 \pm 0.006$ over five noise realisations, against the true 0.4 (Figure 43). A least-squares fit of the exact solution of the oscillator to the same data gives 0.400 ± 0.004, so the PINN reaches the accuracy of the classical fit without using the solution formula.

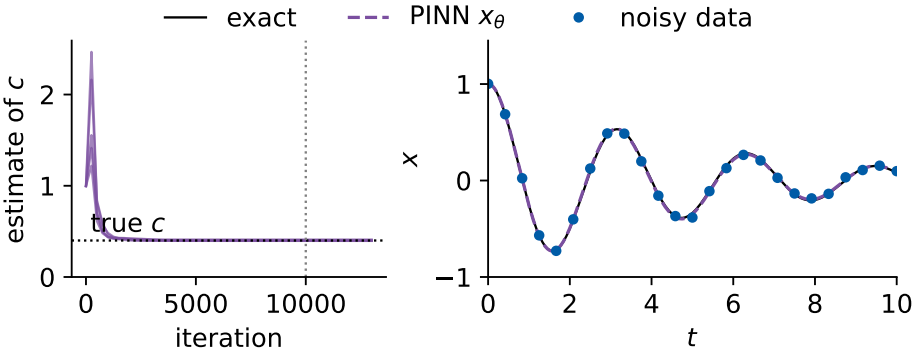


Figure 43: Unknown damping of the oscillator. Left, the estimate of c during training for five realisations of the noise, with the true value as a dashed line. Right, the noisy data, the exact solution and the PINN solution $x_\theta(t)$.

7.6 Diffusion in a rod

The oscillator has one unknown function of one variable. A field that depends on space and time needs a grid in a classical solver, and a PINN treats the extra variable as one more input of the network. The temperature $u(x, t)$ along a rod of unit length whose ends are held at zero obeys the heat equation

$$\partial_t u = D\,\partial_x^2 u, \qquad u(0,t) = u(1,t) = 0, \qquad u(x,0) = \sin \pi x + \tfrac{1}{2} \sin 3\pi x, \tag{63}$$

where D is the thermal diffusivity, here $D = 0.1$, and x denotes the position along the rod. In this subsection and the next two, u denotes the temperature, a function unrelated to the window $\mathbf{u}_k$. Each

sine profile $\sin n\pi x$ keeps its shape and decays exponentially, since $\partial_x^2 \sin n\pi x = -n^2\pi^2 \sin n\pi x$, so the solution is

$$u(x,t) = e^{-D\pi^2 t} \sin \pi x + \tfrac{1}{2} e^{-9D\pi^2 t} \sin 3\pi x. \tag{64}$$

Mode n decays on the time scale $\tau_n = 1/(Dn^2\pi^2)$, which gives $\tau_1 = 1.013$ and $\tau_3 = 0.113$. The mode with three half-waves decays nine times faster than the single arch, and the profile becomes a single arch (Figure 44).

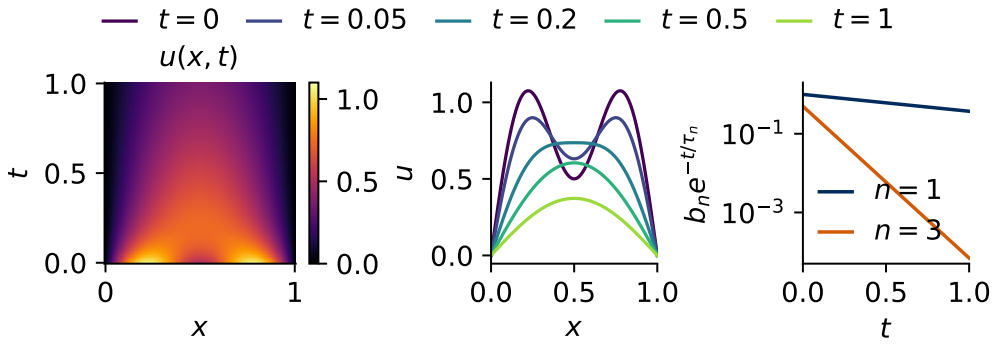


Figure 44: The heat equation in a rod. Left, the temperature $u(x,t)$ over position x and time t. Middle, temperature profiles at several times. Right, the amplitudes of the modes $n = 1$ and $n = 3$ against time on a logarithmic scale.

7.7 A PINN for the heat equation

The PINN for equation (63) is a network $u_\theta(x,t)$ with two inputs. Its residual is $\partial_t u_\theta - D\,\partial_x^2 u_\theta$, evaluated on 1681 collocation points spread over the rod and the time interval, and the loss adds the squared deviations from the initial profile and from the boundary values. All derivatives come from automatic differentiation in the two inputs. With 2241 parameters the PINN reaches a largest error of 9.5×10^{-4} after 44 seconds of training (Figure 45).

The classical reference is the Crank–Nicolson method, which replaces $\partial_x^2 u$ by differences on a grid of points along the rod and steps in time with the average of the old and the new right-hand side, a scheme that is stable for any time step. With 20 grid points it reaches 6.8×10^{-4} in 5.2×10^{-5} seconds, a smaller error than that of the PINN, and with 160 points 1.1×10^{-5} in 9.5×10^{-4} seconds, about 89 times more accurate than the PINN. Both grids are faster than the training of the PINN by between four and six orders of magnitude, so for a forward problem with known initial and boundary conditions the classical solver is the better tool.

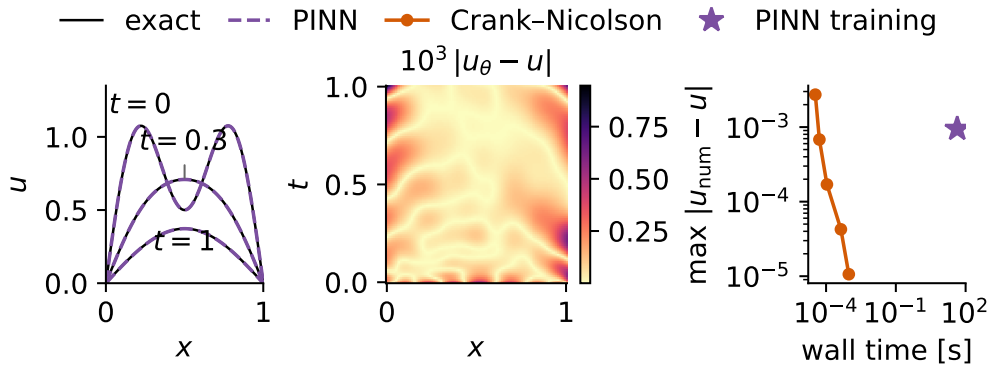


Figure 45: The PINN for the heat equation. Left, exact profiles and PINN profiles at several times. Middle, the error $|u_\theta - u|$ over position and time on a logarithmic colour scale. Right, the largest error against the wall time for Crank–Nicolson with several grid sizes and for the PINN.

7.8 Diffusivity from four thermometers

As for the oscillator, the inverse problem is where the PINN uses its additional freedom. Suppose the diffusivity is unknown and the rod carries 4 thermometers, each read 20 times with noise of standard deviation 0.01. The diffusivity becomes a trainable parameter, started at $D = 0.3$, and the loss combines the readings, the residual and the initial and boundary terms. The PINN finds $D = 0.09983 \pm 0.00057$ over five noise realisations, and at the same time the whole temperature field between the thermometers, with a largest error between 1.6×10^{-3} and 3.3×10^{-3} over the realisations (Figure 46). A least-squares fit of D through the Crank–Nicolson solver gives 0.09978 ± 0.00052, equally good, because here the initial profile is known. A PINN may gain when such information is missing and the field itself has to be reconstructed from scattered measurements, a case these runs do not test.

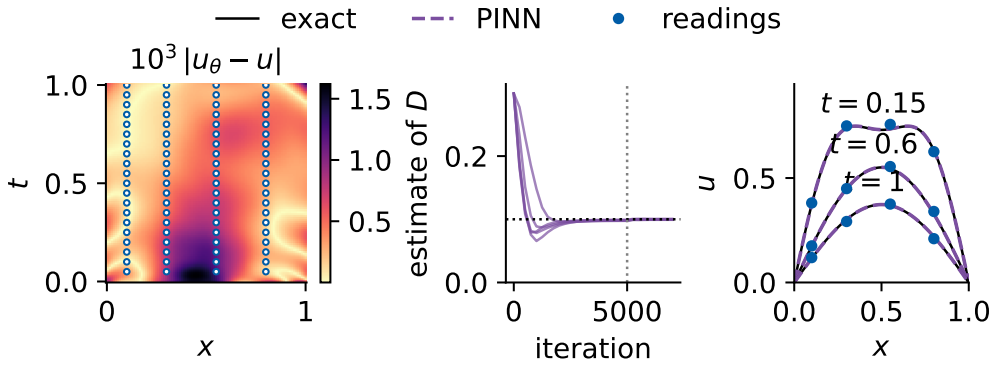


Figure 46: Diffusivity from four thermometers. Left, the error $|u_\theta - u|$ of the field reconstructed by the PINN, multiplied by 10^3, over position x and time t for the first noise realisation, with the 80 thermometer readings as white dots. Middle, the estimate of D during training for the five realisations, with the true value dotted and the switch from Adam to L-BFGS as a vertical line. Right, reconstructed profiles (dashed) and the exact solution at three reading times, with the readings at these times as dots.

7.9 A quantum particle in a harmonic trap

The oscillator and the rod have real unknowns. In quantum mechanics the unknown is a complex wave function, its evolution conserves the total probability, and part of its content, the phase, leaves no trace in the probability density. A particle in a harmonic trap shows all three features, and its evolution is known in closed form, so every prediction of a network can be compared with the true answer.

A particle of mass m_0 in the potential $\frac{1}{2} m_0 \omega^2 x^2$ with the trap frequency ω, which in the quantum examples replaces the meaning of ω as the angular velocity of the pendulum, is described by its wave function $\psi(x, t)$, a complex number at every position x and time t. Its squared modulus $|\psi(x, t)|^2$ is the probability density of finding the particle at x when its position is measured at time t. We measure lengths in units of the oscillator length $\sqrt{\hbar/(m_0\omega)}$, times in units of $1/\omega$ and energies in units of $\hbar\omega$, which sets $\hbar = m_0 = \omega = 1$. The time-dependent Schrödinger equation then reads

$$\mathrm{i}\,\partial_t \psi = \hat{H}\psi, \qquad \hat{H} := -\tfrac{1}{2}\partial_x^2 + V(x), \qquad V(x) = \tfrac{1}{2}x^2, \tag{65}$$

where the Hamiltonian $\hat{H}$ is the sum of the kinetic energy $-\frac{1}{2}\partial_x^2$ and the potential energy V. The equation is of first order in time, so the wave function at one time fixes it at all later times. It is also linear in ψ, so a sum of solutions is again a solution.

The simplest solutions are the stationary states $\psi(x, t) = \psi_n(x)\, e^{-\mathrm{i}E_n t}$, whose density does not change in time. Inserting this form into equation (65) gives the eigenvalue equation $\hat{H}\psi_n = E_n \psi_n$. For the harmonic trap its normalised solutions are the Hermite functions, with the energies $E_n = n + \frac{1}{2}$,

as derived in the textbook of Griffiths and Schroeter [44]. The two lowest are

$$\psi_0(x) = \pi^{-1/4} e^{-x^2/2}, \qquad \psi_1(x) = \sqrt{2}\, x\, \psi_0(x), \tag{66}$$

a Gaussian without a node and an odd function with one node at $x = 0$. Both are real and orthonormal, $\int \psi_m \psi_n \, dx = \delta_{mn}$.

By linearity, a superposition of stationary states evolves by giving each component its own phase. The state $\psi(x, 0) = (\psi_0 + \psi_1)/\sqrt{2}$ becomes

$$\psi(x, t) = \tfrac{1}{\sqrt{2}} (e^{-\mathrm{i}E_0 t} \psi_0(x) + e^{-\mathrm{i}E_1 t} \psi_1(x)), \tag{67}$$

and since ψ_0 and ψ_1 are real, its density is

$$|\psi(x, t)|^2 = \tfrac{1}{2}[\psi_0(x)^2 + \psi_1(x)^2] + \psi_0(x)\, \psi_1(x) \cos\left((E_1 - E_0)\, t\right). \tag{68}$$

The first term is constant, and the interference term oscillates with the angular frequency $E_1 - E_0 = 1$. The product $\psi_0 \psi_1$ is positive for $x > 0$ and negative for $x < 0$, so the density leans to the right at $t = 0$ and to the left at $t = \pi$, and it returns to its initial form after the period 2π. The mean position follows from the same expression. The integrals $\int x\, \psi_n^2 \, dx$ vanish because ψ_n^2 is even, and $\int x\, \psi_0 \psi_1 \, dx = \sqrt{2/\pi} \int x^2 e^{-x^2} dx = 1/\sqrt{2}$, so that

$$\langle x \rangle(t) \coloneqq \int x\, |\psi(x, t)|^2 \, dx = \frac{\cos t}{\sqrt{2}}. \tag{69}$$

The wave packet moves back and forth in the trap at the trap frequency, as a classical particle would. The norm $N(t) \coloneqq \int |\psi(x, t)|^2 dx$, the total probability, stays equal to 1 because ψ_0 and ψ_1 are orthogonal. The energy $\langle \psi | \hat{H} | \psi \rangle = \frac{1}{2} E_0 + \frac{1}{2} E_1 = 1$ stays constant as well, because each stationary component keeps its weight $\frac{1}{2}$ (Figure 47).

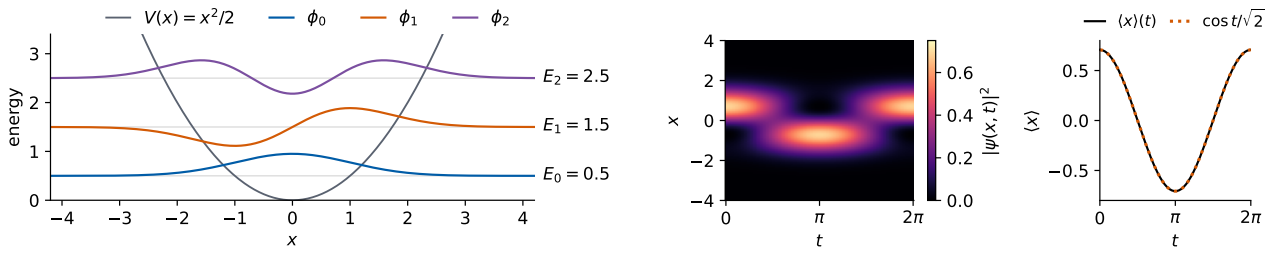


Figure 47: The harmonic trap. Left, the potential $V(x) = x^2/2$, the energies E_n as horizontal lines and the three lowest eigenfunctions $\psi_n(x)$, scaled by 0.6 and shifted vertically by E_n. Right, the exact density $|\psi(x, t)|^2$ of the superposition $(\psi_0 + \psi_1)/\sqrt{2}$ as a map with time t on the horizontal and position x on the vertical axis, and its mean position $\langle x \rangle(t)$ together with the curve $\cos t/\sqrt{2}$.

7.10 Snapshots of a wave function

Suppose that the wave function of the trapped particle has been recorded at a few early times and its evolution over a full period is wanted. A wave function recorded at one time is called a snapshot. One position measurement on one particle gives a single position, and the density $|\psi(x, t_k)|^2$ follows only from many measurements on identically prepared particles. The complex values of $\psi(x, t_k)$, including the phase, require a reconstruction of the quantum state from many measurements of different observables, called quantum state tomography. Even tomography returns the state only up to a global phase factor $e^{\mathrm{i}\alpha}$, since no measurement can detect such a factor. Our snapshots are synthetic. They are computed from equation (67), so they carry the complex values of the exact solution, including its global phase at each time, and they idealise what an experiment can deliver.

The task uses three snapshots at the times $t_k = 0, 0.5, 1$, all within the first sixth of the period. All computations use the box $-L_x \le x < L_x$ with $L_x = 6$, discretised by 256 equally spaced points,

and a snapshot contains ψ at every second of these points, 128 positions x_j. Every value carries independent Gaussian noise of standard deviation $\sigma = 0.01$ on its real and on its imaginary part,

$$\psi_{jk} = \psi(x_j, t_k) + \sigma\,(\varepsilon_{jk} + \mathrm{i}\,\varepsilon'_{jk}), \qquad \varepsilon_{jk},\ \varepsilon'_{jk} \sim \mathcal{N}(0,1), \tag{70}$$

which gives $N_d = 384$ complex data values. A prediction ψ_θ is judged by its largest deviation from the exact wave function at the time t,

$$e(t) \coloneqq \max_x |\psi_\theta(x,t) - \psi(x,t)|, \tag{71}$$

evaluated on the 256 grid points at 129 equally spaced times in $0 \le t \le 2\pi$. The largest error inside the snapshot window is the maximum of $e(t)$ over $t \le 1$, and the largest error after the snapshots is its maximum over $t > 1$. The modulus $|\psi|$ of the exact solution never exceeds 0.83, and an error of this size means that the prediction carries no information.

7.11 A network for a complex wave function

A network returns real numbers, so the complex wave function is built from two real functions. A multilayer perceptron with the two inputs x/L_x and $t/\pi - 1$, both rescaled to the interval $[-1, 1]$, four hidden layers of 50 tanh units and a linear output layer returns the two numbers $n_\theta^{\mathrm{re}}(x,t)$ and $n_\theta^{\mathrm{im}}(x,t)$. It has 7902 parameters. Its outputs are multiplied by a fixed Gaussian envelope,

$$u_\theta(x,t) \coloneqq e^{-x^2/4}\, n_\theta^{\mathrm{re}}(x,t), \qquad v_\theta(x,t) \coloneqq e^{-x^2/4}\, n_\theta^{\mathrm{im}}(x,t), \qquad \psi_\theta \coloneqq u_\theta + \mathrm{i}\, v_\theta. \tag{72}$$

where u_θ and v_θ are the real and imaginary parts of the predicted wave function, unrelated to the temperature u and the window $\mathbf{u}_k$. The envelope makes ψ_θ decay at large $|x|$, as every bound state of the trap does. It is wider than the ground state $\psi_0 \propto e^{-x^2/2}$ by the factor $\sqrt{2}$, so the network still has to produce the Gaussian shape of ψ_0 and ψ_1 itself. The envelope fixes where ψ_θ can be large and leaves its normalisation free, so the amplitude of ψ_θ has to come from the data. At the edges of the box the envelope equals $e^{-9} \approx 1.2 \times 10^{-4}$, so a boundary term that keeps $\psi_\theta(\pm L_x, t)$ small adds little to what the envelope already enforces (Figure 48).

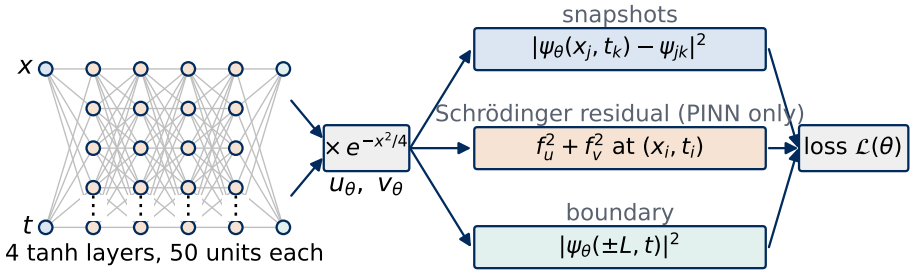


Figure 48: The network for the Schrödinger equation. A multilayer perceptron with inputs (x,t) returns two numbers, the envelope $e^{-x^2/4}$ turns them into u_θ and v_θ, and $\psi_\theta = u_\theta + \mathrm{i}v_\theta$ enters three loss terms, the misfit to the snapshot values ψ_{jk}, the residual of the Schrödinger equation at collocation points, used by the PINN only, and the boundary term $|\psi_\theta(\pm L_x, t)|^2$.

7.12 Fitting the snapshots alone

The first model, called model A, fits the snapshots with the loss

$$\begin{aligned}\mathcal{L}_{\mathrm{A}}(\theta) = \frac{1}{N_d}\sum_{j,k} \left|\psi_\theta(x_j,t_k) - \psi_{jk}\right|^2 \\ + \frac{1}{2N_b}\sum_{b=1}^{N_b} \Big(|\psi_\theta(L_x,t_b)|^2 + |\psi_\theta(-L_x,t_b)|^2\Big),\end{aligned} \tag{73}$$

where the first sum runs over all snapshot values and the second over $N_b = 100$ equally spaced times t_b in $[0, 2\pi]$. Training uses 6000 steps of Adam followed by 2000 iterations of L-BFGS and takes 15 seconds.

Inside the snapshot window the fit is good, with a largest error of 1.3×10^{-2}, comparable with the noise. After the last snapshot the error reaches 1.2×10^{0}, the size of ψ itself (Figure 49, left). The density moves towards negative x and stays there instead of returning, the mean position leaves the curve $\cos t/\sqrt{2}$ soon after the last snapshot, and the norm wanders between 0.719 and 1.652, so even the total probability is lost. Nothing in the loss refers to times after $t = 1$, and the network continues the data in a smooth way set by its architecture and its initial weights. Three snapshots fix the wave function at three times, and without a law of motion they contain no information about later times.

7.13 The Schrödinger residual

The law of motion supplies this information. Inserting ψ_θ into equation (65) gives the complex residual

$$r_\theta \coloneqq \mathrm{i}\,\partial_t \psi_\theta - \hat{H}\psi_\theta = \mathrm{i}\,\partial_t \psi_\theta + \tfrac{1}{2}\partial_x^2 \psi_\theta - V\psi_\theta. \tag{74}$$

A loss needs real numbers, so the residual is split into its real and imaginary parts. The time derivative gives $\mathrm{i}\,\partial_t \psi_\theta = \mathrm{i}\,\partial_t u_\theta - \partial_t v_\theta$. The Hamiltonian contains only real coefficients, since the potential V is real, so it maps a real function to a real function, and $\hat{H}\psi_\theta = \hat{H}u_\theta + \mathrm{i}\,\hat{H}v_\theta$ with real $\hat{H}u_\theta$ and $\hat{H}v_\theta$. Collecting the real and the imaginary terms gives

$$\begin{aligned} f_u &\coloneqq \operatorname{Re} r_\theta = -\partial_t v_\theta - \hat{H}u_\theta = -\partial_t v_\theta + \tfrac{1}{2}\partial_x^2 u_\theta - V\,u_\theta, \\ f_v &\coloneqq \operatorname{Im} r_\theta = \partial_t u_\theta - \hat{H}v_\theta = \partial_t u_\theta + \tfrac{1}{2}\partial_x^2 v_\theta - V\,v_\theta. \end{aligned} \tag{75}$$

Both vanish at every point if and only if ψ_θ solves the Schrödinger equation. For a solution the two conditions $f_u = 0$ and $f_v = 0$ read

$$\partial_t u = \hat{H}v, \qquad \partial_t v = -\hat{H}u, \tag{76}$$

two coupled real equations of first order in time, in which the imaginary part drives the change of the real part and the real part drives the change of the imaginary part. The derivatives ∂_t and ∂_x^2 of u_θ and v_θ are computed by automatic differentiation through the envelope and the network.

The residual vanishes for every solution of the Schrödinger equation, and it therefore cannot single out the one solution that was measured. It is linear in the wave function, $r[a\psi + b\chi] = a\,r[\psi] + b\,r[\chi]$ for complex numbers a and b, so the functions with zero residual form a vector space. This space contains every stationary state $\psi_n(x)e^{-\mathrm{i}E_n t}$, every superposition of stationary states and the function $\psi = 0$. The equation is of first order in time, so a solution is fixed by the wave function at one time, and the residual leaves this information open. The residual states how a wave function moves, and the snapshots state which wave function it is. Model A has the snapshots without the law, and the PINN combines both.

The PINN, model B, adds the mean squared residual to the loss of model A,

$$\mathcal{L}_{\mathrm{B}}(\theta) = \mathcal{L}_{\mathrm{A}}(\theta) + \frac{1}{N_r}\sum_{i=1}^{N_r}\Big[f_u(x_i, t_i)^2 + f_v(x_i, t_i)^2\Big], \tag{77}$$

with $N_r = 3072$ collocation points (x_i, t_i) drawn uniformly from $[-L_x, L_x] \times [0, 2\pi]$, the whole space and time window. It starts from the same initial weights as model A, uses the same optimisers and trains in 73 seconds.

With the residual the largest error is 6.7×10^{-3} inside the snapshot window and 4.0×10^{-3} after it, so the error does not grow when the network leaves the data (Figure 49, right). The density returns to its initial form at $t = 2\pi$, the mean position completes one oscillation, and the norm stays between 0.999 and 1.002. The residual carries the information of the snapshots through the whole period.

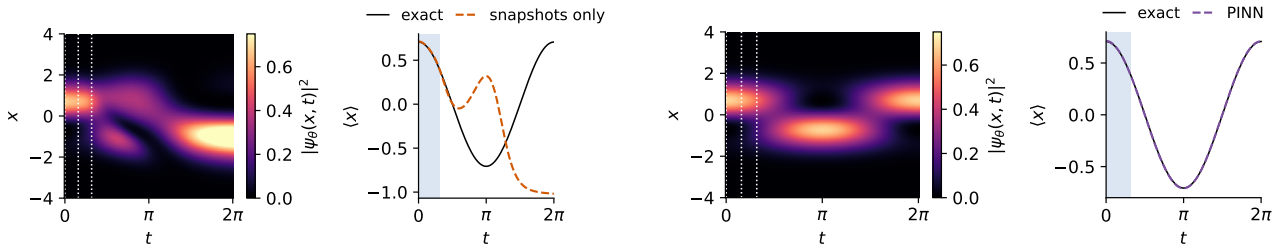


Figure 49: The harmonic trap from three snapshots. Left pair, model A, fitted to the snapshots alone. Right pair, model B, the PINN with the Schrödinger residual. In each pair, the map shows the density $|\psi_\theta(x,t)|^2$ over time and position on the colour scale of Figure 47, with the snapshot times as dotted white lines, the one at $t = 0$ lying on the axis. The curves show the mean position $\langle x\rangle(t)$ of the model and the exact one, with the snapshot window $0 \le t \le 1$ shaded.

7.14 Norm, energy and phase

The density, the norm and the mean position are all built from $|\psi|^2$, and none of them tests the phase. Two wave functions that differ by a phase factor that depends on the position have the same density at one time, and they evolve differently afterwards, since the gradient of the phase sets the probability current. The energy expectation value is sensitive to the phase. For a wave function of any norm it is

$$E(t) := \frac{\langle\psi_\theta|\hat{H}|\psi_\theta\rangle}{\langle\psi_\theta|\psi_\theta\rangle} = \frac{\int \left[\tfrac{1}{2}|\partial_x\psi_\theta|^2 + V\,|\psi_\theta|^2\right]dx}{\int |\psi_\theta|^2 dx}, \tag{78}$$

where the second form follows from integration by parts, since ψ_θ vanishes at the edges of the box. Dividing by the norm makes $E(t)$ the energy per unit probability, so that an error in the norm does not enter it. The kinetic term $|\partial_x\psi_\theta|^2$ contains the phase gradient, and for the exact evolution $E(t) = 1$ at all times. A second test compares the real and imaginary parts of ψ_θ at $t^* = 3\pi/2$, three quarters of the period and far after the last snapshot at $t = 1$, where the exact wave function is $(e^{-3\pi\mathrm{i}/4}\psi_0 + e^{-9\pi\mathrm{i}/4}\psi_1)/\sqrt{2}$.

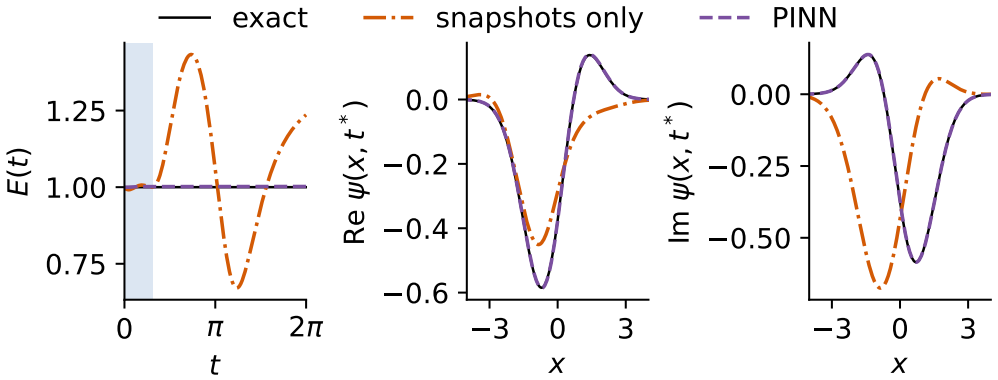


Figure 50: Energy and phase. Left, the energy $E(t)$ of equation (78) for the exact solution, model A and the PINN, with the snapshot window shaded. Middle and right, the real and imaginary parts of $\psi(x,t^*)$ at $t^* = 3\pi/2$ for the exact solution and the two models.

The PINN keeps the energy within 2.3×10^{-3} of 1 and reproduces both parts of the wave function at t^*, with a largest error of 2.8×10^{-3} (Figure 50). Model A has the correct energy only near the data, later its energy swings between 0.67 and 1.43, and its wave function at t^* is wrong by 7.8×10^{-1}.

Neither conservation law appears in the loss. For the exact evolution the conservation of the norm follows from the Schrödinger equation and the reality of the potential. Written for the time derivative, equation (65) reads $\partial_t\psi = \frac{\mathrm{i}}{2}\partial_x^2\psi - \mathrm{i}V\psi$, and its complex conjugate $\partial_t\psi^* = -\frac{\mathrm{i}}{2}\partial_x^2\psi^* + \mathrm{i}V\psi^*$, where the real V is unchanged by conjugation. The density then changes as

$$\partial_t|\psi|^2 = \psi^*\partial_t\psi + \psi\,\partial_t\psi^* = \tfrac{\mathrm{i}}{2}(\psi^*\partial_x^2\psi - \psi\,\partial_x^2\psi^*) = -\partial_x j, \qquad j := \mathrm{Im}\,(\psi^*\partial_x\psi), \tag{79}$$

since the two potential terms $-\mathrm{i}V|\psi|^2$ and $+\mathrm{i}V|\psi|^2$ cancel, and $\psi^*\partial_x^2\psi - \psi\,\partial_x^2\psi^* = \partial_x(\psi^*\partial_x\psi - \psi\,\partial_x\psi^*) = 2\mathrm{i}\,\partial_x j$. Equation (79) is a continuity equation, and j is the probability current, which contains the phase gradient. Integrating it over the box gives

$$\frac{dN}{dt} = \int_{-L_x}^{L_x} \partial_t|\psi|^2\,dx = -\big[j\big]_{x=-L_x}^{x=L_x} = 0 \tag{80}$$

for a wave function that vanishes at the edges. Probability moves from place to place through the current and is neither created nor destroyed. A complex potential $V - \mathrm{i}\Gamma$ with $\Gamma > 0$ would leave the term $-2\Gamma|\psi|^2$ in equation (79) and describe absorption, so the reality of V is what conserves the norm. The energy is conserved because $\hat{H}$ does not depend on time.

The network obeys equation (65) only up to its residual. Solving equation (74) for the time derivative gives $\partial_t\psi_\theta = -\mathrm{i}\hat{H}\psi_\theta - \mathrm{i}\,r_\theta$, and with the norm $N_\theta(t) \coloneqq \langle\psi_\theta|\psi_\theta\rangle$ and the inner product $\langle\chi|\chi'\rangle \coloneqq \int \chi^*\chi'\,dx$,

$$\frac{dN_\theta}{dt} = 2\,\mathrm{Re}\,\langle\psi_\theta|\partial_t\psi_\theta\rangle = 2\,\mathrm{Im}\,\langle\psi_\theta|r_\theta\rangle. \tag{81}$$

The term $\langle\psi_\theta|\hat{H}\psi_\theta\rangle$ has dropped out because it is real, which is the integrated form of equation (79) and holds when ψ_θ vanishes at the edges of the box. The norm of the network therefore changes only through the part of the residual that is proportional to $\mathrm{i}\psi_\theta$. With $\|\chi\| \coloneqq \langle\chi|\chi\rangle^{1/2}$, the Cauchy–Schwarz inequality turns equation (81) into

$$\|r_\theta(\cdot,t)\| \geq \frac{1}{2\|\psi_\theta(\cdot,t)\|}\,\Big|\frac{dN_\theta}{dt}\Big|, \tag{82}$$

so a measured drift of the norm is a lower bound on the size of the residual at that time. A drifting norm proves that the network violates the law. A constant norm proves less, since the parts of the residual orthogonal to $\mathrm{i}\psi_\theta$ leave the norm unchanged. The loss controls the residual only at the collocation points, and a drifting norm reveals residual between them, while a constant norm does not exclude it.

Conservation of the norm and of the energy is a necessary check and an insufficient one. With any real number $\tilde{E}$, the wave function

$$\tilde{\psi}(x,t) = \frac{\psi_0(x)\,e^{-\mathrm{i}E_0t} + \psi_1(x)\,e^{-\mathrm{i}\tilde{E}t}}{\sqrt{2}} \tag{83}$$

has the norm 1 and the energy $\langle\tilde{\psi}|\hat{H}|\tilde{\psi}\rangle = (E_0 + E_1)/2 = 1$ at all times, since ψ_0 and ψ_1 are orthonormal eigenstates of $\hat{H}$. Its density oscillates with the frequency $|\tilde{E} - E_0|$, which differs from the true $E_1 - E_0 = 1$ unless $\tilde{E} = E_1$ or $\tilde{E} = 2E_0 - E_1$. Inserting $\tilde{\psi}$ into the residual gives

$$\mathrm{i}\,\partial_t\tilde{\psi} - \hat{H}\tilde{\psi} = \frac{(\tilde{E} - E_1)\,\psi_1(x)\,e^{-\mathrm{i}\tilde{E}t}}{\sqrt{2}}, \tag{84}$$

which vanishes only for $\tilde{E} = E_1$. At $\tilde{E} = 2E_0 - E_1$ even the density oscillates correctly, while the phase evolves wrongly. A whole family of wrong motions passes both conservation checks, and only the residual, the phase and the comparison with the exact solution exclude it.

A run of the same PINN without the envelope, $(u_\theta, v_\theta) = (n_\theta^{\mathrm{re}}, n_\theta^{\mathrm{im}})$ and everything else unchanged, shows that conservation of the norm is not guaranteed. Its largest error after the snapshots reaches 3.6×10^{-1}, and its norm falls to 0.545. The residual is linear in ψ_θ, so a smaller amplitude lowers the residual term, and away from the snapshots no term of the loss fixes the amplitude. For the tested initialisation and training budget the run with the envelope kept the norm over the full period and the run without it did not, and which property of the envelope is responsible remains untested.

7.15 Unitary evolution and the classical flow map

The quantum and the classical PINNs penalise the residual of a law whose solution is fixed by the state at one time, the position and velocity of the oscillator or the wave function of the particle. The

two laws generate evolutions of different kinds, and the difference shows in what happens to errors. The exact solution of equation (65) is $\psi(\cdot,t)=\hat{U}(t)\psi(\cdot,0)$ with the evolution operator

$$\hat{U}(t)\coloneqq e^{-\mathrm{i}\hat{H}t},\qquad \hat{U}(t)^{\dagger}\hat{U}(t)=\mathbf{1},\qquad \hat{U}(t+t')=\hat{U}(t)\,\hat{U}(t'),\tag{85}$$

where $\hat{U}^{\dagger}$ is the adjoint and unitarity, $\hat{U}^{\dagger}\hat{U}=\mathbf{1}$, follows from $\hat{H}^{\dagger}=\hat{H}$. An operator with this property is called unitary. It is linear, and it preserves the inner product of any two wave functions, $\langle\hat{U}\chi|\hat{U}\chi'\rangle=\langle\chi|\chi'\rangle$. For $\chi'=\chi$ this is the conservation of the norm, and for the difference of two wave functions it gives

$$\|\hat{U}(t)\chi-\hat{U}(t)\chi'\|=\|\chi-\chi'\|,\tag{86}$$

so the distance between two quantum states does not change in time.

A classical system $\dot{\mathbf{x}}=\mathbf{F}(\mathbf{x})$ evolves by its flow map φ_t, which carries every initial state to the state reached after the time t, $\mathbf{x}(t)=\varphi_t(\mathbf{x}(0))$, and has the same composition rule, $\varphi_{t+t'}=\varphi_t\circ\varphi_{t'}$. In general it is nonlinear, and it changes distances and volumes. The volume $\mathcal{V}$ of a small set of initial states changes at the rate set by the divergence of the vector field,

$$\frac{d\mathcal{V}}{dt}=\int_{\varphi_t(\mathrm{set})}\nabla\cdot\mathbf{F}\,d^d\mathbf{x},\tag{87}$$

as for the damped pendulum above. For the oscillator of the first PINN, with the state (x,p), $\dot{x}=p/m_0$ and $\dot{p}=-k_s x-c\,p/m_0$, the divergence is $-c/m_0$, every area of the phase plane shrinks as e^{-ct/m_0}, and all trajectories approach the state of rest. Two different initial states end in the same place, and an error in the initial state is forgotten. For a Hamiltonian system, $\dot{q}_i=\partial H/\partial p_i$ and $\dot{p}_i=-\partial H/\partial q_i$, the divergence is $\sum_i\left(\partial^2 H/\partial q_i\partial p_i-\partial^2 H/\partial p_i\partial q_i\right)=0$, and phase-space volume is preserved, which is Liouville's theorem. The Hamiltonian flow preserves more than the volume, the sum of the oriented areas of the projections of any small two-dimensional surface onto the planes (q_i,p_i), a property called symplectic [45].

The Schrödinger equation is a linear Hamiltonian system of this kind. With the energy functional

$$\mathcal{H}[u,v]\coloneqq\tfrac{1}{2}\langle u|\hat{H}|u\rangle+\tfrac{1}{2}\langle v|\hat{H}|v\rangle=\tfrac{1}{2}\langle\psi|\hat{H}|\psi\rangle,\tag{88}$$

where $\psi=u+\mathrm{i}v$ and the cross terms cancel because $\hat{H}$ is Hermitian and has real coefficients, the variations are $\delta\mathcal{H}/\delta v=\hat{H}v$ and $\delta\mathcal{H}/\delta u=\hat{H}u$, and equations (76) take the form $\partial_t u=\delta\mathcal{H}/\delta v$, $\partial_t v=-\delta\mathcal{H}/\delta u$ of Hamilton's equations, with the real part as the coordinate and the imaginary part as the momentum at every position. The unitary evolution is therefore the linear counterpart of a volume-preserving flow, and equation (86) states the stronger fact that it preserves all distances. For the exact law this means that an error in the wave function at one time is neither damped, as in the dissipative oscillator, nor amplified, as in a chaotic flow. It is carried along with unchanged size. The network obeys the law only up to its residual, $\mathrm{i}\,\partial_t\psi_\theta=\hat{H}\psi_\theta+r_\theta$, so the difference $\delta\coloneqq\psi_\theta-\psi$ obeys $\mathrm{i}\,\partial_t\delta=\hat{H}\delta+r_\theta$, whose solution is $\delta(t)=\hat{U}(t)\delta(0)-\mathrm{i}\int_0^t\hat{U}(t-t')\,r_\theta(t')\,dt'$. With equation (86) this gives

$$\|\psi_\theta(t)-\psi(t)\|\le\|\psi_\theta(0)-\psi(0)\|+\int_0^t\|r_\theta(\cdot,t')\|\,dt',\tag{89}$$

so the error of the network grows at most by the accumulated size of its residual. The penalty on the residual is the same construction for both kinds of law. Unitarity and dissipation are properties of the law, and the trained network inherits them only as far as its residual is small.

7.16 Number of snapshots and noise

Figure 51 varies the data, with half the training budget, 3000 steps of Adam and 1000 iterations of L-BFGS, and with the largest error of $e(t)$ over the whole window $0\le t\le 2\pi$ as the measure. The snapshots are taken at $t=0,0.5,1,1.5,2$, and the models use the first 1, 2, 3 or 5 of them. Model A keeps an error between 1.1×10^{0} and 1.9×10^{0} for all four sets. The PINN reaches 2.2×10^{-2}

with the single snapshot at $t = 0$, which is an initial condition for the Schrödinger equation, and the residual alone carries it through the period. With two, three and five snapshots it reaches 2.0×10^{-2}, 7.0×10^{-3} and 7.9×10^{-3}. Further snapshots average the noise of the data, and beyond three we see no further gain at this budget. With three snapshots the PINN error is 7.8×10^{-3} without noise and 7.0×10^{-3} at $\sigma = 0.01$, so at this noise level the training sets the error. At $\sigma = 0.05$ it grows to 3.1×10^{-2}, by a factor of about four for a fivefold increase of the noise.

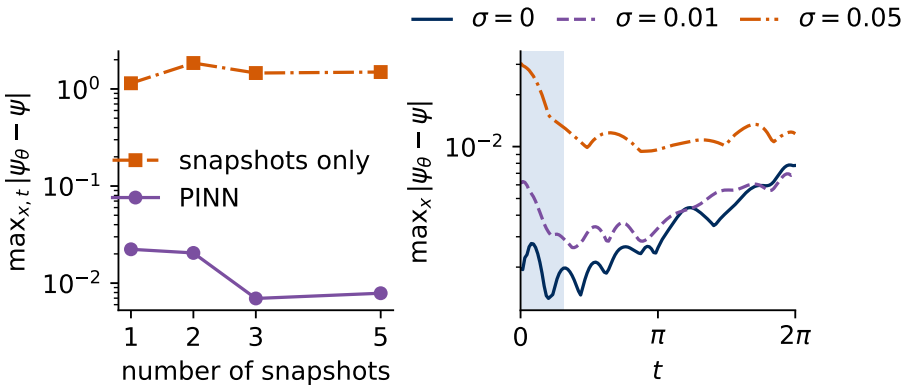


Figure 51: Snapshots and noise, with half the training budget. Left, the largest error $\max_{x,t} |\psi_\theta - \psi|$ over $0 \le t \le 2\pi$ against the number of snapshots, taken at $t = 0, 0.5, 1, 1.5, 2$ in this order, for model A and the PINN. Right, the PINN error $e(t) = \max_x |\psi_\theta - \psi|$ against time for three snapshots at three noise levels σ, with the snapshot window shaded.

7.17 Two classical references

The PINN uses the Hamiltonian, and a fair comparison gives a classical method the same knowledge. The first classical reference expands the snapshots in the eigenstates of the trap. If $\psi(x,t) = \sum_n c_n \psi_n(x) e^{-\mathrm{i}E_n t}$, the overlap of the snapshot at t_k with ψ_n equals $c_n e^{-\mathrm{i}E_n t_k}$. Undoing this phase and averaging over the three snapshots gives the estimate

$$\hat{c}_n = \frac{1}{3} \sum_k e^{\mathrm{i}E_n t_k} \sum_j \psi_n(x_j)\, \psi_{jk}\, \Delta x_d, \tag{90}$$

where the sum over j approximates the overlap integral with the spacing Δx_d of the data points. With the ten states $\psi_0, \ldots, \psi_9$, the expansion $\sum_n \hat{c}_n \psi_n(x) e^{-\mathrm{i}E_n t}$ predicts the wave function with a largest error of 6.1×10^{-3} over the whole window. The projection on ten smooth functions averages the noise of all 384 data values. The PINN, with 6.7×10^{-3} inside the snapshot window and 4.0×10^{-3} after it, matches this reference without using the eigenstates, after 73 seconds of training.

The second reference is the standard solver of the time-dependent Schrödinger equation, the split-step Fourier method of Fleck, Morris and Feit [41]. The potential energy acts on ψ as a multiplication in position space. The kinetic energy acts as the multiplication by $p^2/2$ in momentum space, where the momentum p equals, for $\hbar = 1$, the wave number of the plane wave $e^{\mathrm{i}px}$, and the fast Fourier transform moves between the two representations. Each part alone is solvable in closed form over a time step Δt, as a multiplication by a phase. The two parts do not commute, however, so the product of their exponentials differs from $e^{-\mathrm{i}\hat{H}\Delta t}$. The symmetric splitting of Strang [42]

$$\psi(t + \Delta t) \approx e^{-\mathrm{i}V\Delta t/2}\, \mathcal{F}^{-1}\, e^{-\mathrm{i}p^2\Delta t/2}\, \mathcal{F}\, e^{-\mathrm{i}V\Delta t/2}\, \psi(t), \tag{91}$$

with the discrete Fourier transform $\mathcal{F}$ on the periodic grid of the box, makes an error of order Δt^3 per step through the commutator of the two parts, and the error after a fixed time decreases as Δt^2. Every factor is a phase, so the norm is conserved to round-off for any step. Started from the exact $\psi(x, 0)$ with 1024 steps of $\Delta t = 0.0061$, the method reaches an error of 8.4×10^{-6} at $t = 2\pi$ in 9.1×10^{-3} seconds, with a norm that deviates from 1 by at most 1.4×10^{-13}. Halving Δt lowers the

error by a factor of four, as the order Δt^2 predicts, from 8.7×10^{-3} at $\Delta t = 0.196$ to 2.1×10^{-6} at $\Delta t = 0.0031$. This error is measured at $t = 2\pi$ only, while the PINN errors are maxima over many times, so the comparison holds for orders of magnitude. For the forward problem with a known initial state the split-step method is about three orders of magnitude more accurate than the PINN and about 10^4 times faster.

Started from the first noisy snapshot, interpolated linearly to the full grid, the split-step method has an error between 2.5×10^{-2} and 4.0×10^{-2} at all times from $t = 0$ on. The noise of the starting snapshot is part of its initial state, and by equation (86) the unitary evolution carries it along with unchanged size. This comparison mixes two differences, the method and the amount of data, since the PINN uses three snapshots. With the single snapshot at $t = 0$ and half the training budget the PINN reached 2.2×10^{-2}, comparable with the split-step value. The eigenstate expansion uses the same three snapshots and the same Hamiltonian, and it is the fair reference.

7.18 An unknown trap frequency

An unknown parameter of the law is the task in which the PINN combines the data and the law in one fit. The spectrum of $\hat{H}$ enters through the time dependence. A function with vanishing residual is a superposition of stationary states with the phases $e^{-\mathrm{i}E_n t}$, so the residual fixes the frequencies $E_n - E_{n'}$ of every observable, such as the period 2π of $\langle x \rangle(t)$ in the trap. A different trap frequency changes these frequencies, and the snapshots together with the residual select the value consistent with both. We now assume that the frequency ω in $V(x) = \frac{1}{2}\omega^2 x^2$ is unknown, while lengths and times keep the units of the true trap, whose frequency is 1. The PINN writes $\omega = e^s$, which keeps it positive, and trains s together with the weights on the loss $\mathcal{L}_\mathrm{B}$ with this potential, started at $\omega = 1.5$. In a single training run it finds $\omega = 0.9952$ (Figure 52). The classical reference is a least-squares fit through the split-step solver. For a trial ω it propagates the first snapshot with $\Delta t = 0.01$, compares the result with the two later snapshots and minimises the sum of squared deviations over ω. On the same data it gives $\omega = 0.9962$, and over 20 further realisations of the noise its estimates have the mean 1.0000 and the standard deviation 0.0033. The PINN estimate lies within 1.5 standard deviations of the true value, and the two estimates agree within this statistical spread. The spread of the PINN estimate is unknown, since it rests on one run, and both methods use the known form of the Hamiltonian.

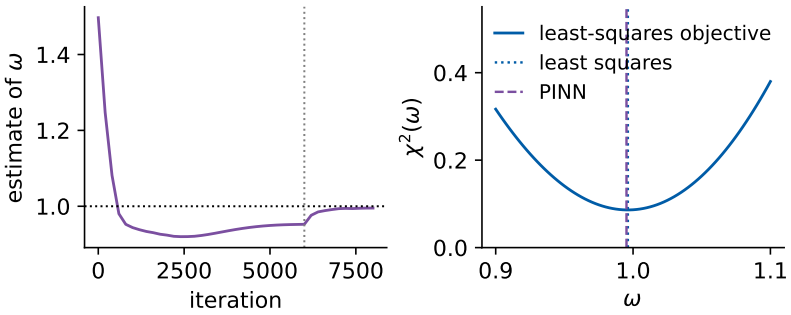


Figure 52: Unknown trap frequency. Left, the PINN estimate of ω against the training iteration, with the true value 1 dotted and the switch from Adam to L-BFGS as a vertical dotted line. Right, the least-squares objective $Q(\omega)$, the sum of squared deviations of the split-step solution from the two later snapshots, with the least-squares and the PINN estimates as vertical lines.

7.19 Tunnelling in a double well

A particle in a double well shows an effect that classical mechanics forbids. The potential

$$V(x) = V_0 (x^2/a^2 - 1)^2, \qquad V_0 = 4, \quad a = 1, \tag{92}$$

has two minima $V = 0$ at $x = \pm a$, separated by a barrier of height V_0 at $x = 0$. We keep $\hbar = m_0 = 1$ and the box of the trap. Near each minimum the potential is approximately harmonic, with the

curvature $V''(\pm a) = 8V_0/a^2$ and the well frequency $\omega_w = \sqrt{8V_0}/a = 5.66$.

The eigenstates are computed by diagonalising the Hamiltonian discretised on the grid. The two lowest levels $E_0 = 2.290$ and $E_1 = 2.752$ lie below the barrier and form a close pair, separated by the splitting $\Delta E = E_1 - E_0 = 0.462$, while the next level $E_2 = 5.941$ lies 3.19 above the pair. The ground state ψ_+ is even and has no node, and the first excited state ψ_- is odd, with a node at $x = 0$. We choose the signs so that $\psi_+ > 0$ everywhere and $\psi_- > 0$ in the left well. The combinations

$$\psi_L := \frac{\psi_+ + \psi_-}{\sqrt{2}}, \qquad \psi_R := \frac{\psi_+ - \psi_-}{\sqrt{2}} \tag{93}$$

then add up in the left well and nearly cancel in the right one, so ψ_L is localised in the left well, and $\psi_R(x) = \psi_L(-x)$ in the right one. A small fraction $P_R(0) := \int_{x>0} \psi_L^2 \, dx = 0.022$ of the probability of ψ_L lies on the right side.

A particle started in $\psi(x,0) = \psi_L$ is a superposition of two stationary states and evolves like the superposition of equation (67). The common phase is removed by shifting the zero of energy to the middle of the pair, $\bar{E} := (E_0 + E_1)/2 = 2.521$. Replacing V by $V - \bar{E}$ multiplies every solution by the global phase $e^{\mathrm{i}\bar{E}t}$ and changes neither the density nor any expectation value. In this frame the two energies are $\mp\Delta E/2$, and with $e^{\pm\mathrm{i}\alpha} = \cos\alpha \pm \mathrm{i}\sin\alpha$,

$$\psi(x,t) = \frac{\psi_+(x)\, e^{\mathrm{i}\Delta E t/2} + \psi_-(x)\, e^{-\mathrm{i}\Delta E t/2}}{\sqrt{2}} = \cos\frac{\Delta E\, t}{2}\, \psi_L(x) + \mathrm{i}\, \sin\frac{\Delta E\, t}{2}\, \psi_R(x). \tag{94}$$

The real part of the wave function sits in the left well and the imaginary part in the right one. Since ψ_L and ψ_R are real and their coefficients differ by the factor i, the density has no cross term, $|\psi|^2 = \cos^2(\Delta E t/2)\, \psi_L^2 + \sin^2(\Delta E t/2)\, \psi_R^2$. The probability of finding the particle on the right side follows from the mirror symmetry $\psi_R(x) = \psi_L(-x)$,

$$P_R(t) := \int_{x>0} |\psi(x,t)|^2 dx = P_R(0) + [1 - 2P_R(0)] \sin^2\frac{\Delta E\, t}{2} \approx \sin^2\frac{\Delta E\, t}{2}, \tag{95}$$

where the approximation neglects the small $P_R(0)$. The particle moves to the right well within the time $\pi/\Delta E$, where P_R reaches $1 - P_R(0) = 0.978$, and returns after the tunnelling period $T_{\mathrm{tun}} := 2\pi/\Delta E = 13.587$. The mean energy $\bar{E}$ lies below the barrier. A classical particle with this energy moves between the two turning points of one well and never reaches the other side, so its P_R stays zero for a start in the left well (Figure 53).

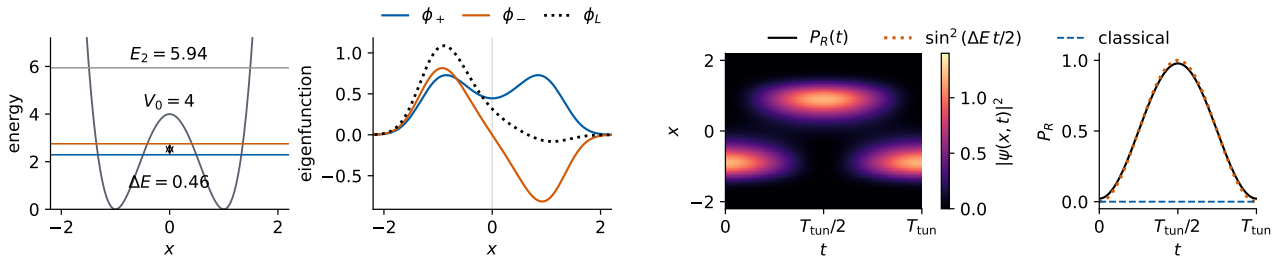


Figure 53: The double well. Far left, the potential $V(x)$ with the energies E_0, E_1 and E_2 as horizontal lines and the splitting ΔE. Centre left, the eigenfunctions ψ_+ and ψ_- and the left-localised combination ψ_L. Centre right, the exact density $|\psi(x,t)|^2$ over time and position for the start in ψ_L. Far right, the probability $P_R(t)$ on the right side, the two-level formula $\sin^2(\Delta E\, t/2)$ and the classical value zero.

7.20 Tunnelling from early snapshots

The snapshots are now taken at $t = 0, 1, 2$, with the same noise as before. They cover 14.7 % of the tunnelling period, and at the last snapshot $P_R = 0.212$, so about a fifth of the probability has tunnelled. The task is to predict the full oscillation over $0 \le t \le T_{\mathrm{tun}}$. The snapshots are computed

from equation (94). They therefore carry the global phase of the exact solution in the frame with the energy zero at $\bar{E}$, a convention that state tomography could not provide.

The network and the loss are those of the trap with four adaptations. The time input is rescaled to the new window, $2t/T_{\mathrm{tun}} - 1$. The envelope is narrower, e^{-x^2} in place of $e^{-x^2/4}$, since the states of the double well are concentrated in $|x| \lesssim 2$. The residual uses the shifted potential $V - \bar{E}$, so the network has to represent the slow evolution of equation (94). The shift $\bar{E}$ is computed before training by diagonalising the known Hamiltonian and uses no information about the evolution. The 3072 collocation points are drawn from $[-3, 3] \times [0, T_{\mathrm{tun}}]$, since beyond $|x| = 3$ the potential exceeds 250 and the wave function is negligible. The boundary term and the optimisers are unchanged. As in the trap, model A fits the snapshots alone, and the PINN adds the residual.

Model A fits the snapshots with a largest error of 2.0×10^{-2} and fails afterwards, with an error of 3.3×10^{0} (Figure 54). Its density flows into the right well and keeps growing there. Its P_R follows the exact curve for a short time after the last snapshot, then rises above it, passes 1 before $T_{\mathrm{tun}}/2$ and reaches 4.988 at $t = T_{\mathrm{tun}}$, while the norm grows to 7.601. The network continues the trend of the snapshots, the growth of the density on the right, and nothing in its loss stops it.

The PINN predicts the transfer of the particle to the right well and back. Its largest error is 3.9×10^{-3} inside the snapshot window and 7.9×10^{-3} after it, its norm stays between 0.989 and 1.001, and its P_R deviates from the exact one by at most 3.3×10^{-3}. The largest predicted P_R, 0.976, lies at $t = 6.79$, the grid time closest to $T_{\mathrm{tun}}/2$. A fit of $q + (1 - 2q)\sin^2(\Omega t/2)$ with the two free parameters q and Ω to the predicted $P_R(t)$ gives the period $2\pi/\Omega = 13.60$, against the exact 13.587. The split-step propagation of the first noisy snapshot reaches 3.7×10^{-2} in ψ and 9.2×10^{-3} in P_R, so in this run the PINN, which fits all three snapshots under the constraint of the equation, is more accurate in both quantities.

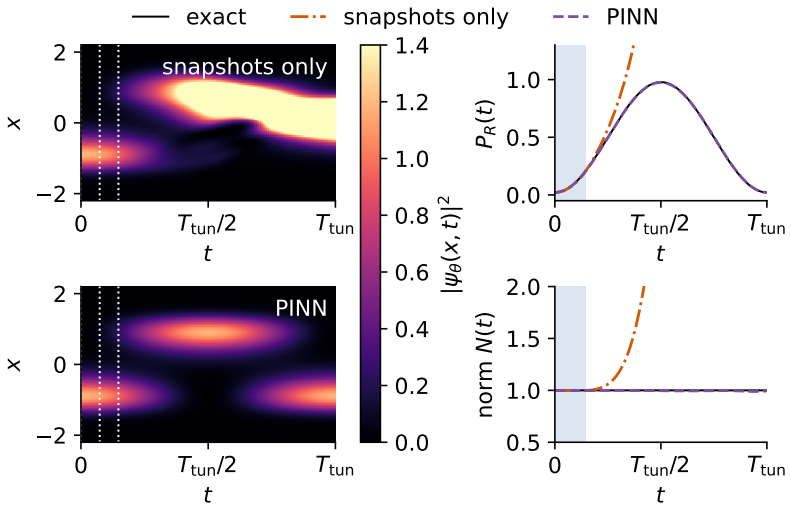


Figure 54: Tunnelling from early snapshots. Left panels, the density $|\psi_\theta(x,t)|^2$ of model A (top) and of the PINN (bottom) over time and position, with the snapshot times as dotted white lines, the one at $t = 0$ lying on the axis. Densities above 1.4 appear in the brightest colour. Right panels, the probability $P_R(t)$ on the right side (top) and the norm $N(t)$ (bottom) of both models and of the exact solution, with the snapshot window $0 \le t \le 2$ as a shaded strip. Model A leaves the plotted range of both right panels.

Two further runs of the PINN, one run each, test the narrow envelope and the energy shift. With the envelope $e^{-x^2/4}$ of the trap the norm falls to 0.004. With the energy zero at the bottom of the wells, where the snapshots carry the additional phase $e^{-\mathrm{i}\bar{E}t_k}$ and the residual uses V itself, the norm falls below 10^{-3}. In both runs the PINN decayed towards $\psi = 0$, the solution with zero residual that only the snapshot term excludes, so in our runs both choices were needed. Without the shift the global phase advances by $\bar{E}T_{\mathrm{tun}} \approx 34$ radians over the window, a fast rotation that the network of this size failed to follow at this training budget. The two runs describe the same physics, since they differ only by the global phase $e^{-\mathrm{i}\bar{E}t}$, which no measurement detects, and the failure concerns the

representation and the training. The restriction of the collocation points to $|x| \leq 3$ and the rescaling of the time input were chosen without a separate test.

7.21 An unknown barrier height

The splitting ΔE depends on the barrier height, and with it the tunnelling period. For a high barrier the semiclassical theory gives $\ln \Delta E \simeq -\int \sqrt{2\,(V(x) - E)}\,dx$ up to a slowly varying term, with the integral over the classically forbidden region under the barrier, as derived in the textbook of Landau and Lifshitz [46]. The integrand grows as $\sqrt{V_0}$ and the width of the region as a, so in this limit $\ln \Delta E$ decreases roughly in proportion to $a\sqrt{V_0}$. Our barrier is far from this limit, since E_0 and E_1 lie above half the barrier height. The splitting computed by diagonalisation falls from 0.692 at $V_0 = 2$ to 0.297 at $V_0 = 6$, with the logarithmic derivative $d \ln \Delta E / dV_0 = -0.22$ at $V_0 = 4$. The tunnelling period therefore carries information on V_0.

When V_0 is unknown, the energy zero $\bar{E}$, which depends on V_0, is unknown too, so the residual uses the potential $V_0(x^2/a^2 - 1)^2 - \varepsilon$ with a second unknown constant ε. A constant shift of the potential changes only the global phase of the solution, so ε is fixed by the global phase carried by the synthetic snapshots. The PINN trains $\ln V_0$ and ε together with the weights, started at $V_0 = 3$ and at the value $\varepsilon = 2.156$ that the diagonalisation gives for $\bar{E}$ with this barrier. In a single run it finds $V_0 = 4.100$ and $\varepsilon = 2.555$, against the true 4 and $\bar{E} = 2.521$ (Figure 55). During the Adam stage the estimate of V_0 first drops and then settles slightly above 4, and L-BFGS moves it to the final value. The least-squares reference propagates the first snapshot with the split-step solver to $t = 2$ with $\Delta t = 0.01$ in the trial potential, compares the result with the two later snapshots and minimises over both parameters V_0 and ε with the Nelder–Mead simplex method from the same starting point. On the same data it gives $V_0 = 4.094$ and $\varepsilon = 2.553$, and over 20 further noise realisations its estimates of V_0 have the mean 3.973 and the standard deviation 0.079. Both estimates lie within 1.3 standard deviations of the true value. The estimated barriers imply the tunnelling periods 13.89 (PINN) and 13.87 (least squares), against the exact 13.587, so the error of 2.5 % in V_0 of the PINN becomes an error of 2.2 % in T_{tun}, computed from the unrounded periods.

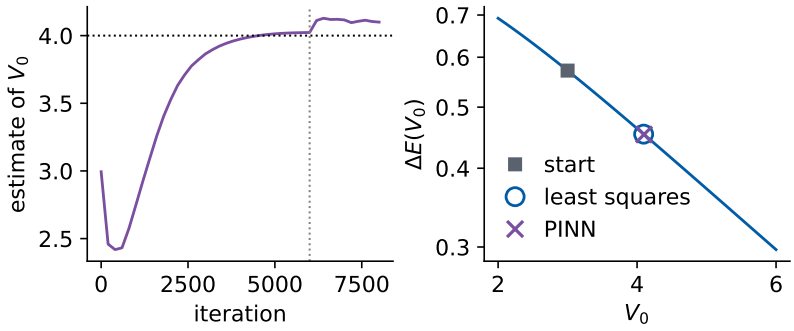


Figure 55: Unknown barrier height. Left, the PINN estimate of V_0 against the training iteration, with the true value 4 dotted and the switch from Adam to L-BFGS as a vertical dotted line. Right, the splitting ΔE against V_0 on a logarithmic axis, computed by diagonalisation, with the starting value, the least-squares estimate and the PINN estimate marked.

7.22 Limits of physics-informed networks

The trap and the double well were followed over one period of their motion. Over several periods a single network can fail even for a classical system. For the pendulum $\ddot{\theta} = -\omega_p^2 \sin\theta$ with $\omega_p = 1$, started at rest at $\theta(0) = 2$, the period is $T_p = 8.350$. A PINN with the network and the training budget of the oscillator, 100 collocation points per period and the residual together with the initial-condition term, was trained on one, two and four periods, each from three random initialisations of the weights. On one period all three runs reach largest errors between 3.7×10^{-5} and 6.9×10^{-5}. On two periods all three converge to the same wrong function, which starts near $\theta = 2$, leaves the solution at $t \approx 1.95$ and

settles near the upright position $\theta = -\pi$, with an error of 5.1×10^{0}, the size of the swing (Figure 56). On four periods two runs succeed, with errors of 6.5×10^{-2} and 5.2×10^{-2}, and the third fails in the same way.

The equation with its initial condition has a unique solution, so the failure does not lie in the law. A function that follows the solution for a while, turns over in a short interval and then rests at the upright equilibrium, which satisfies the equation, violates the law only in that interval. Its mean residual over a finite set of collocation points is small, and the optimiser can settle there. The three failed runs on two periods end with the loss 3.1×10^{-3}, against values between 6.7×10^{-9} and 6.4×10^{-8} for the runs on one period, so a loss of this size accompanies a trajectory error of the size of the swing. The residual term is a mean over the collocation points, and placing more points does not raise the weight of the short interval in which the law is violated. The initial conditions enter the loss with unit weights, so the final loss bounds the error of the starting angle by $\sqrt{3.1 \times 10^{-3}} \approx 0.056$, and the wrong function does start near $\theta = 2$. A small training loss measures the sampled penalties, and the accuracy of the trajectory needs a separate check against the equation or a reference solution. Krishnapriyan and coauthors documented failures of PINNs on convection, reaction and diffusion equations with large coefficients and traced them to the difficulty of optimising a loss that contains the residual, while networks of the same size could represent the solution [61]. Learning the solution in successive time windows, each started from the end of the previous one, reduced their errors. Training one network per period, each started from the end state of the previous one, avoided the failure in these runs, with an error that grows from 6.9×10^{-5} in the first period to 1.4×10^{-3} in the fourth, since every period starts from the slightly wrong end state of the previous one.

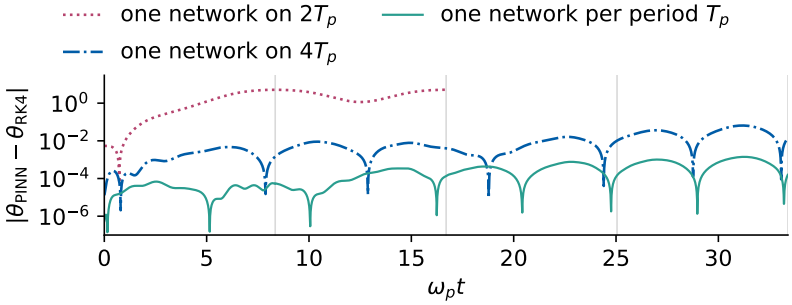


Figure 56: The pendulum over several periods. The difference between the PINN solution and an RK4 solution against $\omega_p t$ on a logarithmic scale, for one network on two and on four periods, both from the first of the three initialisations, and for one network per period, each started from the end of the previous one. Grey lines mark the period boundaries.

A PINN is a suitable tool when a law is known and the data are sparse and noisy, in particular when a parameter of the law is unknown, and it needs no grid, which helps for irregular domains and many dimensions. For a plain forward problem it is far slower than a classical solver. Its training is a hard optimisation, since loss terms of very different size pull the weights in different directions [40], and over long times it can settle on wrong functions with a small residual. The PINN needs the whole law. Between the two extremes, a known law and an unknown vector field, lies the common case of a law that is known except for one term.

8 Hybrid models with one learned term

8.1 Known terms and a learned term

The Neural ODE assumed only the form $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x})$ of the law, and the damped pendulum showed the price, a field that is reliable only where the data are. The PINN assumed the whole law. Suppose a pendulum is filmed while it swings and slowly comes to rest. Newton's law says that its angular acceleration is the sum of two contributions, the pull of gravity, $-\omega_p^2 \sin\theta$, and the braking by friction. Gravity is known to high accuracy. The friction is not, since it may grow with the speed, with the

square of the speed, or stay constant, and its strength is unknown as well. Three models can be built from this knowledge. The first keeps gravity only. It has nothing to learn, and its pendulum swings forever. The second is a Neural ODE, which learns the whole right-hand side and ignores the gravity that we know, so it has to learn gravity from the data too and can only guess beyond them. The third keeps gravity and gives a network only the missing piece, the friction.

Models of the third kind are called hybrid models, or universal differential equations [47]. In general they keep the trusted terms of a law and replace only the uncertain one by a network,

$$\dot{\mathbf{x}} = \mathbf{f}_{\text{known}}(\mathbf{x}) + \mathbf{U}_\theta(\mathbf{x}), \tag{96}$$

and they are trained through the solver in the same way as a Neural ODE (Figure 57). Compared with a Neural ODE the network has much less to learn. Compared with a PINN the law need not be complete. After training, the learned term is a function given by a network. A second step compares its shape with a list of simple candidate formulas and keeps a short combination that matches it, so that the network is turned back into a formula.

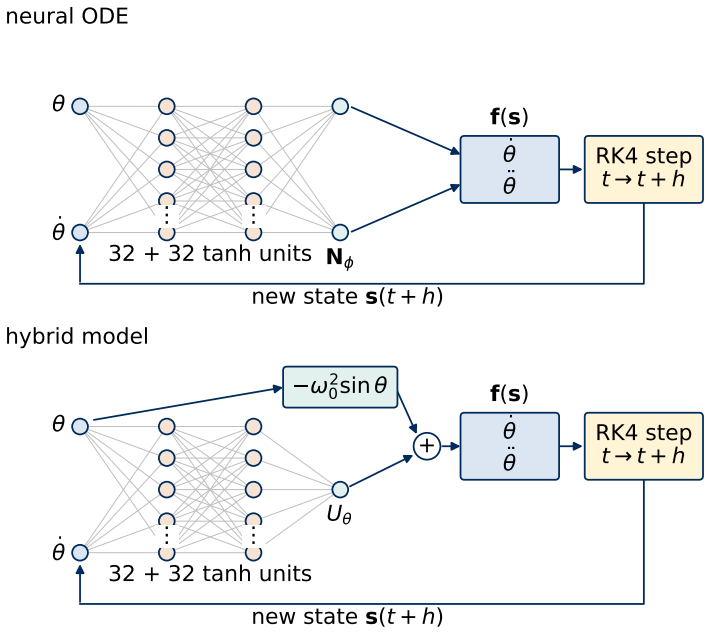


Figure 57: Neural ODE and hybrid model of the pendulum. Top, the network $\mathbf{F}_\theta$ of the Neural ODE returns the whole vector field $(\dot\theta, \ddot\theta)$ of the state $\mathbf{x} = (\theta, \dot\theta)$. Bottom, the network U_θ returns only the friction, which is added to the known gravitational term $-\omega_p^2 \sin\theta$. In both, an RK4 step advances the state from t to $t + \Delta t$ and the new state is fed back. The drawing writes the state as $\mathbf{s}$, the vector field as $\mathbf{f}(\mathbf{s})$, the network of the Neural ODE as N and the solver step as h.

8.2 A pendulum with unknown friction

In equations, the pendulum obeys $\ddot\theta = -\omega_p^2 \sin\theta + F(\dot\theta)$, with $\omega_p = 1$, and loses energy through a friction force F, per unit moment of inertia, that depends on the material of the bearing and the shape of the bob. Three laws are common. Viscous friction, $F = -\gamma\dot\theta$, describes slow motion in a fluid. Quadratic friction, $F = -c\,\dot\theta|\dot\theta|$, describes air drag. Dry friction, $F = -\mu \tanh(\dot\theta/\varepsilon)$ with a small $\varepsilon = 0.05$, is the Coulomb friction of two solid surfaces with its sign function smoothed. In this subsection and the next two $\gamma = 0.1$, $c = 0.15$ and $\mu = 0.05$ denote these friction coefficients. The mechanical energy $E = \frac{1}{2}\dot\theta^2 + \omega_p^2(1 - \cos\theta)$ changes at the rate

$$\dot E = \dot\theta\,\ddot\theta + \omega_p^2 \sin\theta\,\dot\theta = \dot\theta\,F(\dot\theta) \le 0, \tag{97}$$

negative for all three laws, since each force opposes the velocity.

Each law leaves its own signature in the amplitudes A_n at the successive turning points. For a small amplitude a half swing is close to $\theta = A\cos t$, with $\dot\theta = -A\sin t$ for $0 \le t \le \pi$, and the energy is close to $E = \frac{1}{2}A^2$, so the change of amplitude over a half swing follows from $A\,\Delta A = \Delta E = \int_0^\pi \dot\theta F\,dt$. Viscous friction gives $\Delta E = -\gamma A^2 \int_0^\pi \sin^2 t\,dt = -\frac{\pi}{2}\gamma A^2$, so $\Delta \ln A = -\pi\gamma/2$ and $\ln A_n$ falls linearly with n. Quadratic friction gives $\Delta E = -cA^3 \int_0^\pi \sin^3 t\,dt = -\frac{4}{3}cA^3$, so $\Delta(1/A) = -\Delta A/A^2 = 4c/3$ and $1/A_n$ grows linearly. Dry friction gives $\Delta E = -\mu A \int_0^\pi \sin t\,dt = -2\mu A$ for $\varepsilon \to 0$, so $\Delta A = -2\mu$ and A_n itself falls linearly. The slopes fitted at small amplitudes, -0.159, 0.201 and -0.103, agree with the predicted $-\pi\gamma/2 = -0.157$, $4c/3 = 0.200$ and $-2\mu = -0.100$ (Figures 59 and 58).

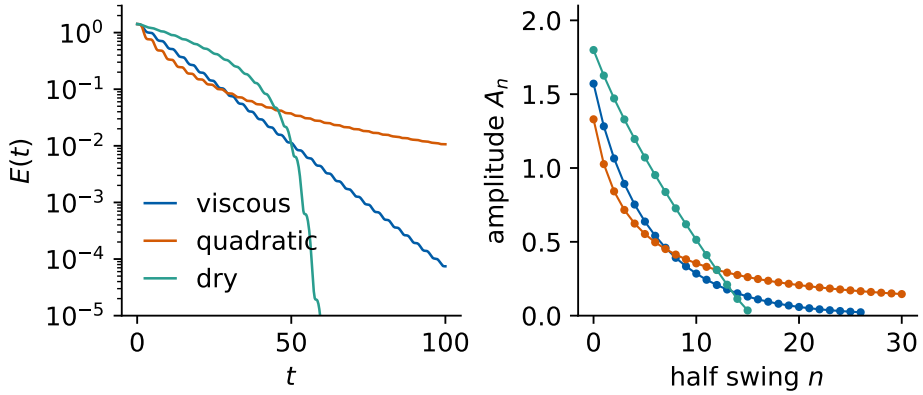


Figure 58: Energy loss of the pendulum for the three friction laws. Left, the energy $E = \frac{1}{2}\dot\theta^2 + \omega_p^2(1-\cos\theta)$ against time on a logarithmic scale. Right, the amplitude A_n at the n-th turning point against n.

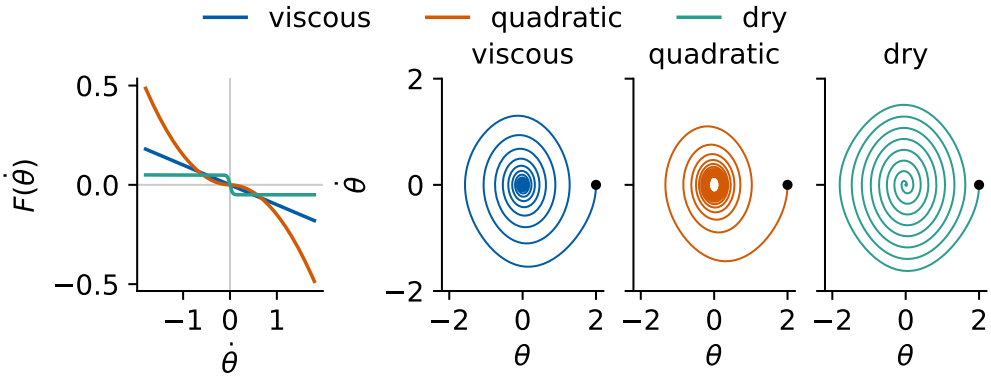


Figure 59: Three friction laws. Left, the friction force F against the angular velocity $\dot\theta$ for viscous, quadratic and dry friction. Right, the phase portraits $(\theta, \dot\theta)$ of the pendulum with each law, spiralling into the hanging position.

The data are one swing with quadratic friction, whose strength $c = 0.15$ the models never see, 201 states measured every 0.1 time units over $0 \le t \le 20$, about three periods, with Gaussian noise of standard deviation $\sigma = 0.02$ on the angle and on the angular velocity. Three models are compared. Gravity alone, without friction, has no parameters. A Neural ODE learns the whole vector field with 1218 parameters. The hybrid model keeps gravity and learns only the friction, $\ddot\theta = -\omega_p^2 \sin\theta + U_\theta(\theta, \dot\theta)$, with 1185 parameters. The network receives both the angle and the angular velocity, so that it could also represent a force that depends on the angle. Both networks are trained through the solver on segments of 30 samples, about half a period, that start at the measured states.

8.3 The learned friction and its formula

The network U_θ of the hybrid model is a function of the state. Evaluated at the measured states and compared with the true friction F at the same states, it has the relative error $\mathrm{rms}(U_\theta - F)/\mathrm{rms}(F) = 0.076$, where rms denotes the root mean square over the measured states, although no formula for

the friction was given (Figure 60). Points with the same velocity and different angles would scatter vertically if the network had learned a dependence on the angle, and they follow one parabola. The learned function is still a network, which does not say which formula it represents.

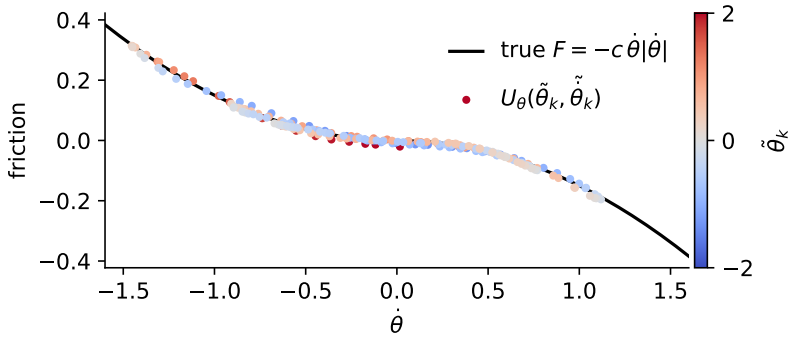


Figure 60: Learned friction of the hybrid model. Dots, the network output U_θ at the measured states against the angular velocity $\dot\theta$, coloured by the angle. Line, the true friction law $F = -c\,\dot\theta|\dot\theta|$.

Sparse regression supplies the formula [48]. The network output at the measured states $\tilde{\mathbf{s}}_k$ is written as a short sum of candidate terms,

$$U_\theta(\tilde{\mathbf{s}}_k) \approx \sum_j w_j\,\Theta_j(\tilde{\mathbf{s}}_k), \qquad \Theta = \{1,\ \sin\theta,\ \theta^2,\ \theta\dot\theta,\ \dot\theta,\ \dot\theta|\dot\theta|,\ \tanh(\dot\theta/\varepsilon),\ \dot\theta^3\}, \tag{98}$$

with a library Θ of eight candidate functions and coefficients w_j. The library contains the three friction laws, a cubic velocity term and three functions that involve the angle. Every column is divided by its root mean square over the data, so that a coefficient measures the typical size of its term. Sequentially thresholded least squares fits all coefficients, sets every coefficient whose modulus is smaller than a threshold ϑ to zero, refits the remaining terms and repeats until the set of active terms no longer changes. The four velocity functions are strongly correlated on the data, since all of them are odd in $\dot\theta$ and grow with $|\dot\theta|$. For nearly parallel columns plain least squares returns large coefficients of opposite sign that cancel each other, and no single coefficient then falls below the threshold. Each fit inside the iteration therefore minimises a ridge-penalised error, $\sum_k \left(U_\theta(\tilde{\mathbf{s}}_k) - \sum_j w_j \Theta_{kj}\right)^2 + 0.01\,n\,\|\mathbf{w}\|^2$ with n the number of data points, which suppresses such cancelling combinations, and the coefficients of the final active set are refitted without the penalty. For the quadratic swing one term survives, $-c\,\dot\theta|\dot\theta|$ with $c = 0.1497$ against the true 0.15, for thresholds between 0.251 and 0.45 times $\mathrm{rms}(U_\theta)$, a window less than a factor of two wide (Figure 61). Below it more terms survive, above it every term is removed, and the value $\vartheta = 0.35\,\mathrm{rms}(U_\theta)$ lies inside it.

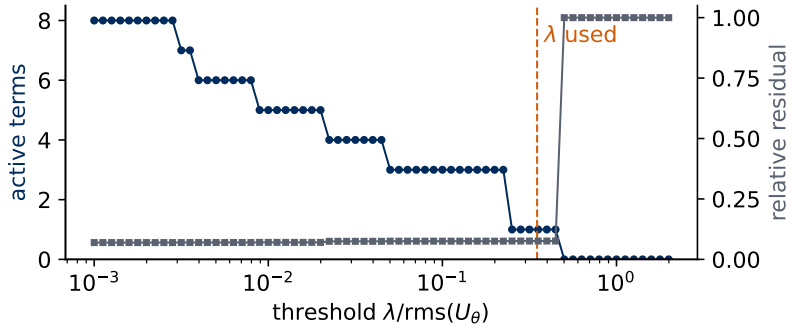


Figure 61: Choice of the threshold in sequentially thresholded least squares. The number of active library terms (left axis) and the relative residual $\mathrm{rms}(U_\theta - \Theta\mathbf{w})/\mathrm{rms}(U_\theta)$ of the fit (right axis) against the threshold $\vartheta/\mathrm{rms}(U_\theta)$, labelled λ in the drawing, on a logarithmic scale. The dashed line marks the threshold used, inside the range that keeps one term.

The network is needed for this step. The same regression applied directly to the measured states needs the acceleration, which a central finite difference $(\tilde{\dot\theta}_{k+1} - \tilde{\dot\theta}_{k-1})/(2\Delta t)$ estimates with the noise of

standard deviation $\sigma/(\sqrt{2}\,\Delta t)$, here $0.02/(\sqrt{2}\cdot 0.1) \approx 0.141$. This noise exceeds the root-mean-square size of the friction itself, 0.100 on the measured states, and the regression finds no term at all. When the true column $\dot\theta|\dot\theta|$ is the only one offered, its coefficient from the same finite differences is $c = 0.148$, so the noisy derivatives carry the information about the coefficient and are too noisy to select the term. The hybrid model never differentiates the data. Its training compares integrated segments with the measured states, which averages the noise, so the network output is a far less noisy target.

Repeated with new data for viscous and dry friction, with the same library, threshold and penalty, the pipeline selects the single correct term in every case. With the network of angle and velocity the recovered coefficients are $\gamma = 0.1008$, $c = 0.1497$ and $\mu = 0.0488$, against the true 0.1, 0.15 and 0.05. A second form of the hybrid model builds in that friction depends only on the angular velocity of the bob, with a network $U_\theta(\dot\theta)$ of the angular velocity alone. A single swing passes through every velocity of its range many times, so a function of $\dot\theta$ alone is constrained over that whole range. This network gives $\gamma = 0.1017$, $c = 0.1505$ and $\mu = 0.0489$, and it approximates the true friction better, with relative errors of 0.09, 0.04 and 0.31 against 0.17, 0.08 and 0.72 for the network of both variables (Figure 62). Dry friction is the hardest case. The smooth units of the network do not form the step of width ε from these data, and the learned function oscillates around the two plateaus. The projection onto the library still selects $\tanh(\dot\theta/\varepsilon)$, since no other library function follows the step, while plain thresholded least squares without the ridge penalty keeps four correlated velocity terms for dry friction with both networks. The library contains the smoothed sign function with the same width ε as the simulation, which makes the dry case easier than it would be for measured data.

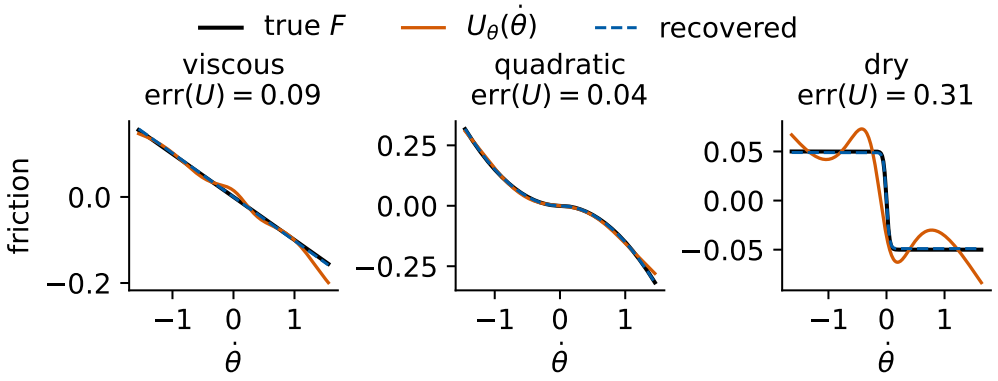


Figure 62: Three friction laws recovered with the network of the angular velocity alone. For viscous, quadratic and dry friction, the true law $F(\dot\theta)$, the network output U_θ on a grid of velocities inside the measured range and the formula found by sparse regression, against the angular velocity, with the relative error $\mathrm{rms}(U_\theta - F)/\mathrm{rms}(F)$ of the network in each title.

8.4 The recovered law far beyond the data

The models differ most far beyond the data. Integrated to $t = 100$, five times the measured window, gravity alone keeps swinging with a root-mean-square error of the angle of 1.479 over $20 < t \le 100$, the Neural ODE reaches 0.176, the hybrid model 0.047, and gravity together with the recovered formula 0.006 (Figure 63). Beyond the measured window the swing enters small amplitudes that the data never visited. The networks are reliable only near their training data, while the recovered formula has the functional form of the true friction at every amplitude. It holds far from the data as far as the true law has this form, which the library assumed. The energy at $t = 100$ shows the same, 0.0107 for the recovered equation against the true 0.0107.

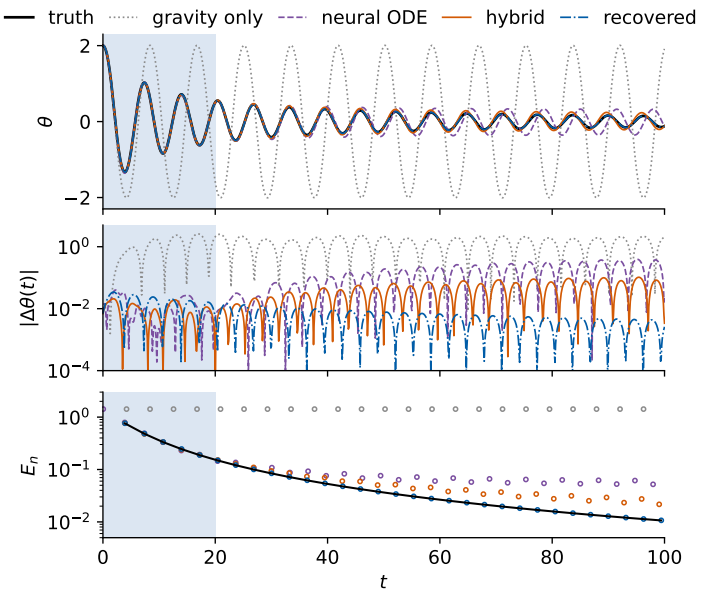


Figure 63: Pendulum forecasts to $t = 100$ from the first measured state. Top, the angle of the truth and of the four models, gravity only, Neural ODE, hybrid and recovered equation, with the data window shaded. Middle, the error of the angle on a logarithmic scale. Bottom, the energy at the turning points.

8.5 Predators and prey

The friction of the pendulum is a function of one variable. A second example has an unknown interaction between two variables. The Lotka–Volterra equations [50, 51] describe prey x and predators y,

$$\dot{x} = \alpha x - \beta xy, \qquad \dot{y} = -\delta y + \gamma xy, \tag{99}$$

where in this section α is the birth rate of prey, δ the death rate of predators, and the products βxy and γxy describe encounters that remove prey and feed predators. With $\alpha = \beta = \gamma = 1$ and $\delta = 0.8$ the steady state lies at $(x^*, y^*) = (\delta/\gamma, \alpha/\beta) = (0.8, 1)$. The equations have a conserved quantity,

$$\mathcal{H}(x, y) := \gamma x - \delta \ln x + \beta y - \alpha \ln y, \tag{100}$$

whose time derivative vanishes along every solution, so every orbit is a closed curve $\mathcal{H} = \text{const}$ around the steady state, and its amplitude is fixed by the start (Figure 64).

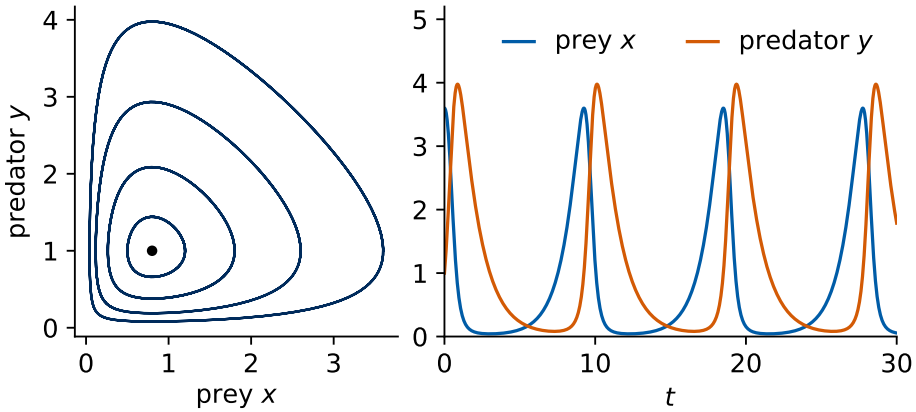


Figure 64: The Lotka–Volterra equations with $\alpha = \beta = \gamma = 1$ and $\delta = 0.8$. Left, closed orbits in the plane of prey x and predators y for several starts. Right, the two populations against time for one start, the predators lagging behind the prey.

Real populations behave differently. A predator needs time to handle each prey, so the number of prey eaten per predator saturates at high prey density, and the prey have a carrying capacity K, the

largest population the environment supports. The Rosenzweig–MacArthur model [53] includes both,

$$\dot{x} = \alpha x - \frac{\alpha}{K} x^2 - \frac{\beta x y}{1 + h x}, \qquad \dot{y} = -\delta y + \frac{\gamma x y}{1 + h x}, \tag{101}$$

where h is the saturation constant, set by the handling time, and the term $-\alpha x^2/K$ limits the growth of the prey. In this model K and h carry only these meanings, unrelated to the Koopman matrix and the measurement function. We keep the rates $\alpha = \beta = \gamma = 1$, $\delta = 0.8$ and take $h = 0.5$ and $K = 6$. The steady state moves to $x^* = \delta/(\gamma - \delta h) \approx 1.333$ and $y^* = (\alpha/\beta)(1 - x^*/K)(1 + h x^*) \approx 1.296$. The Jacobian there has the eigenvalues $0.0444 \pm 0.609\,\mathrm{i}$, with a positive real part, so the steady state is an unstable focus, around which small deviations grow while they rotate. Every start ends on one limit cycle, a closed orbit that attracts all nearby trajectories, with the period 12.53 (Figure 65). This model generates the synthetic data.

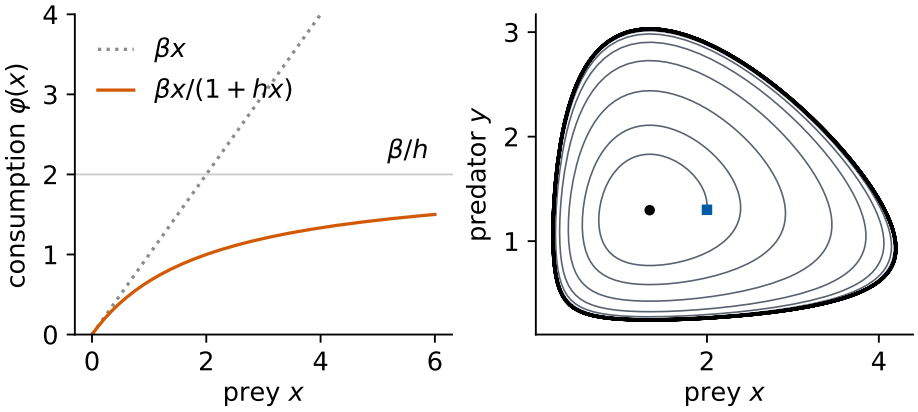


Figure 65: The saturating predator. Left, the number of prey eaten per predator and unit time against the prey density, linear for Lotka–Volterra and saturating for the Holling response [52] $\beta x/(1 + hx)$. Right, trajectories of the Rosenzweig–MacArthur model spiralling onto the limit cycle from inside and outside.

The growth of isolated prey and the death of starving predators can be measured in separate experiments, so the hybrid model keeps these two single-species terms and learns the rest,

$$\dot{x} = \alpha x + U_1(x, y), \qquad \dot{y} = -\delta y + U_2(x, y), \tag{102}$$

with the network (U_1, U_2) of two inputs and two outputs. The self-limitation $-\alpha x^2/K$ of the prey is a single-species term as well, and it is not given to the model, so U_1 has to learn it together with the interaction. The terms that the network has to learn are $U_1 = -\alpha x^2/K - \beta x y/(1 + hx)$ and $U_2 = \gamma x y/(1 + hx)$. The data are 401 population counts over $0 \le t \le 40$ with a relative noise of about 5%, taken while the populations still spiral out towards the cycle. The learned terms follow the truth on the data with relative errors 0.015 and 0.022, and on the limit cycle, which the data never reached, the error of U_2 grows to 0.08 of its largest value (Figure 66).

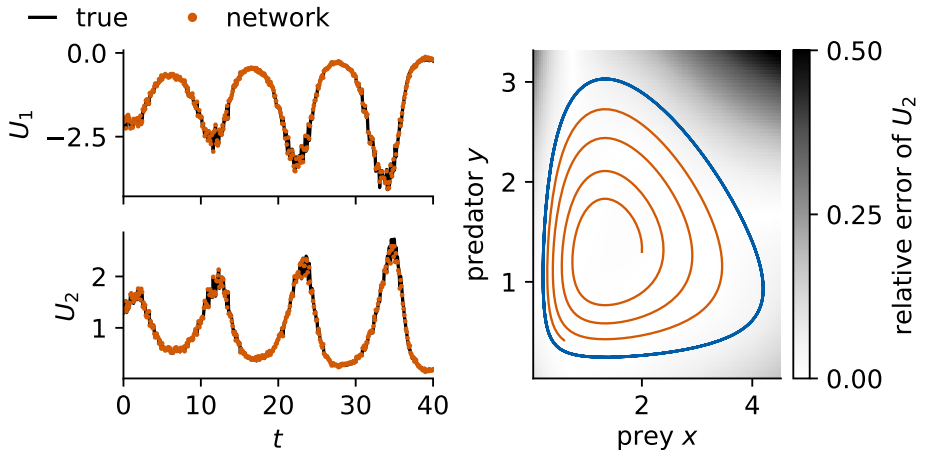


Figure 66: The learned interaction. Left, U_1 and U_2 along the measured trajectory against time, true and learned. Right, the relative error of U_2 over the plane of prey and predators, with the limit cycle (blue) and the data (orange).

8.6 A rational term from a linear fit

The saturating interaction $\gamma xy/(1+hx)$ is no polynomial, and a polynomial library returns a truncated Taylor series of $1/(1+hx)$ with no readable law. The saturation constant sits in the denominator, so it cannot be a coefficient of a library that is linear in its coefficients. Multiplying the relation $U_2 = \gamma xy/(1+hx)$ by the denominator turns it into $U_2 = \gamma\, xy - h\,(x\,U_2)$, which is linear in the unknown γ and h, with the product $x\,U_2$ of the prey and the learned output as an extra column. This implicit form makes a linear fit possible [49]. A least-squares fit with the network output in place of U_2 gives $\hat{h} = 0.474$, against the true 0.5. Scanning h in a library with the single column $xy/(1+hx)$ added to the monomials gives the smallest residual at the true value, and within 10% of it only for $0.45 \le h \le 0.50$ (Figure 67).

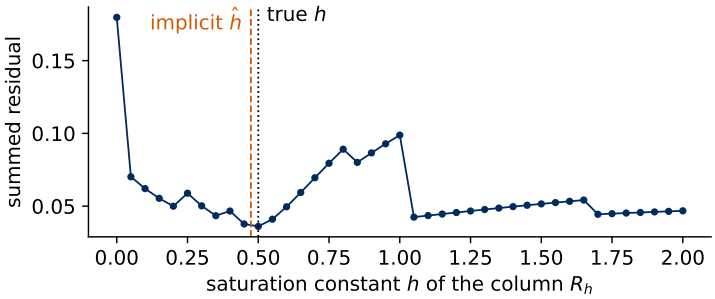


Figure 67: Residual of the sparse fit of U_1 and U_2 with the library term $xy/(1+hx)$ against the saturation constant h, with the implicit estimate $\hat{h}$ and the true value marked.

With $R \coloneqq xy/(1+\hat{h}x)$ the final library is $\{x,\ y,\ x^2,\ xy,\ y^2,\ R\}$. Sparse regression keeps the true terms and no others, $U_1 = a_1x^2 + a_2R$ and $U_2 = a_3R$, with $a_1 = -0.165$, $a_2 = -0.973$ and $a_3 = 0.970$ against the true $-1/6$, -1 and 1. A final fit of the short formula through the solver, with its four numbers a_1, a_2, a_3 and h as the only parameters, gives $h = 0.492$ and coefficients that, together with h, lie within 1.6 % of the truth.

The long forecasts separate the models (Figure 68). The true limit cycle has the period 12.53 and a largest prey number of 4.20. A Lotka–Volterra model fitted to the data gives 11.40 and 2.01, the Neural ODE 11.75 and 3.68, the hybrid model 11.70 and 3.78, and the refitted formula 12.40 and 4.14. The data never reached the cycle, the largest measured prey number being 3.45, so the networks extrapolate into a region they never saw and produce a cycle that is too small and too fast, while the recovered formula, whose form the library contained, comes closest to the true cycle.

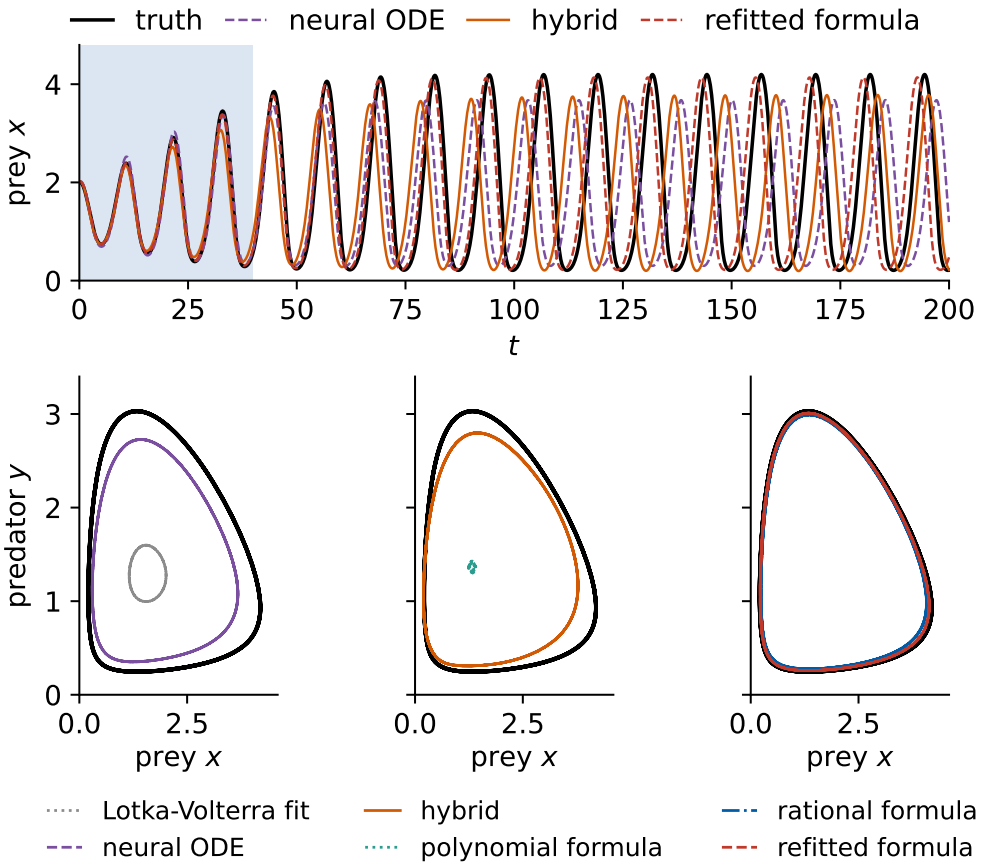


Figure 68: Predator–prey forecasts to $t = 200$. Top, prey against time for the truth and the models, with the data window shaded. Bottom, the late limit cycles in the plane of prey and predators.

8.7 A driver outside the state

Prey growth often depends on the season, through rain or temperature. With $\alpha(t) = \alpha_0(1 + a_s \sin \omega t)$, $\alpha_0 = 1$, $a_s = 0.2$ and a season of $2\pi/\omega = 20$ time units, the prey dynamics contains the term $\alpha_0 a_s \sin(\omega t)\, x$, which no function of (x, y) alone can describe. Given the driver as an additional input, $\mathbf{U}_\theta(x, y, \sin \omega t, \cos \omega t)$, the hybrid model learns the seasonal pattern, and sparse regression finds $0.193\, x \sin \omega t$ against the true $0.2\, x \sin \omega t$. Without the driver the model can only produce a single closed cycle with equal maxima, and its error is 0.754 already within the data window (Figure 69). An influence that is not a function of the state has to be given to the model as an input.

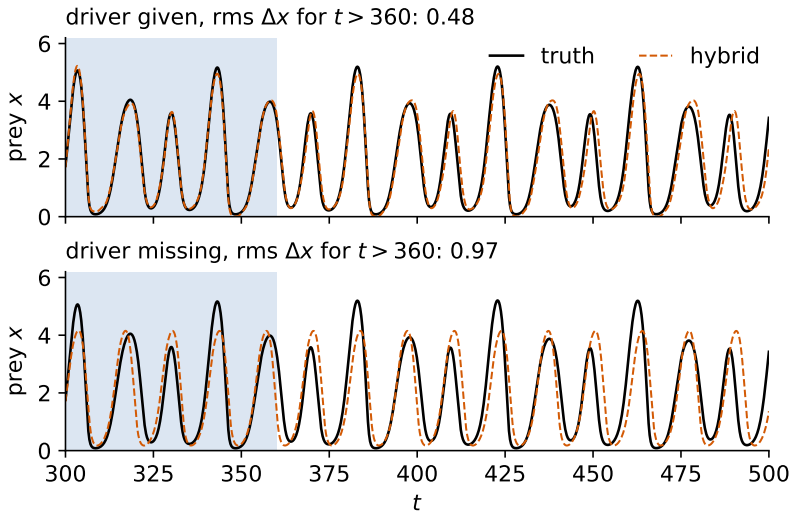


Figure 69: A seasonal driver. Prey against time for the driven truth and the hybrid model, top with the driver given as input, bottom without it, each with its root-mean-square error late in the run.

9 The laser record revisited

The Santa Fe laser record of Figure 2 opened these notes as an irregular sequence of 8-bit numbers, and every tool met since then can now be applied to it. A record poses two questions, whether it can be forecast and which law produced it, and for the laser the tools answer both in part.

The second question comes first, because its answer decides which models are worth fitting. The record contains structure beyond a linear random process. A surrogate of the record is a series with the same distribution of values and approximately the same power spectrum, built by reordering the measured values [9], and it realises the hypothesis that the record is linear Gaussian noise seen through a static measuring device. The forecast that predicts the next value by the value that followed the most similar delay vector of the past has a one-step error of 0.208 standard deviations for the record and at least 0.670 for each of 19 surrogates, so the hypothesis is rejected at the level $1/20 = 0.05$. Delay vectors of $m = 3$ values spaced by $\kappa = 2$ samples, chosen on the training block, unfold the record. The correlation dimension D_2, the exponent of the power law r^{D_2} with which the fraction of pairs of delay vectors closer than r falls at small r, saturates at $D_2 \approx 2.46$ as m grows [10], while noise with the same spectrum keeps filling every added dimension. The leading Lyapunov exponent is positive, 0.029 per sample from the growth of distances between pairs of neighbouring delay vectors [7] and 0.065 from linear maps fitted to neighbourhoods [8]. The two estimates differ on a record that is short, stored as integers from 0 to 255 of which 243 occur, and fills its neighbourhoods along only two directions. The measured record is thus consistent with a deterministic motion on an attractor of dimension between two and three that amplifies small errors by a factor e every 15 to 34 samples. The physics points the same way. A single-mode laser is described by the Maxwell–Bloch equations for three variables, the cavity field, the polarisation of the medium and the population inversion, and after the rescaling found by Haken these equations take the form of the Lorenz equations (7) with the pump in the role of ρ [11]. The parameters of the ammonia laser differ from the convection values of equation (7), so the law has three variables, and its dimension has to come from the record. These statements describe this record of 4000 samples, and each diagnostic has the limits of measurement noise, record length and a specific null hypothesis.

The first question has a quantitative answer on the test block (Table 2). Persistence reaches $C_{\text{tot}} = 5.5$ of the possible 50, the linear autoregression 16.5, since a linear map cannot represent the folded forecast map, NVAR with products of past values 34.7, and the recurrent networks with a learned state between 42.5 and 44.3. The best models keep $C(\tau) > 0.5$ up to the last horizon of 50 samples, between about 50/34 and 50/15 Lyapunov times. Forecasts made from measured windows are therefore useful over the whole range of horizons tested, while free runs, in which a model feeds on its own output, stay within 0.2 of the measured pulses, in units of the rescaled record, for 9 to

110 steps, depending on the model, as expected for a chaotic record and for models trained only on measured inputs.

The tools that learn a law, the Neural ODE, the physics-informed network and the hybrid model, have not been applied to the laser. Each of them needs the state in which the law is written, or a law of known form for the measured variables, and the record gives one variable, the intensity, proportional to the square of the field. The delay vectors recover the state only up to an unknown smooth change of coordinates, so a vector field learned on them would be a law in these coordinates, and its terms could not be compared directly with the Maxwell–Bloch equations. A hybrid model in the field, the polarisation and the inversion, with the decay rates and the pump as unknown parameters, would need a measurement of more than the intensity or a model of the measurement, the intensity as the square of the field, inside the loss. The laser record therefore marks the boundary between the two groups of models of these notes, the models that learn a forecast from a record alone and the models that learn a law from a measured state.

10 Known and learned parts of a model

A measured record is a projection onto one number per step of a motion in a state space that we do not see. A window of past values recovers the state, so a forecast map from the window to future values exists, and the Lyapunov time sets how far ahead any such map can reach. Forecasts are judged on a subsequent part of the record, by the capacity $C = r^2$ read together with a scatter plot, and against persistence.

The models of these notes differ in what they take as known and what they learn from the data, and Table 3 collects them. The first group learns from the record alone. Linear models fit a linear function of the window, which is the Koopman approximation with the raw past values as the dictionary, and they cannot follow a folded dynamics. NVAR adds products of past values chosen in advance, and neural networks learn their features, recurrent ones with a state that carries the past and gated cells, LSTM and GRU, whose memory path passes sensitivities back over many steps. The second group uses a measured state and physics. A Neural ODE learns the vector field of a law of known form, which can then be analysed like a law of motion, within the region of the data. A PINN uses a known law as part of its loss and suits unknown parameters and sparse data. A hybrid model keeps the trusted terms of a law and learns the missing one, which sparse regression turns into a formula that extrapolates as far as the true law has the form contained in the library.

model	known	learned	limit
AR, ARMA	the record	linear readout of past values	no fold, forecasts relax to the mean
Koopman, dynamic mode decomposition	the record, a dictionary	linear map on the dictionary	closure error of a finite dictionary
NVAR	the record	linear readout of products	features fixed, unstable free runs
MLP, RNN, LSTM, GRU	the record	features and readout	training through time, many parameters
Neural ODE	form $\dot{\mathbf{x}} = \mathbf{F}(\mathbf{x})$, measured state	vector field $\mathbf{F}_\theta$	reliable only where the data are
PINN	the whole law	solution, unknown parameters	slow for forward problems, long times
hybrid model	the law except one term	missing term, then a formula	needs a library that contains the law

Table 3: The forecasting models of these notes, by what each takes as known, what it learns and where it reaches its limit.

References

[1] F. Takens, Detecting strange attractors in turbulence, Lect. Notes Math. **898**, 366 (1981), doi:10.1007/BFb0091924.

[2] T. Sauer, J. A. Yorke and M. Casdagli, Embedology, J. Stat. Phys. **65**, 579 (1991), doi:10.1007/BF01053745.

[3] J. Dambre, D. Verstraeten, B. Schrauwen and S. Massar, Information processing capacity of dynamical systems, Sci. Rep. **2**, 514 (2012), doi:10.1038/srep00514.

[4] H. Kantz and T. Schreiber, *Nonlinear Time Series Analysis*, 2nd ed., Cambridge University Press (2003), doi:10.1017/CBO9780511755798.

[5] E. N. Lorenz, Deterministic nonperiodic flow, J. Atmos. Sci. **20**, 130 (1963), doi:10.1175/1520-0469(1963)020<0130:DNF>2.0.CO;2.

[6] M. C. Mackey and L. Glass, Oscillation and chaos in physiological control systems, Science **197**, 287 (1977), doi:10.1126/science.267326.

[7] M. T. Rosenstein, J. J. Collins and C. J. De Luca, A practical method for calculating largest Lyapunov exponents from small data sets, Physica D **65**, 117 (1993), doi:10.1016/0167-2789(93)90009-P.

[8] M. Sano and Y. Sawada, Measurement of the Lyapunov spectrum from a chaotic time series, Phys. Rev. Lett. **55**, 1082 (1985), doi:10.1103/PhysRevLett.55.1082.

[9] T. Schreiber and A. Schmitz, Improved surrogate data for nonlinearity tests, Phys. Rev. Lett. **77**, 635 (1996), doi:10.1103/PhysRevLett.77.635.

[10] P. Grassberger and I. Procaccia, Characterization of strange attractors, Phys. Rev. Lett. **50**, 346 (1983), doi:10.1103/PhysRevLett.50.346.

[11] H. Haken, Analogy between higher instabilities in fluids and lasers, Phys. Lett. A **53**, 77 (1975), doi:10.1016/0375-9601(75)90353-9.

[12] U. Hübner, N. B. Abraham and C. O. Weiss, Dimensions and entropies of chaotic intensity pulsations in a single-mode far-infrared NH_3 laser, Phys. Rev. A **40**, 6354 (1989), doi:10.1103/PhysRevA.40.6354.

[13] A. S. Weigend and N. A. Gershenfeld (eds.), *Time Series Prediction: Forecasting the Future and Understanding the Past*, Addison-Wesley (1994), reissued by Routledge (2018), doi:10.4324/9780429492648.

[14] A. M. Fraser and H. L. Swinney, Independent coordinates for strange attractors from mutual information, Phys. Rev. A **33**, 1134 (1986), doi:10.1103/PhysRevA.33.1134.

[15] M. B. Kennel, R. Brown and H. D. I. Abarbanel, Determining embedding dimension for phase-space reconstruction using a geometrical construction, Phys. Rev. A **45**, 3403 (1992), doi:10.1103/PhysRevA.45.3403.

[16] G. E. P. Box, G. M. Jenkins, G. C. Reinsel and G. M. Ljung, *Time Series Analysis: Forecasting and Control*, 5th ed., Wiley (2015).

[17] H. Akaike, A new look at the statistical model identification, IEEE Trans. Autom. Control **19**, 716 (1974), doi:10.1109/TAC.1974.1100705.

[18] A. E. Hoerl and R. W. Kennard, Ridge regression: biased estimation for nonorthogonal problems, Technometrics **12**, 55 (1970), doi:10.1080/00401706.1970.10488634.

[19] B. O. Koopman, Hamiltonian systems and transformation in Hilbert space, Proc. Natl. Acad. Sci. USA **17**, 315 (1931), doi:10.1073/pnas.17.5.315.

[20] P. J. Schmid, Dynamic mode decomposition of numerical and experimental data, J. Fluid Mech. **656**, 5 (2010), doi:10.1017/S0022112010001217.

[21] M. O. Williams, I. G. Kevrekidis and C. W. Rowley, A data-driven approximation of the Koopman operator: extending dynamic mode decomposition, J. Nonlinear Sci. **25**, 1307 (2015), doi:10.1007/s00332-015-9258-5.

[22] S. L. Brunton, M. Budišić, E. Kaiser and J. N. Kutz, Modern Koopman theory for dynamical systems, SIAM Rev. **64**, 229 (2022), doi:10.1137/21M1401243.

[23] A. Mauroy, I. Mezić and J. Moehlis, Isostables, isochrons, and Koopman spectrum for the action–angle representation of stable fixed point dynamics, Physica D **261**, 19 (2013), doi:10.1016/j.physd.2013.06.004.

[24] S. T. M. Dawson, M. S. Hemati, M. O. Williams and C. W. Rowley, Characterizing and correcting for the effect of sensor noise in the dynamic mode decomposition, Exp. Fluids **57**, 42 (2016), doi:10.1007/s00348-016-2127-7.

[25] D. J. Gauthier, E. Bollt, A. Griffith and W. A. S. Barbosa, Next generation reservoir computing, Nat. Commun. **12**, 5564 (2021), doi:10.1038/s41467-021-25801-2.

[26] J. L. Elman, Finding structure in time, Cogn. Sci. **14**, 179 (1990), doi:10.1207/s15516709cog1402_1.

[27] Y. Bengio, P. Simard and P. Frasconi, Learning long-term dependencies with gradient descent is difficult, IEEE Trans. Neural Netw. **5**, 157 (1994), doi:10.1109/72.279181.

[28] R. Pascanu, T. Mikolov and Y. Bengio, On the difficulty of training recurrent neural networks, ICML (2013), arXiv:1211.5063.

[29] A. M. Saxe, J. L. McClelland and S. Ganguli, Exact solutions to the nonlinear dynamics of learning in deep linear neural networks, arXiv:1312.6120.

[30] S. Hochreiter and J. Schmidhuber, Long short-term memory, Neural Comput. **9**, 1735 (1997), doi:10.1162/neco.1997.9.8.1735.

[31] F. A. Gers, J. Schmidhuber and F. Cummins, Learning to forget: continual prediction with LSTM, Neural Comput. **12**, 2451 (2000), doi:10.1162/089976600300015015.

[32] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, Learning phrase representations using RNN encoder–decoder for statistical machine translation, EMNLP (2014), doi:10.3115/v1/D14-1179.

[33] R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, Neural ordinary differential equations, NeurIPS (2018), arXiv:1806.07366.

[34] K. He, X. Zhang, S. Ren and J. Sun, Deep residual learning for image recognition, CVPR (2016), arXiv:1512.03385.

[35] P. Kidger, *On Neural Differential Equations*, PhD thesis, University of Oxford (2021), arXiv:2202.02435.

[36] Y. Rubanova, R. T. Q. Chen and D. Duvenaud, Latent ODEs for irregularly-sampled time series, NeurIPS (2019), arXiv:1907.03907.

[37] I. E. Lagaris, A. Likas and D. I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. **9**, 987 (1998), doi:10.1109/72.712178.

[38] M. Raissi, P. Perdikaris and G. E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. **378**, 686 (2019), doi:10.1016/j.jcp.2018.10.045.

[39] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang and L. Yang, Physics-informed machine learning, Nat. Rev. Phys. **3**, 422 (2021), doi:10.1038/s42254-021-00314-5.

[40] S. Wang, Y. Teng and P. Perdikaris, Understanding and mitigating gradient flow pathologies in physics-informed neural networks, SIAM J. Sci. Comput. **43**, A3055 (2021), doi:10.1137/20M1318043.

[41] J. A. Fleck, J. R. Morris and M. D. Feit, Time-dependent propagation of high energy laser beams through the atmosphere, Appl. Phys. **10**, 129 (1976), doi:10.1007/BF00896333.

[42] G. Strang, On the construction and comparison of difference schemes, SIAM J. Numer. Anal. **5**, 506 (1968), doi:10.1137/0705041.

[43] J. Crank and P. Nicolson, A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type, Math. Proc. Camb. Philos. Soc. **43**, 50 (1947), doi:10.1017/S0305004100023197.

[44] D. J. Griffiths and D. F. Schroeter, *Introduction to Quantum Mechanics*, 3rd ed., Cambridge University Press (2018), doi:10.1017/9781316995433.

[45] V. I. Arnold, *Mathematical Methods of Classical Mechanics*, 2nd ed., Graduate Texts in Mathematics **60**, Springer (1989), doi:10.1007/978-1-4757-2063-1.

[46] L. D. Landau and E. M. Lifshitz, *Quantum Mechanics: Non-Relativistic Theory*, 3rd ed., Pergamon (1977), doi:10.1016/C2013-0-02793-4.

[47] C. Rackauckas, Y. Ma, J. Martensen, C. Warner, K. Zubov, R. Supekar, D. Skinner, A. Ramadhan and A. Edelman, Universal differential equations for scientific machine learning, arXiv:2001.04385.

[48] S. L. Brunton, J. L. Proctor and J. N. Kutz, Discovering governing equations from data by sparse identification of nonlinear dynamical systems, Proc. Natl. Acad. Sci. USA **113**, 3932 (2016), doi:10.1073/pnas.1517384113.

[49] N. M. Mangan, S. L. Brunton, J. L. Proctor and J. N. Kutz, Inferring biological networks by sparse identification of nonlinear dynamics, IEEE Trans. Mol. Biol. Multi-Scale Commun. **2**, 52 (2016), doi:10.1109/TMBMC.2016.2633265.

[50] A. J. Lotka, Analytical note on certain rhythmic relations in organic systems, Proc. Natl. Acad. Sci. USA **6**, 410 (1920), doi:10.1073/pnas.6.7.410.

[51] V. Volterra, Fluctuations in the abundance of a species considered mathematically, Nature **118**, 558 (1926), doi:10.1038/118558a0.

[52] C. S. Holling, Some characteristics of simple types of predation and parasitism, Can. Entomol. **91**, 385 (1959), doi:10.4039/Ent91385-7.

[53] M. L. Rosenzweig and R. H. MacArthur, Graphical representation and stability conditions of predator-prey interactions, Am. Nat. **97**, 209 (1963), doi:10.1086/282272.

[54] J. L. Kaplan and J. A. Yorke, Chaotic behavior of multidimensional difference equations, in *Functional Differential Equations and Approximation of Fixed Points*, Lect. Notes Math. **730**, 204 (1979), doi:10.1007/BFb0064319.

[55] G. U. Yule, On a method of investigating periodicities in disturbed series, with special reference to Wolfer's sunspot numbers, Phil. Trans. R. Soc. Lond. A **226**, 267 (1927), doi:10.1098/rsta.1927.0007.

[56] G. Walker, On periodicity in series of related terms, Proc. R. Soc. Lond. A **131**, 518 (1931), doi:10.1098/rspa.1931.0069.

[57] I. Mezić, Spectral properties of dynamical systems, model reduction and decompositions, Nonlinear Dyn. **41**, 309 (2005), doi:10.1007/s11071-005-2824-x.

[58] D. P. Kingma and J. Ba, Adam: a method for stochastic optimization, ICLR (2015), arXiv:1412.6980.

[59] E. Hairer, S. P. Nørsett and G. Wanner, *Solving Ordinary Differential Equations I. Nonstiff Problems*, 2nd ed., Springer (1993), doi:10.1007/978-3-540-78862-1.

[60] D. C. Liu and J. Nocedal, On the limited memory BFGS method for large scale optimization, Math. Program. **45**, 503 (1989), doi:10.1007/BF01589116.

[61] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby and M. W. Mahoney, Characterizing possible failure modes in physics-informed neural networks, NeurIPS (2021), arXiv:2109.01050.